



como sabes soy unity developer.

Quiero crear un proyecto VR donde aparezca un personaje con el que el usuario pueda hablar. Este personaje esta conectado a una API con una IA por lo que se podrá conversar en tiempo real. La voz debe estar sincronizada con el avatar 3D via lipsync. Dime el stack tecnologico que necesito usar, versiones de Unity y plugins etc y toda la configuracion necesaria de VR.

Se va a ejecutar en meta quest 2 y 3 y es importante que vaya muy fluido y la comunicacion con este personaje sea fluida. No va a haber ningun tipo de exploracion ni interaccion en VR. Solo ponerse las gafas y siempre ver a este personaje delante con un fondo especifico.

Por otro lado crees que es viable el desarrollo en 2 semanas ?

Sí, es un proyecto viable para Quest 2/3 si lo planteas como una experiencia muy contenida: un solo entorno, un solo avatar, cámara fija frente al personaje y pipeline de voz/IA muy simple. En ese formato, el mayor riesgo no es el VR sino la latencia end-to-end entre STT/LLM/TTS/lipsync y la estabilidad del avatar en standalone.^{[1] [2] [3]}

Stack recomendado

Te recomendaría esta base técnica para 2026: Unity 6.1+ o 6 LTS, render pipeline URP, XR provider Unity OpenXR Plugin con soporte Meta, y Meta XR Core SDK para Quest. Meta recomienda Unity 6+ con OpenXR para SDKs recientes, y marca Oculus XR Plugin como legado para proyectos viejos.^{[2] [4] [5]}

Para lipsync, usaría Oculus LipSync solo como sistema de visemas en runtime, porque sigue siendo la vía oficial/documentada para generar los 15 visemas desde audio o micrófono. Eso sí: no lo mezclaría con el Spatializer antiguo, porque Meta indica que el Oculus Spatializer está EOL y debe sustituirse por Meta XR Audio SDK si necesitas audio espacial.^{[6] [7] [1]}

El stack concreto sería:

- Unity 6.1 o superior; si puedes, una rama LTS/estable de Unity 6 para reducir riesgo.^[2]
- URP Universal 3D template como base.^[2]
- OpenXR Plugin + OpenXR Meta package/features.^[2]
- Meta XR Core SDK.^{[4] [2]}
- Oculus LipSync para visemas del avatar.^{[1] [6]}
- Meta XR Audio SDK si quieres que la voz del personaje “salga” de su boca con espacialización correcta.^{[7] [8]}

- Backend externo con 3 servicios separados: STT streaming, LLM, TTS streaming. Esa separación te da más control de latencia que una integración demasiado “mágica”, especialmente en Quest standalone.^[3] ^[9]

Arquitectura útil

Como no habrá locomoción ni interacción VR compleja, no necesitas Interaction SDK completo ni XR Interaction Toolkit salvo para utilidades mínimas. Puedes montar una escena muy ligera con XR Origin/Camera Rig, un canvas diegético o world-space muy pequeño para estados (“escuchando”, “pensando”, “hablando”), y un avatar siempre a 1.5-2 metros frente al usuario.^[9] ^[2]

La arquitectura de conversación más robusta para fluidez sería:

1. Captura de micrófono en Quest.
2. Envío en streaming a STT.
3. En cuanto detectes final parcial o endpoint corto, lanzas request al LLM.
4. Pides TTS en streaming.
5. Reproduces chunks de audio en cuanto llegan.
6. Alimentas el lipsync con ese mismo audio reproducido, no con una pista distinta.^[6] ^[1]

La clave es evitar esperar a “tener toda la frase” para empezar a hablar; si haces TTS chunked/streaming, la percepción de fluidez mejora mucho aunque la IA tarde algo más en terminar la respuesta. Eso es más importante para UX que ganar 5 ms de render.^[3] ^[9]

Avatar y lipsync

Aquí hay una decisión crítica: usa un avatar cuyo facial rig ya venga preparado con blendshapes compatibles o fácilmente mapeables a los 15 visemas de OVR LipSync: si1, PP, FF, TH, DD, kk, CH, SS, nn, RR, aa, E, ih, oh, ou. Si eliges un personaje bonito pero sin buen rig facial, se te va la mitad del tiempo de producción en adaptar morph targets.^[1] ^[6]

Opciones prácticas:

Opción	Ventaja	Riesgo
Avatar humano stylized con blendshapes propios	Control visual total. ^[1]	Necesitas mapping fino de visemas y quizá dientes/lower face separados. ^[6]
Avatar framework que ya soporte OVR LipSync	Integración más rápida. ^[10]	Menos libertad visual o dependencia de asset/plugin externo. ^[10]
Meta Avatars	Pipeline Meta más integrado para Quest. ^[2]	Para un NPC con look específico puede limitar bastante el estilo. ^[2]

Para una demo en 2 semanas, yo evitaría full facial animation avanzada y me quedaría con:

- Jaw open/close.
- 15 visemas de OVR LipSync.

- Eye blink procedural.
- Idle breathing/head micro-motion.
- 2-3 expresiones simples por estado, por ejemplo neutral, happy, thinking.^[6] ^[1]

Configuración VR

Como tu app es “ponerse las gafas y ver siempre al personaje”, la configuración ideal es muy minimalista:

- Una sola escena principal.^[9]
- Usuario sentado o de pie, sin locomoción.^[2]
- Avatar anclado frente a la cámara a distancia cómoda.^[9]
- Fondo estático o muy barato de renderizar, preferiblemente skybox simple o geometry muy low poly.^[11] ^[9]
- Sin físicas complejas, sin sombras dinámicas costosas, sin postprocesado caro.^[11] ^[9]

En Unity/Quest dejaría esto como base:

- Build target: Meta Quest / Android.^[2]
- Scripting backend: IL2CPP.^[12] ^[13]
- Architecture: ARM64.^[13] ^[12]
- XR Plug-in Management: OpenXR en Android.^[2]
- OpenXR features Meta: activar Meta XR Feature, Meta XR Foveation y Meta XR Subsampled Layout.^[2]
- Ejecutar Project Setup Tool y aplicar/fijar todo para Android.^[2]

En render:

- URP con renderer mobile-friendly.^[11] ^[2]
- MSAA bajo o moderado; si vas sobrado, 2x o 4x, pero probar en Quest 2 real.^[11]
- Una sola luz principal baked o incluso unlit si el look lo permite.^[9]
- Sin realtime GI y casi sin sombras.^[9]
- Dynamic Resolution o Foveation si hiciera falta mantener frame-time estable.^[9] ^[2]

Rendimiento en Quest 2 y 3

Meta indica que las apps interactivas deben ir al menos a 72 FPS, mientras que 60 FPS puede valer para media apps. Como esto sigue siendo una app interactiva de conversación en VR, yo diseñaría para 72 FPS sólidos en Quest 2 y dejaría Quest 3 con margen extra.^[3]

Tus cuellos de botella probables no serán el escenario sino:

- Skinned mesh del avatar.
- Blendshapes de lipsync.

- Decodificación/reproducción de audio en streaming.
- Networking.
- Garbage generation en el pipeline de audio/texto.^[3] ^[9]

Por eso haría estas restricciones desde el día 1:

- Un solo avatar visible.^[9]
- Un único skinned mesh facial o como mucho cabeza+dientes; evita rigs muy fragmentados.^[6] ^[3]
- Texturas comprimidas y pocas variantes de material.^[9]
- Shader único/simple; evitar variants innecesarias.^[3]
- Nada de transparencia grande ni fondos caros.^[9]
- Pooling para buffers de audio y mensajes de red.^[3]

Para una app así, Quest 2 es tu baseline real; si corre fluido ahí, Quest 3 irá bien.^[7] ^[3]

Backend de voz e IA

Para conversación natural, el diseño correcto no es “Unity llama a una API monolítica y espera”, sino una tubería asíncrona:

- STT streaming con endpointing corto.
- LLM con respuestas breves por turno.
- TTS streaming que empiece a emitir audio antes de completar todo el texto.
- Opcional: cancelación de respuesta si el usuario vuelve a hablar.^[3] ^[9]

Recomendaciones de implementación:

- WebSocket o streaming HTTP para audio/texto parcial.
- Respuestas cortas del NPC, 1-3 frases.
- Límite de tokens duro.
- Prompt del personaje muy controlado.
- Cache de system prompt y contexto resumido para no disparar coste/latencia.^[3]

Si quieres una sensación “muy fluida”, el objetivo práctico debería ser algo como:

- Transcripción parcial visible o interna casi inmediata.
- Inicio de voz del NPC en ~1-2 segundos ideales tras final del usuario.
- Respuesta total corta, menos de 8-10 segundos.

No es una cifra oficial de Meta, pero sí una meta de UX razonable para que no se sienta torpe en headset.^[3] ^[9]

Plugins y paquetes

Yo intentaría quedarme con el mínimo número posible de dependencias:

- `com.unity.xr.openxr`.^[2]
- Unity OpenXR: Meta / OpenXR Meta package.^{[5] [2]}
- Meta XR Core SDK.^[2]
- Oculus LipSync.^[1]
- Meta XR Audio SDK si usas espacialización.^{[14] [7]}

Evitaría, salvo necesidad clara:

- XR Interaction Toolkit completo.
- Interaction SDK completo.
- Full-body tracking, hand tracking, passthrough, MR utilities.
- Plugins pesados de avatar conversacional “todo en uno”.^{[9] [2]}

Tu caso de uso es casi un “kiosk VR scene”, así que cada plugin extra añade riesgo y tiempo de integración sin darte valor real.^[9]

Plan de 2 semanas

Sí, en 2 semanas es viable para un vertical slice sólido, pero no para un producto pulido y robusto para store release.^{[3] [2]}

Lo que sí veo posible en 2 semanas:

- App standalone en Quest 2/3.
- Un personaje frente al usuario.
- Conversación real con IA.
- Voz TTS reproducida en tiempo real.
- Lipsync funcional.
- Escena visual sencilla pero presentable.
- Rendimiento estable a 72 FPS en Quest 2, si el avatar y shaders son modestos.^{[1] [3] [9]}

Lo que me parece arriesgado en 2 semanas:

- Avatar facial muy realista.
- Emociones faciales avanzadas.
- Barge-in impecable.
- Resiliencia total ante cortes de red.
- UX de audio muy refinada.
- Soporte para varios personajes, idiomas o memorias largas.^{[1] [3]}

Un plan realista sería:

- Días 1-2: proyecto Unity 6, OpenXR, Quest build, escena base, profiling inicial.^[2]
- Días 3-4: integración micrófono + STT + LLM.
- Días 5-6: TTS streaming + reproducción estable.
- Días 7-8: avatar + blendshapes + OVRlipSync.
- Días 9-10: UI mínima de estados + fondo + audio espacial.
- Días 11-12: optimización Quest 2.
- Días 13-14: bugs, latencia, recovery, packaging.^{[11] [3] [9]}

Recomendación final

Si quieres maximizar probabilidades en 2 semanas, haría este recorte de alcance:

- Unity 6.1+/6 LTS + URP + OpenXR + Meta XR Core SDK.^{[4] [2]}
- Un avatar con blendshapes ya listos para visemas.^{[6] [1]}
- Oculus LipSync para labios, Meta XR Audio SDK para voz espacial.^{[7] [1]}
- Escena única, sin locomoción, sin manos, sin interacción física.^[9]
- Target de 72 FPS en Quest 2.^[3]
- Backend con STT/LLM/TTS en streaming y respuestas cortas.^[3]

Mi estimación: sí, un MVP convincente en 2 semanas es alcanzable para alguien con tu perfil de Unity dev, pero solo si reutilizas un avatar ya rigged y evitas inventarte tooling propio para facial animation o networking. Un primer prototipo funcional podría salir en 5-7 días; lo que consume la segunda semana es bajar latencia, arreglar audio y estabilizar el build en dispositivo real.^{[1] [3] [9]}

¿Quieres que te lo convierta en un plan técnico aún más accionable, por ejemplo con una checklist exacta de Project Settings de Unity, paquetes a instalar y orden de implementación día por día?

✱

Unity VR AI Avatar for Meta Quest 2/3

Overview

This document defines a production-oriented setup and step-by-step development plan for a Unity VR application in which the user wears a Meta Quest 2 or 3 headset and immediately sees a conversational 3D character in front of them. The character is connected to an external AI API for real-time conversation, speaks back with generated voice, and animates through runtime lip sync. The project target is a highly polished standalone Quest build with a one-month development schedule starting today.^{[31] [32] [33]}

The technical direction in this document prioritizes low latency, stable frame time, reduced integration risk, and a small feature surface. Meta recommends the Unity OpenXR Plugin for

Quest on Unity 6+, Unity’s Meta-specific OpenXR package adds Quest-specific extensions, and Meta’s Quest performance guidance targets 72 FPS minimum for interactive applications. [\[33\]](#) [\[34\]](#) [\[31\]](#)

Product scope

The application scope is intentionally narrow:

- The user launches directly into a single VR scene.
- There is no locomotion, grabbing, hand interaction, room exploration, or world navigation.
- A single avatar is always positioned in front of the user.
- The environment is visually controlled and lightweight.
- The primary loop is: listen, think, speak, lip sync, return to listening. [\[32\]](#) [\[33\]](#)

That scope is a major advantage. It keeps the rendering budget, interaction complexity, QA matrix, and player-state management small enough that development effort can go into the parts that matter: latency, avatar quality, voice pipeline stability, and overall polish. [\[32\]](#) [\[33\]](#)

Recommended stack

Unity version

Use **Unity 6.1 or newer in a stable production branch**, ideally the most stable Unity 6 LTS-style release available in Unity Hub at project start. Meta’s current Unity Quest setup guidance recommends the Unity OpenXR Plugin on Unity 6 or later, and states that the recommended OpenXR Plugin version is 1.15.1 for Unity 6 with Meta XR SDKs v74 or later. [\[31\]](#)

Avoid starting this project on an experimental beta editor. A one-month timeline with strong polish requirements leaves little room for regression hunting across editor updates. The safest path is a stable Unity 6 release with known Android and XR package compatibility. [\[33\]](#) [\[31\]](#)

XR stack

Use this XR foundation:

Layer	Recommendation	Why
XR runtime integration	Unity OpenXR Plugin	Meta recommends it for Unity 6+ Quest development. [31]
Meta-specific extensions	OpenXR: Meta package	Adds Meta Quest-specific features beyond core OpenXR. [34]
Meta SDK	Meta XR Core SDK	Required for current Meta Unity workflows and tools. [31] [35]
Interaction framework	None by default	This project does not need locomotion or object interaction, so extra frameworks add risk and overhead. [33]

Do not add XR Interaction Toolkit or Meta Interaction SDK unless a real feature depends on them. For this product, those packages usually create unnecessary scene complexity, extra input actions, and more things to debug on device.^[33]

Rendering

Use URP from day one. Unity's Meta Quest package guidance explicitly points to creating a URP-based project for Quest workflows, and URP is the sensible middle ground between visual control and mobile VR performance.^[34]

Use a mobile-conscious renderer profile:

- One directional light at most.
- Prefer baked or unlit lighting where possible.
- Minimal post-processing.
- No screen-space effects unless profiling proves they are cheap enough on Quest 2.^[32] ^[33]

Audio and lip sync

Use Oculus LipSync for runtime viseme generation. Meta documents that the Unity plugin analyzes microphone or audio input streams and produces viseme values for facial animation.^[36]

Use Meta XR Audio SDK only if positional or spatial audio around the avatar is needed. If the character simply speaks from a fixed point in front of the user, this can improve presence, but it is secondary to the main goals of low latency and stable frame time.^[31]

Backend architecture

Use a modular voice stack instead of a monolithic black-box service:

- Speech-to-text streaming service.
- LLM text generation service.
- Text-to-speech streaming service.
- A lightweight backend gateway that hides provider-specific auth and request formats from the Quest client.

This architecture gives better control over latency, retries, buffering, logging, and swapping providers later. It also prevents shipping API secrets inside the standalone application.^[33]

Final project architecture

Client application layers

Structure the Unity project around clear subsystems:

1. **Bootstrap layer:** app launch, config load, scene start.
2. **XR rig layer:** camera, guardian-safe placement, seated/standing alignment.
3. **Avatar presentation layer:** mesh, animation controller, idle states, eye blink, lip sync bindings.
4. **Conversation layer:** microphone capture, voice activity detection or push-to-talk logic, speech state machine.
5. **Network layer:** websocket or streaming HTTP connections, retries, error handling, timeouts.
6. **Audio playback layer:** buffering, stream decoding, AudioSource scheduling, interruption support.
7. **UI/state feedback layer:** listening, processing, speaking, reconnecting, muted, network error.
8. **Telemetry/debug layer:** latency timings, frame time, session logs, build diagnostics.

This separation matters because the main production challenge is not raw feature count but making asynchronous systems cooperate without glitches.^{[32] [33]}

Conversation state machine

The app should be driven by a strict state machine. A recommended first version is:

- Booting
- Idle
- Listening
- UserSpeechDetected
- SendingSTT
- AwaitingLLM
- ReceivingTTS
- Speaking
- Interrupted
- Recovering
- Error

The state machine should be explicit in code and visible in an on-device debug overlay for development builds. This is the fastest way to diagnose user-reported bugs such as double playback, cut-off replies, microphone lockups, or avatar speaking while the backend is still streaming text.^[33]

Unity project setup

1. Create the project

Create a new Unity project using a **Universal 3D (URP)** template. Unity's Quest package guidance explicitly supports a URP project path for Meta Quest development.^[34]

Suggested project name:

VR_AI_Avatar_Quest

Suggested folder structure inside Assets/:

```
Assets/  
  _Project/  
    Art/  
      Avatars/  
      Materials/  
      Textures/  
      Environments/  
    Audio/  
      Music/  
      SFX/  
      Voices/  
    Code/  
      App/  
      XR/  
      Avatar/  
      Audio/  
      Network/  
      AI/  
      UI/  
      Debug/  
      Tools/  
    Data/  
      ScriptableObjects/  
      Config/  
    Prefabs/  
      Avatar/  
      UI/  
      Systems/  
    Scenes/  
      Boot/  
      Main/  
      Test/  
    Settings/  
      URP/  
      Input/  
      Quality/
```

Keep third-party packages isolated from `_Project/` as much as possible to reduce merge noise and make upgrades safer.

2. Enable the Quest platform

Meta's setup guide says that under Build Profiles, Meta Quest should be enabled, and if the profile is absent in earlier versions, Android should be used. On Unity 6.1+, Meta Quest should appear directly as a target profile in supported installs.^[35] ^[31]

Actions:

1. Open Build Profiles.
2. Select Meta Quest and enable the platform, or switch to it if already installed.^[35] ^[31]
3. If Unity prompts for `com.unity.xr.openxr`, install it.^[31]

3. Install XR packages

Install the following packages from Package Manager:

- XR Plug-in Management.
- Unity OpenXR Plugin.
- OpenXR: Meta package.
- Meta XR Core SDK.^[34] ^[31]

After installation, go to **Project Settings > XR Plug-in Management** and set **OpenXR** for the Android/Meta Quest target. Meta's documentation explicitly instructs selecting OpenXR in the Android Meta Quest settings tab.^[35] ^[31]

Then open the **OpenXR** settings for Android and enable the **Meta XR** feature group.^[35]

4. Run Meta project validation

Meta's documentation points to using the project setup workflow tools after adding Meta XR SDKs.^[35]

Actions:

1. Open the Meta project setup / validation tool from the top menu.
2. Apply all required fixes for Quest standalone.
3. Re-run the validation until the project is clean.

Do this again every time a major package changes. It prevents subtle Android and OpenXR configuration drift.

5. Android player settings

Set these Android-focused values early and never leave them to the final week:

- Scripting Backend: IL2CPP.
- Target Architecture: ARM64 only.
- Internet permission: enabled if required by backend communication.

- Microphone permission: required for speech input.
- API level: use the Meta-recommended modern Android baseline supported by the installed SDK packages.
- Strip unused engine code: enabled after testing plugin compatibility.

The exact Android API number can be adjusted to the current Meta and Unity package requirements at setup time, but IL2CPP and ARM64 should be treated as non-negotiable for production Quest deployment.^[31]

6. Scene bootstrap

Create two scenes:

- `Boot.unity`
- `MainConversation.unity`

`Boot` should do only the following:

- Initialize config.
- Initialize services and logging.
- Validate permissions and connectivity.
- Load `MainConversation`.

This keeps startup deterministic and prevents the main scene from becoming a tangled initialization script.

7. XR rig setup

Because the experience is static, use the smallest XR rig possible. The app needs head tracking, recentering support, and safe placement of the avatar relative to the user, but not controller gameplay systems.^[33]

Rig rules:

- One camera rig.
- No teleportation.
- No locomotion providers.
- No grab interactors.
- Optional controller tracking only if needed for a future hidden debug menu.
- Recenter support on scene start and from a hidden debug input.

8. Avatar setup

The avatar is the hero asset, so choose it before backend integration starts. Oculus LipSync drives blendshape-based visemes, and Meta's sample guidance states that `OVRipSyncContextMorphTarget` requires a `SkinnedMeshRenderer` with the full viseme blend targets available.^[37]

Required avatar checklist:

- Head mesh with stable blendshape support.
- Clear viseme mapping plan.
- Acceptable Quest polycount for head/body.
- Mobile-friendly materials and shaders.
- Separate lower teeth mesh only if the avatar system requires it.^{[38] [37]}

If the avatar uses a custom facial rig, create a viseme mapping document immediately. Meta's sample references the 15-viseme order used by the system: `sil`, `PP`, `FF`, `TH`, `DD`, `kk`, `CH`, `SS`, `nn`, `RR`, `aa`, `E`, `IH`, `oh`, `ou`.^[38]

9. Lip sync integration

Meta's sample setup says that a scene using LipSync must include `OVRipSync`, and that `OVRipSyncContext` is added per controlled audio source. For morph targets, `OVRipSyncContextMorphTarget` bridges viseme output to the target mesh blendshapes.^[37]

Implementation plan:

1. Add the core `OVRipSync` interface prefab or equivalent setup to the scene.^[37]
2. Add one `AudioSource` for the avatar voice output.
3. Add `OVRipSyncContext` linked to that `AudioSource`.^{[38] [37]}
4. Add `OVRipSyncContextMorphTarget` to the avatar head mesh.^[37]
5. If necessary, add a second morph target component for lower teeth.^[38]
6. Configure all 15 viseme indices and test each one manually.
7. Verify smoothing so the mouth animation does not chatter during streaming playback.

Do not wait for the full networking stack to test this. Use pre-recorded clips on day one of lip sync implementation.

Performance baseline

Frame target

Meta's Quest performance guidance states that interactive applications must achieve a minimum of 72 FPS, while 60 FPS may be acceptable for media apps. The basic optimization workflow expresses this as a 13.7 ms frame budget for 72 FPS.^{[32] [33]}

For this project, the true target should be:

- Quest 2: stable 72 FPS in the main conversation flow.
- Quest 3: equal or better, with headroom for higher quality if needed.

Quest 2 should be treated as the baseline device. If the app is smooth there, Quest 3 will usually be easy to support.^[32] ^[33]

Geometry and draw-call discipline

Meta's performance analysis guidance cites rough Quest targets of around 500–1,000 draw calls per frame and 1-2 million triangles or vertices per frame.^[33]

This project should stay well under those upper bounds. A single-avatar conversational scene can and should be much cheaper than a typical VR game. A healthy target is to keep the scene visually elegant but technically restrained:

- One hero avatar.
- One lightweight environment shell or backdrop.
- Minimal transparent materials.
- Minimal dynamic lights.
- Extremely limited VFX.

Rendering budget rules

Set these production rules at the beginning:

- No real-time shadows unless profiling on Quest 2 proves they are worth it.
- No screen-space ambient occlusion.
- No real-time reflections.
- No expensive post-process stack by default.
- One-pass simple materials wherever possible.
- Avoid multiple skinned meshes when one can do the job.

Profiling workflow

Meta's optimization guides emphasize profiling and performance analysis as a normal part of Quest development rather than a last-step cleanup activity.^[32] ^[33]

Adopt this rule:

- Every Friday is performance lock review day.
- Build to Quest 2 and test in-device.
- Measure CPU frame time, GPU frame time, GC allocations, spikes during streaming, and audio glitches.

- Track a simple spreadsheet with key metrics and regressions.

Network and backend integration

Communication pattern

For a polished experience, avoid a naive request-response loop that waits for the user to finish speaking, then blocks on STT, then blocks on the LLM, then blocks on final TTS. That interaction feels slow inside a headset.^[33]

Instead, use a progressive pipeline:

1. Capture microphone input continuously or during press-to-speak.
2. Stream audio chunks to STT.
3. Use short endpointing or VAD to decide when the user turn ends.
4. Send text to the LLM immediately.
5. Start TTS generation as soon as text becomes available.
6. Start playback as soon as a safe initial audio buffer is ready.
7. Drive lip sync from the actual audio being played.

This reduces perceived latency and keeps the animation, speech, and voice output aligned.

Client-side service interfaces

Create interfaces such as:

```
public interface ISpeechToTextService {}
public interface IConversationService {}
public interface ITextToSpeechService {}
public interface IStreamingAudioPlayer {}
public interface IConversationTelemetry {}
```

The exact methods can vary, but the interfaces should isolate provider-specific details and enable mock implementations for offline testing.

Error handling rules

Define these failure behaviors before coding the full path:

- If STT fails, show a quick retry state and return to listening.
- If LLM fails, play a short fallback response and log the issue.
- If TTS fails, optionally display text and suppress mouth motion.
- If the stream cuts out mid-response, stop playback cleanly and transition back to idle or reconnecting.

- If backend latency exceeds a threshold, reflect that in the UI rather than appearing frozen.

Audio pipeline

Microphone capture

Build microphone capture as a separate subsystem, not inside the NPC controller. This makes it easier to test permissions, echo issues, gain tuning, and buffer sizing independently.

Checklist:

- Request microphone permission on first start.
- Confirm sample rate and backend format expectations.
- Add a debug waveform meter or RMS indicator in development builds.
- Prevent overlapping recording sessions.
- Add optional local monitoring only for debug builds.

TTS playback buffering

Most quality problems in conversational XR are not caused by the model but by playback stutter. A smooth streaming TTS pipeline needs a buffered playback strategy.

Recommended rules:

- Do not start playback on the very first tiny chunk.
- Buffer enough audio to survive minor network jitter.
- Keep total startup delay low but predictable.
- Support interruption and cancelation cleanly.
- Drive lip sync from the post-buffer playback stream, not from raw network chunk arrival.

Spatial audio decision

If the character is always directly ahead, full advanced spatial processing is optional. However, subtle localization from the avatar's mouth position can improve presence. Use Meta XR Audio SDK only after the base voice and lip sync flow is stable.^[31]

Scene and UX design

Visual layout

Because there is no navigation, the scene must feel intentionally composed rather than empty. The recommended composition is:

- Avatar centered slightly below eye line.
- Background or environment with depth cues but low rendering cost.

- Soft focal lighting on the avatar.
- Small world-space status UI outside the main facial focus area.
- Optional floor or pedestal to ground the character visually.

The user should understand the experience within two seconds of putting on the headset.

UX states

At minimum, the app needs visible feedback for these states:

State	User feedback
Booting	Brief loading or fade-in
Ready	Calm idle pose and subtle ambient motion
Listening	Visual cue such as ring, mic icon, or caption state
Thinking	Reduced idle, thoughtful expression, or subtle indicator
Speaking	Mouth animation, voice playback, optional caption line
Error	Short readable message and auto-recover where possible

Because the user is trapped in a headset, hidden states feel broken. Clear state transitions are part of perceived polish.

Week-by-week production plan

Week 1: Foundation and vertical slice skeleton

The objective for week 1 is to get the entire stack standing up in ugly-but-working form. By the end of week 1, the project should already run on Quest 2 with an avatar in front of the player, microphone permission working, backend connectivity established, and at least a fake or partial response loop.^[31] ^[33]

Day 1

- Install Unity 6 stable branch.
- Create URP project.^[34]
- Enable Meta Quest build profile or Android Meta Quest target.^[35] ^[31]
- Install XR Plug-in Management, Unity OpenXR Plugin, OpenXR: Meta, Meta XR Core SDK.^[34] ^[31]
- Enable OpenXR on Android/Meta Quest.^[31]
- Enable Meta XR feature group.^[35]
- Run Meta project setup validation and fix all issues.^[35]
- Create Boot and MainConversation scenes.
- Make a first APK build to headset as soon as possible.

Day 2

- Set up XR rig.
- Add camera placement and recenter logic.
- Create a placeholder environment and placeholder avatar dummy.
- Create world-space debug/status UI.
- Add app state machine shell.
- Verify stable boot into VR on Quest 2.

Day 3

- Implement configuration loading and environment-specific endpoint config.
- Create network abstraction interfaces.
- Implement a temporary mocked STT/LLM/TTS flow.
- Add timestamps for major events: recording start, STT first partial, LLM start, TTS first byte, playback start.

Day 4

- Implement real microphone capture.
- Handle Android microphone permission.
- Send test payloads to backend.
- Display transcript debug text in development builds.
- Add error messages for no network, no permission, or empty audio.

Day 5

- Integrate real STT provider.
- Confirm first live end-to-end speech-to-text result from Quest hardware.
- Measure average turn latency.
- Implement endpointing/VAD strategy.

Day 6

- Integrate LLM provider.
- Keep prompt and response size short.
- Add persona/system prompt config outside hardcoded scripts.
- Handle timeout and retry behavior.

Day 7

- Integrate first TTS path.
- Play the generated voice through a placeholder speaker object.
- Confirm streaming or pseudo-streaming playback path.
- End week with a complete rough conversation loop, even if the avatar mouth is not yet animated.

Week 2: Avatar, lip sync, and believable presentation

The objective for week 2 is to make the experience look and feel like a real product instead of a technical demo.^[36] ^[37]

Day 8

- Import final or near-final avatar.
- Verify mesh quality and Quest suitability.
- Prepare idle animation set.
- Set up animator layers for idle, listening, speaking, and thinking.

Day 9

- Import Oculus LipSync package.^[36]
- Add OVRLipSync scene object or prefab.^[37]
- Create voice AudioSource on the avatar.
- Add OVRLipSyncContext to match the playback source.^[37]

Day 10

- Add OVRLipSyncContextMorphTarget to head mesh.^[37]
- Map all 15 visemes.^[38] ^[37]
- Add lower-teeth mapping if required.^[38]
- Create an editor test scene with manual viseme sliders for validation.

Day 11

- Test with prerecorded clips.
- Smooth blendshape transitions.
- Fix broken visemes, extreme mouth shapes, and clipping.
- Add jaw and blink support if needed.

Day 12

- Connect live TTS playback to lip sync.
- Verify sync under chunked audio playback.
- Fix start-of-sentence mouth popping.
- Add a short pre-roll buffer if needed.

Day 13

- Improve character presence: idle breathing, slight head motion, eye blink, micro-saccades if lightweight enough.
- Ensure the character does not move so much that it feels uncanny.
- Lock animation update costs to something Quest-friendly.

Day 14

- Improve scene composition, background, avatar lighting, and world-space UI placement.
- Run on Quest 2 and Quest 3.
- Capture first near-final internal milestone video.

Week 3: Latency reduction, reliability, and Quest optimization

The objective for week 3 is to make the app feel fast and stable enough that it can survive repeated test sessions. [\[32\]](#) [\[33\]](#)

Day 15

- Profile Quest 2 frame time in the main loop.
- Measure CPU, GPU, and GC allocations during idle, listening, and speaking. [\[33\]](#)
- Identify spikes when starting recording, sending audio, starting TTS playback, and updating blendshapes.

Day 16

- Reduce material count and overdraw.
- Simplify environment.
- Remove or downgrade unnecessary lighting and shadows.
- Merge or simplify meshes where useful.

Day 17

- Optimize audio buffering strategy.
- Reduce playback stutter.
- Improve stream startup threshold.
- Add interruption support if the user speaks over the NPC.

Day 18

- Optimize networking.
- Tune chunk sizes and backpressure handling.
- Add reconnect logic.
- Add better timeout messaging.

Day 19

- Optimize avatar update path.
- Avoid per-frame allocations in facial animation scripts.
- Profile blendshape writes and animator costs.
- Disable anything not visible or not useful.

Day 20

- Add optional subtitles for debug and accessibility.
- Improve UX state transitions.
- Add short fades instead of hard cuts between some states.
- Improve perceived responsiveness, even before raw latency is fully minimized.

Day 21

- End-to-end regression test on Quest 2.
- Run a long-form soak session.
- Validate memory growth, audio drift, and reconnection behavior.

Week 4: Polish, QA, and release candidate

The objective for week 4 is to remove rough edges and ship something that feels premium.^[33]

Day 22

- Freeze features.
- Create a bug triage board with severity labels.
- Separate must-fix issues from nice-to-have polish.

Day 23

- Refine UI and visual states.
- Improve loading flow.
- Clean branding, app name, icon, and splash behavior.

Day 24

- Improve fallback behaviors.
- Add graceful handling for no mic permission, poor network, backend unavailable, and partial response interruption.

Day 25

- Conduct focused lip sync polish pass.
- Tune smoothing, expression blending, and voice volume normalization.
- Ensure the character sounds and looks consistent across multiple response lengths.

Day 26

- Conduct performance lock pass.
- Final Quest 2 profiling.
- Verify that the app remains inside the 13.7 ms frame budget target often enough to hold 72 FPS in the core flow.^[32] ^[33]

Day 27

- Full QA pass on Quest 2.
- Full QA pass on Quest 3.
- Verify startup, repeated conversations, long sessions, headset sleep/resume, and network loss recovery.

Day 28

- Fix blocker bugs only.
- No new systems.
- Rebuild and retest.

Day 29

- Prepare release candidate build.
- Capture demo footage.
- Archive project settings and package versions.
- Export final config snapshot.

Day 30

- Final regression test.
- Lock release candidate.
- Tag repository and create deployment notes.

Detailed implementation order

The correct implementation order is not “finish all VR, then all backend, then all avatar.” That sequencing often hides cross-system problems too late. The recommended order is:

1. Get a blank Quest app launching.
2. Add static scene and XR camera.
3. Add mocked conversation state machine.
4. Add microphone capture.
5. Add STT.
6. Add LLM.
7. Add TTS playback.
8. Add avatar.
9. Add lip sync with prerecorded clips.
10. Connect live TTS to lip sync.
11. Optimize latency.
12. Optimize rendering.
13. Polish UX.
14. Run repeated on-device QA.

This order ensures that the product’s true risk areas are tested early rather than disguised by pretty visuals.

Technical recommendations that will matter most

Keep the scene tiny

A single-character VR app does not need game-like scene complexity. Every unnecessary shader, light, material variant, and environment prop reduces the headroom available for audio, network jitter, and facial animation.^[33]

Never trust Editor performance

Quest performance decisions must be made from headset profiling, not from Play Mode assumptions. Meta’s optimization guidance centers on in-device profiling because frame behavior on mobile standalone hardware differs substantially from desktop execution.^{[32] [33]}

Treat audio as a first-class system

Users forgive a simple room faster than they forgive broken voice playback. If the character responds quickly, sounds clean, and animates believably, the product will feel advanced even with visually modest surroundings.

Build debug tooling early

Add an internal debug overlay that can show:

- Current app state.
- Last STT text.
- Latency timings.
- Buffer fill state.
- Current FPS.
- CPU and GPU frame time if available.
- Active backend endpoint.

That overlay will save more time in one month than almost any “clean architecture” refactor.

Risks and mitigation

Risk	Why it matters	Mitigation
Avatar lacks proper viseme blendshapes	Lip sync quality collapses late in production	Lock the avatar choice in week 1 and validate the 15-viseme mapping immediately. [37] [38]
TTS startup feels too slow	Perceived intelligence and polish drop	Use streaming TTS, short responses, and startup buffering tuned on device.
Quest 2 frame drops	Presence breaks and comfort suffers	Treat Quest 2 as baseline and optimize weekly toward 72 FPS. [33] [32]
Too many frameworks	Integration and debugging time expands	Use only OpenXR, Meta Quest packages, and the minimum lip sync/audio packages needed. [31] [34]
Network instability	The app can look frozen or broken	Add explicit reconnect, timeouts, and user-visible state feedback.
Final-week optimization panic	Quality drops under deadline pressure	Profile from week 1 and keep a standing performance checklist. [33]

Definition of done

The project should be considered ready only when all of the following are true:

- The app launches directly into the conversation scene on Quest 2 and Quest 3.
- The avatar appears stably positioned and visually grounded.
- Microphone capture works consistently on device.
- Speech-to-text, LLM, and TTS all function through production endpoints.

- Voice playback starts quickly and does not stutter in normal network conditions.
- Lip sync follows the spoken output believably.
- The experience remains smooth on Quest 2 at the required interactive frame target.^[32] ^[33]
- Errors are surfaced clearly and recover gracefully.
- A 20–30 minute test session can run without escalating instability.

Recommended first-day checklist

The first day should end with these exact outcomes:

- Unity project created in URP.^[34]
- Meta Quest target enabled.^[31] ^[35]
- OpenXR enabled for Android/Meta Quest.^[31]
- Meta XR feature group enabled.^[35]
- Meta XR Core SDK installed.^[31]
- App builds and launches on Quest 2.
- Boot scene transitions into MainConversation.
- Placeholder environment visible in headset.
- A tracked issue list exists for setup friction.

If those are complete on day 1, the month is on good rails.

Recommended second-day checklist

The second day should end with:

- XR rig stable on headset.
- Avatar placeholder or final candidate imported.
- Debug UI visible in world space.
- App state machine functioning.
- Recenter and placement logic working.
- Git repository structure clean and package versions committed.

Final recommendation

This project is highly achievable in one month because the user experience is tightly constrained, but that only remains true if scope discipline is maintained. The technical path should stay focused on Unity 6, URP, OpenXR, Meta-specific OpenXR support, a single optimized avatar, Oculus LipSync for visemes, and a carefully buffered streaming voice pipeline.^[36] ^[34] ^[31]

The most important strategic decision is to treat latency, stability, and avatar believability as the real product. Everything else is support structure. If the voice loop is fast, the animation

feels natural, and Quest 2 holds the frame budget, the experience will feel premium even with a minimal scene.^[32] ^[33]

✱

Plan MVP rápido en Unity VR para Quest 2/3 con avatar conversacional

Objetivo del documento

Este documento define un plan de desarrollo muy orientado a MVP para validar en pocos días la viabilidad de un producto VR donde el usuario se pone las gafas, ve un avatar 3D delante y puede hablar con él en tiempo real. El objetivo principal no es aún construir la versión final, sino comprobar rápido tres cosas: que el backend responde con una experiencia convincente, que el flujo voz-IA-voz funciona con una latencia aceptable, y que el avatar puede acompañar esa respuesta con lip sync básico.^[46] ^[47]

La prioridad aquí es reducir riesgo y tiempo de implementación. Por eso el plan asume el uso de placeholders: un avatar 3D cualquiera descargado de internet, la API de Perplexity como LLM provisional, una escena mínima, y un workflow que permita probar la conversación desde el Editor de Unity sin tener que hacer build continuo a Quest usando Meta Quest Link/OpenXR para iterar más rápido.^[48] ^[49] ^[50]

Enfoque estratégico

La estrategia correcta para este caso no es construir el producto final desde el principio. Lo más eficiente es hacer primero un prototipo funcional muy pequeño que valide el núcleo del producto: escuchar al usuario, enviar audio o texto al backend, generar respuesta, reproducirla, y mover la boca del avatar de forma creíble mientras habla.^[47] ^[51]

Eso significa posponer todo lo que no ayude a validar la propuesta:

- Nada de exploración VR.
- Nada de interacción física.
- Nada de varios personajes.
- Nada de entorno complejo.
- Nada de pipeline final de avatar o arte.
- Nada de infraestructura backend definitiva si un gateway simple o una API placeholder sirve para probar el flujo.

El MVP debe responder a una sola pregunta: ¿esta experiencia resulta suficientemente buena como para justificar la versión final?

Stack recomendado para el MVP

Editor y proyecto base

Usar Unity 6.1 o superior con plantilla Universal 3D (URP). Meta y Unity documentan el flujo actual de Quest sobre OpenXR en Unity 6+, y Unity 6.1 introdujo el build platform de Meta Quest con integración directa del paquete OpenXR: Meta. [\[49\]](#) [\[50\]](#)

Base técnica recomendada:

Capa	Recomendación MVP	Motivo
Unity	Unity 6.1+ estable	Menos fricción con Quest/OpenXR actual. [50] [49]
Render	URP	Buen equilibrio entre simplicidad y rendimiento. [50]
XR	XR Plug-in Management + OpenXR	Es la vía recomendada por Meta para Quest. [46] [49]
Meta	OpenXR: Meta + Meta Quest feature group	Añade soporte específico de Quest. [52] [49]
SDK Meta	Meta XR Core SDK	Base estándar en workflows modernos de Meta. [46]
Lip sync	Oculus LipSync	Solución oficial y rápida para visemas en Unity. [47] [51]
LLM temporal	API de Perplexity	Permite validar backend conversacional sin montar el sistema final.
Avatar temporal	Cualquier avatar con blendshapes o sample compatible	Reduce tiempo de arte y rigging. [53]

Qué evitar en el MVP

No añadir al principio:

- XR Interaction Toolkit.
- Meta Interaction SDK completo.
- Hand tracking.
- Passthrough.
- MR.
- Sistema de navegación.
- Animación facial avanzada fuera del lipsync.
- Persistencia de memoria conversacional compleja.

Cada una de esas cosas aumenta tiempo de setup, superficie de bugs y coste de QA sin aportar valor a la validación temprana del producto. [\[54\]](#) [\[49\]](#)

Requisito clave: probar desde el Editor

Esto sí es totalmente viable. Unity documenta que Meta Quest Link permite ejecutar el proyecto directamente desde el Editor en el visor sin necesidad de hacer build y deploy cada vez. La configuración requiere activar OpenXR también en la pestaña Windows, Mac, Linux, habilitar la feature group de Meta Quest y usar Meta Quest Link en el PC. ^[46] ^[48]

Configuración para poder dar Play desde Unity

1. Instalar y configurar Meta Quest Link en el PC. ^[48]
2. Conectar Quest 2 o 3 por cable o usar Link/Air Link si la estabilidad es suficiente. ^[48]
3. En Unity ir a Edit > Project Settings > XR Plug-in Management. ^[46] ^[48]
4. En la pestaña Windows, Mac, Linux, activar OpenXR y desactivar XR Simulation si estuviera activado. ^[48]
5. En XR Plug-in Management > OpenXR, añadir el perfil Oculus Touch Controller Profile si hace falta para compatibilidad de input. ^[48]
6. En OpenXR Feature Groups, activar Meta Quest también en entorno de escritorio/editor cuando proceda. ^[48]
7. En la pestaña Android / Meta Quest, activar igualmente OpenXR y la Meta Quest feature group para cuando sí se haga build a standalone. ^[49] ^[46]

Con esto el ciclo de iteración recomendado será:

- Para lógica de conversación, UI, avatar, audio y lipsync: probar desde el Editor con Quest Link. ^[48]
- Para rendimiento real, permisos Android, micrófono en standalone y validación final: hacer build puntual a Quest. ^[54] ^[46]

Importante: probar desde el Editor acelera muchísimo el desarrollo del MVP, pero no sustituye la validación real en Quest standalone porque la ejecución en PC no representa el presupuesto de CPU/GPU del dispositivo. ^[54] ^[48]

Alcance exacto del MVP de 2-3 días

El MVP no debe intentar ser un producto completo. El alcance ideal es este:

- Una sola escena VR.
- Un avatar delante del usuario.
- Fondo simple o entorno mínimo.
- El usuario habla por micrófono.
- El audio o texto llega a un backend o gateway sencillo.
- El sistema consulta la API de Perplexity como placeholder de LLM.
- El sistema devuelve una respuesta breve.
- Esa respuesta se convierte en voz con el TTS placeholder elegido.

- El avatar reproduce la voz y mueve la boca con lip sync.
- Todo puede probarse en Play Mode con Quest Link.^[51] ^[47] ^[48]

Si eso funciona, ya tendrás la validación importante: no la del producto final, sino la de la experiencia base.

Arquitectura mínima recomendada

El error típico en un prototipo rápido es mezclar todo en un solo script. Para ganar velocidad sin generar caos, conviene una arquitectura mínima pero limpia.

Subsistemas del MVP

Separar al menos en estos módulos:

- `AppBootstrap`: arranque de servicios y carga de escena.
- `XRSceneSetup`: rig XR, posición del avatar y elementos de escena.
- `ConversationController`: máquina de estados de la conversación.
- `MicrophoneCapture`: grabación o captura continua.
- `PerplexityClient`: llamadas HTTP al backend o a un gateway que use Perplexity.
- `TTSClient`: conversión de texto a voz.
- `StreamingAudioPlayer` O `BufferedAudioPlayer`: reproducción de respuesta.
- `AvatarSpeechController`: conexión entre `AudioSource` y `Oculus LipSync`.
- `StatusUI`: feedback visual de estado.
- `DebugOverlay`: latencia, transcript y errores en development builds.

Con eso el MVP sigue siendo rápido de construir, pero no se vuelve inmantenible en cuanto empieces a sustituir placeholders.

Máquina de estados mínima

Definir desde el principio estados explícitos:

- Idle
- Listening
- Processing
- Speaking
- Error

Eso basta para el MVP. Si más adelante necesitas interrupciones, streaming parcial o cancelación, ya podrás extenderlo.

Setup del proyecto paso a paso

Paso 1: Crear proyecto limpio

1. Abrir Unity Hub.
2. Crear proyecto nuevo con template Universal 3D.^[50]
3. Nombre sugerido: VR_Avatar_MVP.
4. Confirmar que la versión es Unity 6.1 o superior.^[50] ^[49]

Paso 2: Activar Meta Quest y OpenXR

1. Ir a File > Build Profiles.
2. Seleccionar Meta Quest y hacer Switch Platform.^[49]
3. Si Unity pide instalar el paquete OpenXR: Meta, aceptar.^[49]
4. Ir a Edit > Project Settings > XR Plug-in Management.
5. En la pestaña Android / Meta Quest, activar OpenXR.^[46] ^[49]
6. En XR Plug-in Management > OpenXR, activar el grupo Meta Quest.^[49]
7. En la pestaña Windows, Mac, Linux, activar también OpenXR para pruebas en editor con Link.^[46] ^[48]

Paso 3: Configurar Quest Link para iterar en editor

1. Instalar la app de Meta Quest Link en el PC.^[48]
2. Conectar las gafas y lanzar Link desde el visor.^[48]
3. Asegurarse de que el runtime OpenXR correcto esté activo en el entorno Meta Link, tal como indica la guía de Link de Unity.^[48]
4. En Unity, pulsar Play con la escena abierta.
5. Confirmar que la vista entra en VR dentro del visor sin necesidad de build manual.^[48]

Paso 4: Instalar paquetes mínimos

Instalar solo esto al inicio:

- XR Plug-in Management.^[46]
- OpenXR Plugin.^[46]
- Unity OpenXR: Meta / Meta Quest package.^[49]
- Meta XR Core SDK.^[46]
- Oculus LipSync.^[47]

Nada más hasta que el flujo base esté funcionando.

Paso 5: Estructura de carpetas

Crear algo así dentro de Assets/_Project/:

```
Assets/_Project/  
  Art/  
    Avatars/  
    Environment/  
  Audio/  
    TTS/  
    Temp/  
  Code/  
    App/  
    XR/  
    AI/  
    Audio/  
    Avatar/  
    UI/  
    Debug/  
  Prefabs/  
  Scenes/  
  Settings/  
  Data/
```

Paso 6: Crear escenas

Crear dos escenas:

- Boot
- MainMVP

Boot solo inicializa servicios y carga MainMVP. Esto te permitirá luego crecer sin romper la escena principal.

Paso 7: Montar la escena VR mínima

En MainMVP:

- Añadir el XR rig mínimo.
- Poner un fondo simple, por ejemplo un plano curvado, una pared o un pequeño set unlit.
- Colocar al avatar a unos 1.5-2 metros delante de la cámara.
- Añadir una luz direccional suave o iluminación muy simple.
- Añadir un canvas world-space con un texto pequeño de estado.

No hace falta una escena bonita todavía. Hace falta una escena que comunique bien la prueba.

Avatar placeholder recomendado

Para el MVP, el criterio no debe ser “el avatar más bonito”, sino “el avatar que antes se deje conectar a lip sync”. Oculus LipSync funciona bien cuando el personaje dispone de blendshapes compatibles o al menos adaptables al sistema de visemas, y la documentación de sample y proyectos de terceros deja claro que el setup gira alrededor de `AudioSource`, `OVRipSyncContext` y `OVRipSyncContextMorphTarget` sobre un `SkinnedMeshRenderer`.^[53] ^[51] ^[47]

Recomendación práctica

Buscar una de estas opciones:

- Un sample de avatar ya usado con Oculus LipSync.^[53]
- Un avatar de Asset Store o internet con blendshapes faciales claros.
- Un personaje humanoide sencillo con `SkinnedMeshRenderer` en cara y morph targets básicos.

Si el avatar no tiene blendshapes útiles, el MVP se ralentiza muchísimo. Para validar backend, es preferible un avatar mediocre pero técnicamente compatible.

Lip sync paso a paso para el MVP

Meta documenta que Oculus LipSync analiza un audio de micrófono o archivo y genera valores de visemas para animar el avatar.^[47]

Configuración inicial

1. Importar el paquete Oculus LipSync.^[47]
2. Abrir la escena demo/sample si viene incluida para entender el flujo.^[51]
3. Añadir el objeto base `OVRipSync` a la escena o el setup equivalente del paquete.^[55]
4. Añadir un `AudioSource` al avatar o al objeto que reproduzca la voz.^[53]
5. Añadir un `OVRipSyncContext` asociado a ese `AudioSource`.^[53]
6. Añadir `OVRipSyncContextMorphTarget` al mesh facial.^[53]
7. Mapear visemas si el avatar no viene preparado.

Validación rápida

Antes de integrar backend:

- Reproducir un audio clip de voz local.
- Confirmar que el avatar mueve la boca.
- Ajustar smoothing.
- Revisar clipping raro, exageración o falta de movimiento.

Meta también documenta una escena sample que permite alternar entre entrada de micrófono y audio clip, lo que es útil para validar la integración rápidamente sin depender aún

del backend completo.^[51]

Backend placeholder con Perplexity

Para este MVP, la mejor decisión es **no conectar aún el backend final**, sino crear un cliente simple o un minigateway que reciba el texto del usuario y consulte la API de Perplexity como placeholder del modelo. La idea no es optimizar coste ni control fino del prompt, sino validar si la experiencia tiene sentido y si la velocidad general es aceptable.

Arquitectura recomendada para el MVP

Evitar llamar a Perplexity directamente desde Unity si eso complica gestión de claves. Lo más limpio es:

- Unity envía texto o audio a un backend mínimo propio.
- Ese backend llama a Perplexity.
- Devuelve el texto final a Unity.
- Unity lo pasa al TTS placeholder.

Ventajas:

- No expones API keys en el cliente.
- Puedes cambiar Perplexity por tu modelo final más adelante.
- Puedes loguear latencia y respuestas más fácilmente.

Estrategia de velocidad

Para el MVP, no hace falta streaming perfecto desde el primer día. La versión más rápida de validar puede ser esta:

1. El usuario pulsa una tecla/botón o se usa un mock de texto inicial.
2. Se manda texto a Perplexity.
3. Se recibe respuesta breve.
4. Se genera audio TTS.
5. Se reproduce audio en el avatar.
6. Oculus LipSync mueve la boca en base al audio reproducido.

Eso ya sirve para validar la sensación de “hablar con un personaje”.

Cómo probar sin depender aún del STT real

Para ir muy rápido en 2 días, no empieces por micrófono + STT real sino es imprescindible. Haz primero una escalera de validación:

Nivel 1: texto manual

- Un input temporal en pantalla o en inspector.
- Enviar texto a Perplexity.
- Recibir texto de respuesta.
- Pasarlo a TTS.
- Reproducir en el avatar.

Esto valida backend, LLM y lipsync con el menor coste de implementación.

Nivel 2: texto simulado por teclado

- Botones de prueba con frases prefijadas.
- Una frase corta, una media, una larga.
- Medición de tiempos por tipo de respuesta.

Nivel 3: micrófono + STT

- Solo cuando el loop de respuesta ya esté validado.
- Añadir después captación de audio y transcripción.

Este orden te da resultados útiles antes y evita perder el primer día en captación y formatos de audio.

TTS placeholder para el MVP

La elección del TTS provisional debe ser la que te permita llegar antes a un **AudioClip reproducible**. El requisito real del MVP no es tener la mejor voz, sino poder poner audio en un `AudioSource` y alimentar Oculus LipSync con ese audio. ^[47] ^[53]

Qué debe cumplir el TTS temporal:

- API fácil de integrar.
- Respuesta rápida.
- Formato de audio fácil de decodificar en Unity.
- Voces suficientemente claras para evaluar la experiencia.

Si el TTS te devuelve WAV o MP3 sencillo y puedes convertirlo o reproducirlo en Unity de forma fiable, ya es suficiente para el MVP.

Roadmap detallado para tener algo en 2-3 días

Día 1: proyecto, editor VR y prueba visual

Objetivo

Acabar el día con el proyecto arrancando en VR desde el Editor usando Quest Link, un avatar delante y una escena base muy sencilla.^[50] ^[48]

Tareas

1. Crear proyecto URP en Unity 6.1+.^[50]
2. Activar Meta Quest build platform.^[49]
3. Activar OpenXR en Android/Meta Quest.^[49] ^[46]
4. Activar OpenXR en Windows/Mac/Linux para Play Mode con Link.^[48]
5. Configurar Meta Quest feature group.^[49] ^[48]
6. Instalar Meta XR Core SDK.^[46]
7. Probar Play Mode en visor mediante Quest Link.^[48]
8. Crear escena MainMVP.
9. Añadir rig XR mínimo.
10. Colocar un fondo sencillo.
11. Importar un avatar placeholder cualquiera.
12. Posicionarlo bien delante del usuario.
13. Crear UI de estado con texto: Idle, Processing, Speaking, Error.

Resultado esperado

- Das Play en Unity.
- Te pones las Quest con Link.
- Ves el avatar delante sin hacer build.
- Puedes iterar escena y layout rápido.

Día 2: loop conversacional falso pero funcional

Objetivo

Acabar el día con conversación básica: input de texto o frases de prueba, llamada a Perplexity, respuesta, TTS y audio saliendo del avatar.

Tareas

1. Implementar `ConversationController` con estados básicos.
2. Implementar `PerplexityClient` o conexión al backend gateway.
3. Hacer una UI o inspector field para escribir texto manual.
4. Enviar prompt a Perplexity.
5. Recibir respuesta y mostrarla en debug UI.
6. Integrar TTS placeholder.
7. Convertir respuesta en audio reproducible.
8. Reproducirlo con `AudioSource` situado en el avatar.
9. Medir latencias simples: request start, response received, playback start.

Resultado esperado

- Escribes una frase o pulsas un botón.
- La IA responde.
- El avatar “habla” con voz reproducida.

Aunque aún no haya micrófono, ya podrás enseñar una demo funcional del concepto.

Día 3: lip sync y validación del producto

Objetivo

Acabar el día con el avatar moviendo la boca mientras habla, todo probado desde editor y con primera validación seria de viabilidad.

Tareas

1. Importar Oculus LipSync.^[47]
2. Montar el setup `AudioSource` + `OVRipSyncContext` + `OVRipSyncContextMorphTarget`.^[55]
^[53]
3. Validarlo con un clip local.^[51]
4. Conectarlo al audio TTS real del día 2.
5. Ajustar smoothing y visemas.
6. Grabar vídeo interno de la demo.
7. Revisar sensaciones clave: latencia, naturalidad, claridad, credibilidad.

Resultado esperado

- El usuario lanza una frase de prueba.
- El backend responde con Perplexity.
- El TTS genera voz.
- El avatar habla con lip sync básico.
- Ya puedes juzgar si el producto tiene sentido.

Qué haría justo después de este MVP

Si el MVP sale bien, el siguiente orden correcto sería:

1. Sustituir input manual por micrófono real.
2. Añadir STT.
3. Probar conversación completa hablada.
4. Mejorar avatar placeholder si hace falta.
5. Empezar pruebas standalone en Quest 2.
6. Reemplazar Perplexity por el LLM/backend final.
7. Mejorar buffering, TTS streaming y tiempos de respuesta.

No cambiaría primero el arte. Cambiaría primero la calidad real del loop conversacional.

Recomendaciones para no perder tiempo

1. Primero texto, luego voz de entrada

La entrada de voz es útil, pero no es el primer cuello de botella para validar producto. Lo primero es demostrar que una pregunta produce una respuesta con presencia visual y sonora.

2. Editor primero, build después

Para MVP, casi toda la iteración debe hacerse en Editor con Quest Link. Solo cuando el flujo base esté funcionando merece la pena pasar a builds frecuentes en Quest standalone.^[54] ^[48]

3. Avatar compatible antes que avatar bonito

Si el avatar bonito rompe el lipsync, retrasa la validación. El placeholder debe elegirse por compatibilidad técnica.^[53] ^[47]

4. Respuestas cortas

Pide al modelo respuestas breves. Cuanto más cortas, más rápida y convincente será la demo inicial.

5. Instrumentación desde el día 1

Mostrar en pantalla o consola:

- Tiempo hasta respuesta de Perplexity.
- Tiempo hasta audio listo.
- Tiempo hasta playback.
- Texto enviado y recibido.
- Estado actual del flujo.

Ese debug vale oro para decidir viabilidad.

Definición de éxito del MVP

El MVP habrá cumplido su objetivo si en 2-3 días puedes demostrar todo esto:

- La app entra en VR desde el Editor usando Quest Link sin build repetitivo.^[48]
- Hay un avatar 3D delante del usuario.
- El sistema envía una consulta a Perplexity o a tu gateway.
- Llega una respuesta breve útil.
- Esa respuesta se convierte en voz.
- La voz sale del avatar.
- El avatar mueve la boca de forma aceptable mediante Oculus LipSync.^{[51] [47]}
- Puedes medir tiempos y juzgar claramente si la experiencia merece evolucionar al producto final.

Recomendación final

La forma más rápida y sensata de arrancar hoy es construir un MVP de validación, no una primera versión del producto final. El pipeline correcto para maximizar aprendizaje en pocos días es: Unity 6 + URP + OpenXR + Quest Link para iterar en Editor + avatar placeholder compatible + API de Perplexity como LLM temporal + TTS simple + Oculus LipSync.^{[50] [47] [48]}

El mejor orden real de trabajo es: primero ver el avatar en VR desde el Editor, después hacer que responda a texto, luego que hable, y solo después añadir micrófono/STT. Ese orden reduce muchísimo el tiempo hasta una demo útil y te permitirá decidir muy pronto si el backend, la latencia y la sensación del personaje justifican el desarrollo completo del producto.^{[51] [47] [48]}



el editor de unity lo instalo con el android build support no ?

Sí: si vas a trabajar para Meta Quest, instala el editor de Unity con **Android Build Support**. Quest corre sobre Android, y tanto Unity como Meta indican que necesitas ese módulo junto con **Android SDK & NDK Tools** y **OpenJDK** para poder compilar y desplegar builds al visor.^[63]^[64]

Qué marcar

En Unity Hub, en la instalación de tu editor, añade estos módulos: **Android Build Support**, **Android SDK & NDK Tools** y **OpenJDK**. Meta lo especifica así en su guía de setup para Unity, y Unity lo confirma en su manual de entorno Android.^[64] ^[63]

Para tu caso

Aunque vayas a iterar mucho desde el Editor con **Meta Quest Link**, igualmente te conviene tener **Android Build Support** instalado desde el primer día, porque en cuanto quieras validar standalone en Quest necesitarás generar APK para el target Meta Quest/Android.^[65] ^[66] ^[67]

Respuesta práctica

Instálalo así en Unity Hub:

- Unity Editor.
- Android Build Support.
- Android SDK & NDK Tools.
- OpenJDK.^[63] ^[64]

Y como apunte: para **probar en Play Mode con Link** no dependes de hacer build Android cada vez, pero para el proyecto Quest real sí necesitas ese soporte instalado tarde o temprano.^[67]^[64]

¿Quieres que te deje ahora una checklist ultra corta de instalación exacta en Unity Hub + los paquetes que abriría nada más crear el proyecto?



como empujo el repo local al que acabo de crear en github

Sí: si ya has creado el repo vacío en GitHub y tienes el proyecto local en tu PC, lo normal es hacer `git init`, añadir el remoto y hacer el primer push. GitHub documenta ese flujo para subir un proyecto local a un repo nuevo.^[79]

Comandos

Desde la carpeta raíz de tu proyecto:

```
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/TU_USUARIO/TU_REPO.git
git push -u origin main
```

Eso crea el repo local si aún no existía, hace el primer commit, pone la rama en `main`, conecta el remoto `origin` y sube el código a GitHub. [\[80\]](#) [\[79\]](#)

Si ya tenías git iniciado

Si el proyecto ya es un repo Git local, no repitas `git init`. En ese caso bastaría con:

```
git remote add origin https://github.com/TU_USUARIO/TU_REPO.git
git branch -M main
git push -u origin main
```

Eso enlaza tu repo local con GitHub y deja `main` como upstream para futuros `git push` normales. [\[79\]](#) [\[80\]](#)

Si GitHub te da error

Si al crear el repo en GitHub marcaste opciones como README, `.gitignore` o licencia, el repo remoto ya no está vacío y puede darte conflicto en el primer push. En ese caso haz antes un `git pull --rebase origin main` y luego vuelve a empujar, o crea el repo vacío desde GitHub para evitar esa fricción inicial. [\[81\]](#) [\[82\]](#)

Recomendación práctica

Para tu caso, lo más limpio es que el repo de GitHub esté vacío al crearlo, sin README ni `.gitignore`, y luego ejecutar estos comandos:

```
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/TU_USUARIO/TU_REPO.git
git push -u origin main
```

Después de eso, los siguientes cambios ya serán simplemente:

```
git add .
git commit -m "mensaje"
```

¿Quieres que te dé también el `.gitignore` ideal para un proyecto Unity antes de hacer ese primer push?



entonces realmente crees que este enfoque para la conversacion de stt + llm + tts + lipsync es el más rápido y el que mejor va a funcionar?

Sí, para un MVP rápido en Quest creo que ese enfoque es el más sensato en conjunto: STT + LLM + TTS + lipsync, pero con una matización importante: para llegar antes, no empezaría implementando los cuatro bloques a la vez, sino que los activaría por capas. Oculus LipSync está pensado justo para analizar audio de micrófono o archivo y producir visemas, así que encaja muy bien como capa final de presentación sobre una voz ya reproducible.^[94] ^[95]

Lo más rápido

El camino más rápido para validar producto no es “voz completa desde el minuto uno”, sino este orden: texto manual -> LLM -> TTS -> lipsync -> STT. Ese orden reduce muchísimo el riesgo porque primero validas si el personaje responde bien y si el audio + boca convence, y solo después añades la parte más delicada de captura y transcripción.^[95] ^[94]

Para probar backend y viabilidad, el verdadero núcleo es “pregunta entra, respuesta sale, el avatar la dice de forma creíble”. Si eso no funciona, meter STT antes solo añade complejidad sin darte más señal de producto.^[96]

Lo que mejor va a funcionar

Si preguntas por la mejor combinación entre velocidad de desarrollo y probabilidad de que funcione bien, mi respuesta es: sí, pero usando audio reproducido como fuente del lipsync, no intentando sincronizar la boca a partir del texto o de eventos abstractos. Meta documenta precisamente ese flujo con `AudioSource`, `OVR LipSyncContext` y `OVR LipSyncContextMorphTarget`, donde el lipsync se alimenta del audio real.^[97] ^[94]

Eso suele funcionar mejor porque:

- La boca sigue lo que realmente oye el usuario.^[94]
- Evitas desajustes entre texto generado y audio final.^[97]
- Puedes probarlo rápido con clips locales antes de conectar backend.^[95]

Dónde están los riesgos

El mayor riesgo no está en el LLM ni en el lipsync; está en la **latencia acumulada** y en el audio. En Quest, Meta exige 72 FPS mínimos para apps interactivas, así que además de la respuesta conversacional tienes que cuidar frame time, buffering y reproducción para que no haya sensación de lentitud o tirones.^[96]

Los puntos conflictivos reales son:

- STT con endpointing malo, que tarda en detectar fin de frase.
- TTS que no arranca rápido.
- Buffers de audio mal ajustados.
- Avatar placeholder con blendshapes pobres.
- Probar solo en Editor y confiarse con el rendimiento real de Quest.^{[95] [97] [96]}

Mi recomendación real

Para el MVP de ahora, haría exactamente esto:

1. Texto manual desde Unity.
2. Llamada a Perplexity.
3. TTS placeholder.
4. Reproducción en AudioSource.
5. Oculus LipSync sobre ese AudioSource.
6. Prueba en visor desde el Editor con Meta Quest Link.^{[98] [94] [97]}

Y después, cuando eso funcione, añades STT. Eso sigue siendo el enfoque STT + LLM + TTS + lipsync como arquitectura final, pero no como orden inicial de implementación.^{[98] [95]}

Respuesta corta

Así que sí: como **arquitectura final**, me parece el enfoque correcto y probablemente el más práctico para tu caso. Pero como **estrategia de implementación rápida**, lo óptimo es construirlo en este orden: LLM + TTS + lipsync primero, STT después.^{[94] [96] [95]}

Si quieres, te hago ahora mismo un diagrama técnico muy concreto del pipeline MVP y luego el pipeline final, con los scripts/clases mínimas que montaría en Unity.

✱✱

Unity MVP loop conversacional falso pero funcional

Este paquete contiene scripts C# para montar en Unity un loop MVP de conversación basado en:

- Input manual de texto.
- Llamada a API de Perplexity.
- Respuesta de texto mostrada en debug UI.
- Llamada a un backend/gateway TTS placeholder.
- Descarga del audio y reproducción por AudioSource del avatar.
- Medición simple de latencias.

Qué incluye

- ConversationController.cs
- PerplexityClient.cs
- SimpleTTSClient.cs
- ConversationDebugView.cs
- ConversationTypes.cs

Requisitos del setup

Escena

Crea una escena con:

- Un GameObject ConversationSystem con ConversationController.
- Un GameObject del avatar con AudioSource.
- Un Canvas world-space o screen-space con TextMeshPro.
- Un TMP_InputField para escribir el prompt.
- Un botón para lanzar la conversación.
- Un TMP_Text para estado.
- Un TMP_Text para transcript/debug.

Flujo esperado

1. Escribes texto en el TMP_InputField.
2. Pulsas botón o Enter.
3. ConversationController llama a PerplexityClient.
4. Se muestra la respuesta de la IA en debug UI.

5. `SimpleTTSClient` pide un audio al backend TTS.
6. Se descarga un `AudioClip` con `UnityWebRequestMultimedia.GetAudioClip`, que Unity recomienda para obtener clips de audio por HTTP.^[109] ^[110]
7. El `AudioSource` del avatar reproduce el clip.

Importante sobre Perplexity

La documentación de Perplexity expone el endpoint de chat completions en `/v1/sonar` con `model` y `messages` como payload principal.^[111] ^[112]

Para el MVP, se usa ese formato directamente desde Unity. Si prefieres no exponer la API key en el cliente, sustituye `PerplexityClient` por tu gateway backend.

Importante sobre TTS

`SimpleTTSClient` está pensado para un backend **propio** o placeholder que devuelva un archivo de audio accesible por URL o directamente bytes descargables por HTTP. El script está preparado para pedir un recurso WAV o MP3 mediante

`UnityWebRequestMultimedia.GetAudioClip`, que es el flujo recomendado por Unity para descargar `AudioClip` desde web.mínimo** para TTS y proxy de Perplexity, por ejemplo en Node.js o Python, para que puedas levantarlo hoy mismo en local? ^[109]

✱

pero no veo el código

Sí, te lo pego aquí directamente. El fichero se generó como artefacto descargable, pero aquí tienes el código visible en el chat. El `ConversationController` que te preparé hace exactamente el loop básico: toma texto manual, llama a Perplexity, pide TTS, reproduce el `AudioClip` en el `AudioSource` del avatar y mide latencias simples de LLM, TTS y tiempo hasta playback.^[124] ^[125] ^[126]

ConversationController.cs

```
using System.Collections;
using TMPPro;
using UnityEngine;
using UnityEngine.UI;

namespace MVP.Conversation
{
    public class ConversationController : MonoBehaviour
    {
        [Header("References")]
        [SerializeField] private PerplexityClient perplexityClient;
        [SerializeField] private SimpleTTSClient ttsClient;
        [SerializeField] private AudioSource avatarAudioSource;
        [SerializeField] private TMP_InputField inputField;
        [SerializeField] private Button sendButton;
```

```

[SerializeField] private ConversationDebugView debugView;

[Header("Debug")]
[SerializeField] private bool logToConsole = true;
[SerializeField] private string fallbackPrompt = "Hola. Preséntate en una fr

private ConversationState currentState = ConversationState.Idle;
private bool isBusy;

private void Awake()
{
    SetState(ConversationState.Idle);
    if (sendButton != null)
        sendButton.onClick.AddListener(SendFromInputField);
}

public void SendFromInputField()
{
    string text = inputField != null && !string.IsNullOrEmpty(inputFieldc
        ? inputField.text.Trim()
        : fallbackPrompt;

    SendText(text);
}

public void SendText(string text)
{
    if (isBusy)
        return;

    StartCoroutine(RunConversation(text));
}

private IEnumerator RunConversation(string userText)
{
    isBusy = true;
    var result = new ConversationResult { userText = userText };
    result.timing.requestStartTime = Time.realtimeSinceStartupAsDouble;

    SetState(ConversationState.Processing);
    WriteDebug($"USER: {userText}\n\nConsultando Perplexity...");
    SetInteractable(false);

    string assistantResponse = null;
    string llmError = null;
    yield return perplexityClient.SendChat(userText, (text, error) =>
    {
        assistantResponse = text;
        llmError = error;
    });

    result.timing.llmResponseTime = Time.realtimeSinceStartupAsDouble;

    if (!string.IsNullOrEmpty(llmError))
    {
        result.error = llmError;
    }
}

```

```

        SetState(ConversationState.Error);
        WriteDebug($"USER: {userText}\n\nERROR LLM:\n{llmError}");
        FinishRequest();
        yield break;
    }

    result.assistantText = assistantResponse;
    WriteDebug($"USER: {userText}\n\nAI: {assistantResponse}\n\nSolicitando TTS");

    AudioClip clip = null;
    string ttsError = null;
    yield return ttsClient.RequestSpeech(assistantResponse, (audioClip, error) =>
    {
        clip = audioClip;
        ttsError = error;
    });

    result.timing.ttsReadyTime = Time.realtimeSinceStartupAsDouble;

    if (!string.IsNullOrEmpty(ttsError))
    {
        result.error = ttsError;
        SetState(ConversationState.Error);
        WriteDebug($"USER: {userText}\n\nAI: {assistantResponse}\n\nERROR TTS: {ttsError}");
        FinishRequest();
        yield break;
    }

    SetState(ConversationState.Speaking);
    avatarAudioSource.Stop();
    avatarAudioSource.clip = clip;
    avatarAudioSource.Play();
    result.timing.playbackStartTime = Time.realtimeSinceStartupAsDouble;

    WriteDebug(
        $"USER: {result.userText}\n\n" +
        $"AI: {result.assistantText}\n\n" +
        $"Latencia LLM: {result.timing.LlmSeconds:F2}s\n" +
        $"Latencia TTS: {result.timing.TtsSeconds:F2}s\n" +
        $"Tiempo hasta playback: {result.timing.TimeToPlaybackSeconds:F2}s"
    );

    if (logToConsole)
    {
        Debug.Log($"[Conversation] LLM={result.timing.LlmSeconds:F2}s TTS={result.timing.TtsSeconds:F2}s");
        Debug.Log($"[Conversation] USER: {result.userText}");
        Debug.Log($"[Conversation] AI: {result.assistantText}");
    }

    while (avatarAudioSource.isPlaying)
        yield return null;

    SetState(ConversationState.Idle);
    FinishRequest();
}

```

```

        private void FinishRequest()
        {
            isBusy = false;
            SetInteractable(true);
        }

        private void SetInteractable(bool value)
        {
            if (sendButton != null)
                sendButton.interactable = value;
            if (inputField != null)
                inputField.interactable = value;
        }

        private void SetState(ConversationState state)
        {
            currentState = state;
            if (debugView != null)
                debugView.SetState($"State: {currentState}");
        }

        private void WriteDebug(string text)
        {
            if (debugView != null)
                debugView.SetDebug(text);
        }
    }
}

```

PerplexityClient.cs

Este cliente usa el formato de chat completions de Perplexity con `model` y `messages`, que es el patrón documentado en su API. [\[125\]](#) [\[127\]](#)

```

using System;
using System.Collections;
using System.Collections.Generic;
using System.Text;
using UnityEngine;
using UnityEngine.Networking;

namespace MVP.Conversation
{
    public class PerplexityClient : MonoBehaviour
    {
        [Header("Perplexity")]
        [SerializeField] private string apiKey = "YOUR_PERPLEXITY_API_KEY";
        [SerializeField] private string endpoint = "https://api.perplexity.ai/v1/sonar";
        [SerializeField] private string model = "sonar";
        [SerializeField] [TextArea(3, 8)]
        private string systemPrompt = "You are a helpful in-world VR character. Answer the user's questions in a helpful and concise manner. If you are unsure, say so. Do not make up information."

        public IEnumerator SendChat(string userMessage, Action<string, string> onCompletion)
        {
            // Implementation of the chat completion logic
        }
    }
}

```

```

var requestBody = new PerplexityChatRequest
{
    model = model,
    messages = new List<PerplexityMessage>
    {
        new PerplexityMessage { role = "system", content = systemPrompt },
        new PerplexityMessage { role = "user", content = userMessage }
    }
};

string json = JsonUtility.ToJson(requestBody);
using var request = new UnityWebRequest(endpoint, UnityWebRequest.kHttpVerbPOST);
byte[] bodyRaw = Encoding.UTF8.GetBytes(json);
request.uploadHandler = new UploadHandlerRaw(bodyRaw);
request.downloadHandler = new DownloadHandlerBuffer();
request.SetRequestHeader("Content-Type", "application/json");
request.SetRequestHeader("Authorization", $"Bearer {apiKey}");

yield return request.SendWebRequest();

if (request.result != UnityWebRequest.Result.Success)
{
    onComplete?.Invoke(null, request.error + "\n" + request.downloadHandler.text);
    yield break;
}

string responseText = request.downloadHandler.text;
var parsed = JsonUtility.FromJson<PerplexityChatResponse>(responseText);

if (parsed == null || parsed.choices == null || parsed.choices.Length == 0)
{
    onComplete?.Invoke(null, "Respuesta inválida de Perplexity: " + responseText);
    yield break;
}

onComplete?.Invoke(parsed.choices[^8_0].message.content, null);
}

[Serializable]
private class PerplexityChatRequest
{
    public string model;
    public List<PerplexityMessage> messages;
}

[Serializable]
private class PerplexityMessage
{
    public string role;
    public string content;
}

[Serializable]
private class PerplexityChatResponse
{
    public Choice[] choices;
}

```

```

    }

    [Serializable]
    private class Choice
    {
        public Message message;
    }

    [Serializable]
    private class Message
    {
        public string role;
        public string content;
    }
}
}

```

SimpleTTSClient.cs

Este cliente está montado para un backend placeholder que te devuelva audio directamente por HTTP, usando `UnityWebRequestMultimedia.GetAudioClip` y `DownloadHandlerAudioClip`, que es el flujo recomendado por Unity para descargar audio reproducible.^[126] ^[128]

```

using System;
using System.Collections;
using System.Text;
using UnityEngine;
using UnityEngine.Networking;

namespace MVP.Conversation
{
    public class SimpleTTSClient : MonoBehaviour
    {
        public enum TtsRequestMode
        {
            Get,
            PostJson
        }

        [Header("TTS Placeholder")]
        [SerializeField] private string endpoint = "http://localhost:5001/tts";
        [SerializeField] private TtsRequestMode requestMode = TtsRequestMode.Get;
        [SerializeField] private AudioType audioType = AudioType.WAV;
        [SerializeField] private bool streamAudio = false;

        public IEnumerator RequestSpeech(string text, Action<AudioClip, string> onC
        {
            UnityWebRequest request;

            if (requestMode == TtsRequestMode.Get)
            {
                string encodedText = UnityWebRequest.EscapeURL(text);
                request = UnityWebRequestMultimedia.GetAudioClip($"{endpoint}?text=

```

```

else
{
    request = new UnityWebRequest(endpoint, UnityWebRequest.kHttpVerbPOST);
    byte[] body = Encoding.UTF8.GetBytes(JsonUtility.ToJson(new TtsPostBody
    {
        request.uploadHandler = new UploadHandlerRaw(body);
        var downloadHandler = new DownloadHandlerAudioClip(endpoint, audioType);
        {
            streamAudio = streamAudio;
        }
    });
    request.downloadHandler = downloadHandler;
    request.SetRequestHeader("Content-Type", "application/json");
}

if (request.downloadHandler is DownloadHandlerAudioClip dh)
    dh.streamAudio = streamAudio;

yield return request.SendWebRequest();

if (request.result != UnityWebRequest.Result.Success)
{
    onComplete?.Invoke(null, request.error);
    request.Dispose();
    yield break;
}

AudioClip clip = DownloadHandlerAudioClip.GetContent(request);
if (clip == null)
{
    onComplete?.Invoke(null, "TTS no devolvió AudioClip válido");
    request.Dispose();
    yield break;
}

onComplete?.Invoke(clip, null);
request.Dispose();
}

[Serializable]
private class TtsPostBody
{
    public string text;
}
}
}

```

ConversationDebugView.cs

```

using TMPro;
using UnityEngine;

namespace MVP.Conversation
{
    public class ConversationDebugView : MonoBehaviour
    {
        [Header("UI")]
    }
}

```

```

        [SerializeField] private TMP_Text stateText;
        [SerializeField] private TMP_Text debugText;

        public void SetState(string value)
        {
            if (stateText != null)
                stateText.text = value;
        }

        public void SetDebug(string value)
        {
            if (debugText != null)
                debugText.text = value;
        }

        public void Clear()
        {
            SetState(string.Empty);
            SetDebug(string.Empty);
        }
    }
}

```

ConversationTypes.cs

```

using System;

namespace MVP.Conversation
{
    public enum ConversationState
    {
        Idle,
        Processing,
        Speaking,
        Error
    }

    [Serializable]
    public class ConversationTiming
    {
        public double requestStartTime;
        public double llmResponseTime;
        public double ttsReadyTime;
        public double playbackStartTime;

        public double LlmSeconds => llmResponseTime - requestStartTime;
        public double TtsSeconds => ttsReadyTime - llmResponseTime;
        public double TimeToPlaybackSeconds => playbackStartTime - requestStartTime
    }

    [Serializable]
    public class ConversationResult
    {
        public string userText;
        public string assistantText;
    }
}

```

```

        public string error;
        public ConversationTiming timing = new ConversationTiming();
    }
}

```

Cómo montarlo en Unity

En la escena, crea:

- Un `GameObject` `ConversationSystem` con `ConversationController`, `PerplexityClient` y `SimpleTTSClient`.
- Un avatar con `AudioSource`.
- Un `TMP_InputField`.
- Un botón.
- Dos `TMP_Text`, uno para estado y otro para debug.^[129] ^[130]

Luego enlazas en el inspector:

- `avatarAudioSource`
- `inputField`
- `sendButton`
- `debugView`
- `perplexityClient`
- `ttsClient`^[124]

Ojo importante

Hay una limitación práctica: `JsonUtility` de Unity es bastante rígido con JSON anidado y listas, así que este cliente de Perplexity puede necesitar un ajuste pequeño según la respuesta exacta del endpoint. La estructura base de request sí sigue la documentación de Perplexity, pero para producción yo te recomendaría pasar a `Newtonsoft.Json` para parsear robustamente la respuesta.^[127] ^[125]

Mi recomendación

Lo mejor ahora es que te prepare la versión 2 corregida y más robusta, con:

- parser JSON usando `Newtonsoft`,
- soporte Enter para enviar,
- lista de frases de test,
- y un mock TTS local si aún no tienes endpoint.^[125] ^[126]

¿Te la genero?

pero el tts client que me has creado arriba resuelve un servicio local o algo no? quiero que me implementes sobre una api real de tts... no se cual usar? cual me recomiendas?

Sí: el SimpleTTSClient de arriba estaba pensado como placeholder y, tal como está, encaja mejor con un servicio propio sencillo o una URL que te devuelva audio directamente. Si quieres algo real para usar ya, mi recomendación para tu caso sería OpenAI TTS si priorizas simplicidad de integración, o ElevenLabs si priorizas latencia/voz más premium para una demo de personaje conversacional. OpenAI documenta un endpoint de speech con gpt-4o-mini-tts, varias voces integradas y salida en MP3 por defecto; ElevenLabs destaca modelos Flash/Turbo con streaming y latencias muy bajas pensadas para agentes de voz. [131] [132]

Qué usaría yo

Para ti, ahora mismo, usaría OpenAI TTS como primera implementación real. La razón es que la integración es muy directa, la API está bien documentada, devuelve audio listo para reproducir y te permite cerrar el MVP rápido sin añadir otra complejidad innecesaria. [133] [131]

Elegiría ElevenLabs solo si una de estas dos condiciones es importante desde ya:

- Quieres que la voz del personaje impresione más a nivel “demo premium”. [132]
- Vas a empujar fuerte hacia streaming/latencia conversacional muy baja en la siguiente iteración. [134] [132]

Mi recomendación concreta

Mi recomendación práctica sería esta:

Opción	Te la recomiendo si...	Motivo
OpenAI TTS	Quieres integrar hoy algo real y sencillo	Endpoint claro, salida MP3 por defecto y poca fricción. [131] [133]
ElevenLabs	Quieres mejor voz y menor latencia percibida	Flash v2.5 y Turbo v2.5 están orientados a TTS rápido/streaming. [134] [132]
Google Cloud TTS	Quieres algo corporativo y clásico	Funciona bien, pero para este MVP lo veo menos directo y menos “voice agent first”. [135] [136]

Lo que haría en tu MVP

Como ya estás usando Perplexity como placeholder del LLM, yo no metería demasiadas piezas nuevas. Haría:

- Perplexity para texto. [137]
- OpenAI TTS para generar voz. [131]
- Ese audio lo reproduces en el AudioSource del avatar.

- Luego conectas Oculus LipSync al mismo AudioSource.^[138]

Ese stack es bastante rápido de montar y suficiente para validar el producto en pocos días.^{[138] [131]}

Qué no haría

No intentaría ahora mismo:

- montar TTS local,
- montar un gateway extra sino lo necesitas hoy,
- ni ir a una opción super avanzada con streaming parcial si aún no has validado la sensación base.

Primero cerraría el loop real más simple posible con un proveedor TTS serio.^[131]

Respuesta corta

Sí: te recomendaría OpenAI TTS como opción más rápida y equilibrada para implementar ya. Si luego ves que la voz no te convence o quieres una demo más “wow”, cambiar a ElevenLabs sería la siguiente mejora natural.^{[132] [131]}

Siguiente paso útil

Puedo prepararte ahora mismo una **versión nueva de SimpleTTSClient.cs implementada directamente contra OpenAI TTS**, lista para pegar en Unity, y si quieres te hago también otra variante para ElevenLabs para que compares.^{[133] [131]}



cual tiene menor coste ?

De las opciones que comentamos, Google Cloud TTS suele salir más barato en precio puro, porque varias referencias sitúan sus voces Standard en torno a \$4 por 1 millón de caracteres y además con un free tier mensual en ciertas voces. OpenAI TTS aparece bastante por encima, con tts-1 en \$15 por 1 millón de caracteres y tts-1-hd en \$30 por 1 millón, mientras que ElevenLabs suele ir por modelo/plan y, en su pricing de API, Flash/Turbo aparece alrededor de \$0.05 por 1.000 caracteres, es decir, bastante más caro por carácter que Google y normalmente también más caro que OpenAI para uso base.^{[148] [149] [150]}

Orden por coste

Si lo miras solo por coste bruto aproximado, el orden sería:

1. Google Cloud TTS — el más barato de los tres en precio por carácter.^[149]
2. OpenAI TTS — intermedio, bastante razonable para MVP.^{[151] [148]}
3. ElevenLabs — normalmente el más caro, aunque ofrece voces y latencia más “premium”.^{[150] [152]}

Pero para tu MVP

Aunque Google sea más barato, yo seguiría recomendándote OpenAI TTS para este MVP si lo que buscas es llegar antes y con menos fricción técnica. La diferencia de coste en una fase inicial de validación suele ser pequeña en términos absolutos, y OpenAI te da una integración sencilla y directa para salir rápido.^[153] ^[148]

Regla práctica

- Si priorizas coste mínimo, usa Google Cloud TTS.^[149]
- Si priorizas velocidad de implementación + coste razonable, usa OpenAI TTS.^[148] ^[153]
- Si priorizas calidad de voz / efecto wow, usa ElevenLabs.^[154] ^[150]

Mi consejo real

Para un MVP de pocos días, el gasto de TTS probablemente no será el problema principal. En tu caso, el coste de oportunidad de perder tiempo integrando algo menos directo puede salir más caro que pagar un poco más por carácter, así que seguiría con OpenAI TTS salvo que ya tengas cuenta y stack montado en Google Cloud.^[153] ^[148]

Si quieres, te hago ahora una mini comparación con números de ejemplo, por ejemplo cuánto te costaría 100, 500 o 1.000 conversaciones con OpenAI vs Google vs ElevenLabs.

✱✱

Integración de OpenAI TTS en el MVP de Unity

Se ha añadido `OpenAITTSClient.cs` para reemplazar el cliente TTS placeholder por una integración real con la API de OpenAI Speech. OpenAI documenta el endpoint `POST /v1/audio/speech`, recomienda `gpt-4o-mini-tts` para aplicaciones realtime/inteligentes, soporta voces integradas como `coral`, y permite pedir formatos como `mp3` y `wav`. Además, OpenAI indica que para la respuesta más rápida conviene usar `wav` o `pcm`, por lo que en este MVP se deja `wav` como valor por defecto.^[11_1]^[11_2]

Qué hace este script

- Envía texto a `https://api.openai.com/v1/audio/speech`.^[11_1]
- Usa por defecto `model = gpt-4o-mini-tts`.^[11_1]
- Usa por defecto la voz `coral`, una de las voces integradas documentadas por OpenAI.^[11_1]
- Pide `response_format = wav` para reducir fricción de decoding y favorecer latencia baja.^[11_1]
- Guarda temporalmente los bytes de audio en `Application.temporaryCachePath`.
- Carga el archivo temporal como `AudioClip` con `UnityWebRequestMultimedia.GetAudioClip`, que Unity recomienda para descargar o cargar audio como clip reproducible.^[11_3]

Cómo conectarlo al ConversationController

Tienes dos opciones.

Opción rápida

Sustituir el tipo del campo `ttsClient` en `ConversationController` por `OpenAITTSClient`:

```
[SerializeField] private OpenAITTSClient ttsClient;
```

y dejar el resto igual, porque el método público mantiene esta firma:

```
public IEnumerator RequestSpeech(string text, Action<AudioClip, string> onComplete,
```

Opción limpia

Crear una interfaz común `ITTSClient` o un wrapper si luego quieres alternar entre OpenAI y ElevenLabs.

Valores recomendados para el MVP

- `model:gpt-4o-mini-tts`^[11_1]
- `voice:coral` o `alloy`.
- `responseFormat:wav`^[11_1]
- `instructions`: una frase simple con el tono del personaje, sin exagerar.

Notas importantes

1. API key en cliente

Para un MVP local esto te puede valer, pero no es ideal dejar la API key embebida en el cliente Unity. En cuanto valides el flujo, lo recomendable es mover esta llamada a un backend/gateway propio.

2. WAV mejor que MP3 para este caso

OpenAI documenta que `mp3` es el formato por defecto, pero también recomienda `wav` o `pcm` cuando quieres tiempos de respuesta más rápidos y menos coste de decodificación en apps de baja latencia.^[11_2]^[11_1]

3. Voces optimizadas para inglés

OpenAI indica que las voces están optimizadas para inglés, aunque el modelo soporta múltiples idiomas si el texto de entrada está en ese idioma. Para una prueba en español puede funcionar razonablemente bien, pero conviene validar la naturalidad cuanto antes.^[11_1]

Siguiente paso natural

Una vez esto funcione:

1. Conectar el AudioSource del avatar a Oculus LipSync.[^11_4]
2. Validar primero con clips locales y luego con el audio real generado.o)son(body);[^11_5]
byte[] bodyRaw = Encoding.UTF8.GetBytes(json);

```
using var request = new UnityWebRequest(endpoint, UnityWebRequest.kHttpVerb
request.uploadHandler = new UploadHandlerRaw(bodyRaw);
request.downloadHandler = new DownloadHandlerBuffer();
request.SetRequestHeader("Content-Type", "application/json");
request.SetRequestHeader("Authorization", $"Bearer {apiKey}");

yield return request.SendWebRequest();

if (request.result != UnityWebRequest.Result.Success)
{
    onComplete?.Invoke(null, request.error + "\n" + request.downloadHandler
    yield break;
}

byte[] audioBytes = request.downloadHandler.data;
if (audioBytes == null || audioBytes.Length == 0)
{
    onComplete?.Invoke(null, "OpenAI TTS devolvió audio vacío");
    yield break;
}

string extension = responseFormat.ToLowerInvariant() == "mp3" ? ".mp3" : "
AudioType audioType = responseFormat.ToLowerInvariant() == "mp3" ? AudioType
string filePath = Path.Combine(Application.temporaryCachePath, $"openai_tt

try
{
    File.WriteAllBytes(filePath, audioBytes);
}
catch (Exception e)
{
    onComplete?.Invoke(null, "No se pudo escribir el archivo TTS temporal:
    yield break;
}

using var fileRequest = UnityWebRequestMultimedia.GetAudioClip("file://" +
yield return fileRequest.SendWebRequest();

if (fileRequest.result != UnityWebRequest.Result.Success)
{
    onComplete?.Invoke(null, "Error cargando AudioClip temporal: " + fileR
    yield break;
}

AudioClip clip = DownloadHandlerAudioClip.GetContent(fileRequest);
if (clip == null)
{
    onComplete?.Invoke(null, "No se pudo crear AudioClip desde la respuesta
```

```

        yield break;
    }

    onComplete?.Invoke(clip, null);
}

[Serializable]
private class OpenAITtsRequest
{
    public string model;
    public string input;
    public string voice;
    public string instructions;
    public string response_format;
}

}
}

```

Siguiendo el paso útil

El siguiente paso ideal es que te deje también:

- el `ConversationController.cs` ya **adaptado** a `OpenAITTSCClient`,
- y opcionalmente un **proxy backend mínimo** para Perplexity + OpenAI TTS y no expone

¿Quieres que te genere esa versión ya integrada y lista para copiar a tu proyecto?

[11_10][11_11][11_12][11_13][11_14][11_15][11_16][11_17]

<div align="center">✂</div>

[11_1]: <https://platform.openai.com/docs/guides/text-to-speech>
 [11_2]: <https://developers.openai.com/api/docs/guides/text-to-speech>
 [11_3]: <https://docs.unity3d.com/6000.4/Documentation/ScriptReference/Networking.UnityWebRequest.html>
 [11_4]: <https://developers.meta.com/horizon/documentation/unity/audio-ovrlipsync-unity>
 [11_5]: <https://developers.meta.com/horizon/documentation/unity/audio-ovrlipsync-speech>
 [11_6]: <https://docs.ttsopenai.com/resources/tts-text/>
 [11_7]: <https://www.youtube.com/watch?v=ZpAuH2J3M3w>
 [11_8]: <https://learn.microsoft.com/en-us/azure/ai-services/openai/text-to-speech-quickstart>
 [11_9]: <https://developers.openai.com/api/docs/guides/speech-to-text>
 [11_10]: <https://stackoverflow.com/questions/78706287/unitywebrequest-getaudioclip-returns-null>
 [11_11]: <https://docs-dev.ttsopenai.com/resources/tts-text/>
 [11_12]: <https://docs.n8n.io/integrations/builtin/app-nodes/n8n-nodes-langchain-openai/>
 [11_13]: <https://github.com/travisvn/openai-edge-tts>
 [11_14]: <https://docs.newapi.pro/en/docs/api/ai-model/audio/openai/createspeech>
 [11_15]: https://www.reddit.com/r/Unity3D/comments/12j65u4/getting_unitywebrequestm
 [11_16]: <https://learn.microsoft.com/it-it/azure/ai-services/openai/text-to-speech-quickstart>
 [11_17]: <https://simonwillison.net/2024/Oct/18/openai-audio/>

pero donde está el código de OpenAITTSCClient ? no lo veo

Sí, aquí lo tienes pegado directamente. Este `OpenAITTSCClient` llama al endpoint de

```
## OpenAITTSCClient.cs
```

```
``csharp
using System;
using System.Collections;
using System.IO;
using System.Text;
using UnityEngine;
using UnityEngine.Networking;

namespace MVP.Conversation
{
    public class OpenAITTSCClient : MonoBehaviour
    {
        [Header("OpenAI TTS")]
        [SerializeField] private string apiKey = "YOUR_OPENAI_API_KEY";
        [SerializeField] private string endpoint = "https://api.openai.com/v1/audio";
        [SerializeField] private string model = "gpt-4o-mini-tts";
        [SerializeField] private string voice = "coral";
        [SerializeField] [TextArea(2, 5)]
        private string instructions = "Speak in a warm, natural, conversational tone";
        [SerializeField] private string responseFormat = "wav";

        public IEnumerator RequestSpeech(string text, Action<AudioClip, string> onC
        {
            var body = new OpenAITtsRequest
            {
                model = model,
                input = text,
                voice = voice,
                instructions = instructions,
                response_format = responseFormat
            };

            string json = JsonUtility.ToJson(body);
            byte[] bodyRaw = Encoding.UTF8.GetBytes(json);

            using var request = new UnityWebRequest(endpoint, UnityWebRequest.kHttpV
            request.uploadHandler = new UploadHandlerRaw(bodyRaw);
            request.downloadHandler = new DownloadHandlerBuffer();
            request.SetRequestHeader("Content-Type", "application/json");
            request.SetRequestHeader("Authorization", $"Bearer {apiKey}");

            yield return request.SendWebRequest();

            if (request.result != UnityWebRequest.Result.Success)
            {
                onComplete?.Invoke(null, request.error + "\n" + request.downloadHanc
                yield break;
            }

            byte[] audioBytes = request.downloadHandler.data;
            if (audioBytes == null || audioBytes.Length == 0)
            {
                onComplete?.Invoke(null, "OpenAI TTS devolvió audio vacío");
                yield break;
            }
        }
    }
}
```

```

    }

    string extension = responseFormat.ToLowerInvariant() == "mp3" ? ".mp3" :
    AudioType audioType = responseFormat.ToLowerInvariant() == "mp3" ? AudioC
    string filePath = Path.Combine(Application.temporaryCachePath, $"openai

    try
    {
        File.WriteAllBytes(filePath, audioBytes);
    }
    catch (Exception e)
    {
        onComplete?.Invoke(null, "No se pudo escribir el archivo TTS tempor
        yield break;
    }

    using var fileRequest = UnityWebRequestMultimedia.GetAudioClip("file://"
    yield return fileRequest.SendWebRequest();

    if (fileRequest.result != UnityWebRequest.Result.Success)
    {
        onComplete?.Invoke(null, "Error cargando AudioClip temporal: " + fi
        yield break;
    }

    AudioClip clip = DownloadHandlerAudioClip.GetContent(fileRequest);
    if (clip == null)
    {
        onComplete?.Invoke(null, "No se pudo crear AudioClip desde la respu
        yield break;
    }

    onComplete?.Invoke(clip, null);
}

[Serializable]
private class OpenAITtsRequest
{
    public string model;
    public string input;
    public string voice;
    public string instructions;
    public string response_format;
}
}
}

```

Dos cambios que te recomiendo ya

Yo haría estas dos mejoras antes de usarlo en serio:

- Sustituir la API key hardcodeada por una variable cargada desde config local o desde tu gateway, porque dejarla en el cliente es cómodo para test pero malo para seguridad.^[165]

- Empezar con `responseFormat = "wav"` para evitar fricción de decoding y facilitar lipsync sobre el `AudioSource`. OpenAI además recomienda `wav` o `pcm` cuando buscas respuesta rápida.^[165]

Cómo conectarlo

En tu `ConversationController`, cambia:

```
[SerializeField] private SimpleTTSClient ttsClient;
```

por:

```
[SerializeField] private OpenAITTSClient ttsClient;
```

y mantén la llamada a `RequestSpeech(...)` igual, porque la firma es compatible con el flujo que ya te pasé.^[166]

Ojo con un detalle real

Este script funciona para MVP, pero no es la solución más eficiente posible porque hace dos pasos: primero descarga bytes y luego los reabre como archivo temporal para construir el `AudioClip`. Para validar hoy está bien; para una iteración siguiente, te conviene mover esto a un backend/proxy o hacer una carga más optimizada.^[167] ^[165]

Si quieres, ahora te pego también el `ConversationController` ya modificado para usar `OpenAITTSClient` directamente.

✱

dime como crearme cuenta de openaitts para empezar a consumir ya... si no no puedo hacer ninguna prueba..

Sí. Para empezar a consumir OpenAI TTS no necesitas una “cuenta aparte de TTS”; necesitas una cuenta en la plataforma API de OpenAI, activar `billing` y crear una `API key`. OpenAI usa el mismo acceso para TTS, chat y el resto de la API, y su quickstart va justo por ese flujo.^[168] ^[169] ^[170]

Pasos

1. Entra en platform.openai.com y crea cuenta o inicia sesión. OpenAI permite registro por email o proveedores como Google, Microsoft o Apple.^[171] ^[168]
2. Verifica email y, si te lo pide, teléfono. Varias guías actuales indican que la verificación puede ser necesaria para activar el uso inicial de la API.^[172]

3. Ve a **Settings > Billing** y añade método de pago. Las referencias actuales indican que el acceso API funciona con billing/prepago y que debes añadir saldo o tener pago configurado antes de usar la API con normalidad.^[173] ^[174] ^[175]
4. Crea una API key en la sección API Keys. OpenAI y varias guías indican que la clave secreta se muestra una sola vez, así que guárdala bien.^[176] ^[173]
5. Haz una prueba simple contra el endpoint de speech, que en OpenAI está en POST /v1/audio/speech para TTS.^[169] ^[177]

Lo más práctico para ti

Para arrancar hoy, haría esto:

- Meter 5–10 USD de saldo o el mínimo razonable en billing, porque para pruebas de MVP te sobra.^[174] ^[173]
- Crear una key llamada algo como `unity-vr-mvp-tts`.^[176]
- Probar primero con una llamada manual fuera de Unity para confirmar que tu cuenta ya responde.^[169]

Test rápido

Puedes probar con `curl` así, porque OpenAI documenta el endpoint de speech con modelo, voz e input de texto.^[177] ^[169]

```
curl https://api.openai.com/v1/audio/speech \
-H "Authorization: Bearer TU_API_KEY" \
-H "Content-Type: application/json" \
-d '{
  "model": "gpt-4o-mini-tts",
  "voice": "coral",
  "input": "Hola, soy tu personaje virtual en realidad virtual.",
  "response_format": "wav"
}' \
--output speech.wav
```

Sieso te devuelve `speech.wav`, ya tienes la cuenta lista para integrarla en Unity. OpenAI documenta precisamente esos campos como parte del flujo básico de TTS.^[177] ^[169]

Consejo importante

No confundas ChatGPT Plus con acceso API. Tener ChatGPT de pago no implica automáticamente que ya tengas billing configurado para la plataforma API; eso se gestiona en platform.openai.com.^[175] ^[168]

Qué haría justo después

En cuanto tengas la key:

- prueba el `curl`,
- luego pégala en tu `OpenAITTSClient`,
- y haz una prueba mínima en Unity con una frase fija. [\[169\]](#) [\[177\]](#)

Si quieres, te doy ahora el checklist exacto de “clic por clic” dentro de la web de OpenAI para no perderte en ningún menú.

✱

pero una pregunta... perplexity entiendo que usa openAI por detras no? No puedo usar directamente mi billing de perplexity ?Ç

No, en la práctica no puedes reutilizar tu billing de Perplexity para consumir OpenAI TTS. Aunque Perplexity te dé acceso a modelos avanzados en el producto o pueda usar proveedores externos por detrás, su facturación de API es un servicio separado y no te da automáticamente acceso a endpoints de OpenAI fuera de Perplexity. El centro de ayuda de Perplexity dice explícitamente que el acceso API se compra como servicio aparte mediante créditos en la pestaña API, y que no necesitas ni siquiera una suscripción de Perplexity para usar esa API.

Lo importante

Hay dos cosas distintas:

- **Perplexity Pro/Max/web app billing:** te paga el uso del producto de Perplexity.
- **Perplexity API billing:** compra de créditos para la API de Perplexity, separada de la suscripción.

Eso no equivale a tener saldo en **OpenAI API**. Si quieres llamar al endpoint `POST /v1/audio/speech` de OpenAI, necesitas cuenta y billing en la plataforma API de OpenAI. [\[184\]](#) [\[185\]](#)

Para tu caso

Si sigues con:

- **LLM placeholder** = Perplexity
- **TTS** = OpenAI

entonces tendrás dos billings separados:

- créditos o pago en Perplexity API para el texto,
- billing en OpenAI API para el audio TTS. [\[185\]](#) [\[184\]](#)

Alternativa más simple

Si quieres simplificar pagos y credenciales, tienes dos caminos:

- Usar solo Perplexity para el LLM y solo OpenAI para TTS, que es lo que estamos planteando ahora.^[184]
- O cambiar y usar un proveedor que te cubra más piezas del pipeline bajo una sola cuenta, pero eso ya sería otra decisión de stack.

Respuesta corta

Así que no: tu billing de Perplexity no te sirve para consumir directamente OpenAI TTS. Para OpenAI TTS necesitas cuenta API y billing de OpenAI por separado.^[185] ^[184]

Si quieres ahorrar tiempo, una decisión bastante razonable ahora es: Perplexity para texto y OpenAI para TTS, aunque implique dos claves distintas. ¿Prefieres seguir así o quieres que te proponga una alternativa de un solo proveedor para reducir cuentas y billing?

*
**

si, seguimos así. Dime los pasos para continuar con el billing en openai

Sí. Para continuar hoy mismo con el billing de OpenAI y poder probar el TTS, haría esto en este orden: entrar en la plataforma API, añadir método de pago, cargar saldo prepago, crear la API key y hacer un test con curl. Las guías actuales indican que OpenAI funciona con prepago, que el mínimo práctico de entrada suele ser \$5, y que cuando el saldo llega a cero las peticiones dejan de funcionar hasta recargar.^[186] ^[187] ^[188]

Pasos exactos

1. Entra en platform.openai.com con tu cuenta API. El quickstart de OpenAI parte desde esa plataforma, no desde ChatGPT.^[189]
2. Ve a Settings > Billing. Varias guías actuales sitúan ahí la gestión de pagos y saldo.^[187] ^[190]
3. Pulsa Add payment method y añade tu tarjeta. El flujo estándar actual es añadir primero el método de pago en billing.^[191] ^[187]
4. Después pulsa Add to balance o el botón equivalente para cargar saldo prepago. OpenAI Help indica que los créditos son prepago, caducan al año y pueden tardar unos minutos en reflejarse.^[192] ^[186]
5. Mete una cantidad pequeña para arrancar, por ejemplo \$5-10, que para un MVP te sobra de momento. Varias guías actuales recomiendan justo ese rango inicial.^[193] ^[190]
6. Si ves la opción, activa auto-recharge con un umbral bajo para no quedarte bloqueado en mitad de las pruebas, por ejemplo recargar cuando baje de \$2-5. Las guías recientes lo recomiendan para evitar interrupciones.^[188] ^[187]

7. Ve a API Keys y crea una clave nueva. Las guías insisten en que la guardes en el momento porque normalmente solo se muestra una vez.[\[190\]](#) [\[187\]](#)

Lo que deberías ver

Cuando quede bien configurado, deberías tener:

- un método de pago guardado,
- saldo visible en billing,
- una API key creada,
- y capacidad para llamar a la API de speech.[\[186\]](#) [\[187\]](#)

Si acabas de comprar créditos y aún no te deja llamar a la API, OpenAI Help indica que puede haber un pequeño retraso hasta que el balance se refleje.[\[186\]](#)

Prueba inmediata

En cuanto tengas saldo y clave, prueba esto en terminal para confirmar que el TTS ya responde. OpenAI documenta el endpoint de speech y los campos `model`, `voice`, `input` y `response_format`.[\[194\]](#) [\[195\]](#)

```
curl https://api.openai.com/v1/audio/speech \
  -H "Authorization: Bearer TU_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "gpt-4o-mini-tts",
    "voice": "coral",
    "input": "Hola, soy tu personaje virtual en realidad virtual.",
    "response_format": "wav"
  }' \
  --output speech.wav
```

Si te genera `speech.wav`, ya puedes pasar a Unity.[\[195\]](#) [\[194\]](#)

Dos consejos prácticos

- Usa una key separada para este MVP y nómbrala algo como `unity-vr-mvp`, para revocarla fácil después.[\[187\]](#)
- No metas mucho saldo al principio; para testear TTS te basta muy poco. El prepago además es no reembolsable y expira al año según la ayuda oficial.[\[186\]](#)

Orden recomendado ahora

Haz esto exactamente:

- Billing.
- Add payment method.
- Add to balance.

- API key.
- `curl` test.
- Unity. [\[189\]](#) [\[187\]](#) [\[186\]](#)

Cuando tengas el `curl` funcionando, el siguiente paso lógico es pegar tu key en `OpenAITTSClient` y probar una frase fija en escena. ¿En qué paso exacto estás ahora mismo: ya dentro de billing, o aún creando la cuenta API?

**

pero voy a hacer pagos sin saber si funciona correctamente con audio de voz en español ?

Buena duda: sí puedes probar español, pero no tienes garantía total de que la voz te encante antes de pagar. Lo importante es que OpenAI sí soporta español en TTS, porque su documentación indica soporte multilenguaje siguiendo el soporte de Whisper y lista explícitamente Spanish entre los idiomas soportados; además, aclara que las voces están optimizadas para inglés, así que en español puede funcionar bien pero no siempre será tan natural como en inglés. [\[203\]](#)

Lo que eso significa

Traducido a tu caso: **funcionar**, **funciona** en español; la incertidumbre está más en la **calidad/prosodia/accentuación**, no en si el sistema “entiende” el idioma. OpenAI dice que puedes generar audio simplemente pasando el texto en el idioma deseado, pero también advierte que las voces están optimizadas para inglés. [\[204\]](#) [\[203\]](#)

Mi consejo realista

Sí seguiría con OpenAI TTS para el MVP, pero **con gasto mínimo** y una prueba muy acotada. Como el prepago de OpenAI puede arrancar con una cantidad pequeña y el coste por TTS es bajo, lo razonable es meter lo mínimo, hacer 10-20 pruebas con frases reales en español y decidir en menos de una hora si te sirve o no. [\[205\]](#) [\[206\]](#)

Cómo reducir el riesgo

Haz estas pruebas antes de comprometerte más:

- Frases cortas y medianas en español.
- Frases interrogativas y exclamativas.
- Una respuesta conversacional típica de tu NPC.
- Varias voces, por ejemplo `coral`, `alloy`, `nova` o la que más te encaje entre las disponibles. OpenAI documenta varias voces integradas para TTS. [\[203\]](#)

Un test muy representativo sería algo como:

- “Hola, soy tu guía virtual.”

- “Claro, puedo ayudarte con eso.”
- “No estoy completamente seguro, pero creo que esta es la mejor opción.”
- “¿Quieres que te explique eso paso a paso?” [\[203\]](#)

La respuesta honesta

Así que la respuesta corta es: sí, puedes usarlo en español y probablemente te valga para MVP, pero no te diría “paga tranquilo, será perfecto” porque OpenAI no promete optimización específica para español; solo dice que el modelo soporta múltiples idiomas aunque las voces estén optimizadas para inglés. [\[203\]](#)

Lo que haría yo en tu lugar

Yo haría esto:

- Añadir el mínimo saldo.
- Probar 3-4 voces con 10 frases reales en español.
- Elegir en 30 minutos si OpenAI TTS te sirve para el MVP o si saltas a ElevenLabs. [\[205\]](#) [\[203\]](#)

Eso es mejor que bloquear el proyecto por la duda. La inversión para validar es pequeña y te da una respuesta real, no teórica. [\[206\]](#) [\[205\]](#)

Mi recomendación práctica

Si tu objetivo es validar producto rápido, OpenAI TTS sigue siendo una apuesta razonable. Si tu objetivo fuera “voz española final con máxima naturalidad”, ahí sí sería más prudente comparar pronto contra ElevenLabs u otra opción más centrada en voz premium multilingüe. [\[207\]](#) [\[203\]](#)

Si quieres, te preparo ahora una lista de frases de benchmark en español para probar hoy mismo varias voces y decidir objetivamente si OpenAI TTS pasa o no la prueba para tu avatar.

✱

una pregunta... si al jefe le pido saldo para la api de openai, no es mejor resolver todo con la misma api? tanto el stt como el tts y tambien la primera request que se hace con perplexity ?

Sí, para presentárselo al jefe tiene bastante sentido decir “vamos con un único proveedor para todo el pipeline inicial”. OpenAI hoy te cubre STT, LLM y TTS dentro de la misma plataforma API: tiene endpoints de speech-to-text, text-to-speech y modelos de chat, e incluso APIs de audio/realtime para experiencias conversacionales más unificadas. [\[221\]](#) [\[222\]](#) [\[223\]](#)

Mi respuesta corta

Para un MVP rápido, sí: probablemente es mejor simplificar y resolver todo con OpenAI en lugar de mezclar Perplexity + OpenAI. Tener un solo proveedor te reduce cuentas, billing, claves, debugging, latencia de integración y puntos de fallo. [\[222\]](#) [\[224\]](#) [\[221\]](#)

Cuándo mantendría Perplexity

Yo mantendría Perplexity solo si una de estas dos cosas es importante desde el primer día:

- Quieres respuestas **grounded con búsqueda web y citas** como parte central del producto. Perplexity destaca precisamente por su enfoque search-first y citas inline. [\[225\]](#) [\[226\]](#)
- Tu demo necesita responder sobre información muy actual y verificable en web desde ya. [\[225\]](#)

Sino, para un personaje VR conversacional de validación, OpenAI te simplifica bastante más el arranque. [\[224\]](#) [\[223\]](#)

Lo que ganas con OpenAI único

Usar una sola API para todo te da estas ventajas:

- Un solo billing. [\[227\]](#) [\[222\]](#)
- Una sola gestión de claves. [\[227\]](#)
- Un stack más simple: audio/transcriptions para STT, chat/completions o equivalente para texto, y audio/speech para TTS. [\[228\]](#) [\[221\]](#) [\[222\]](#)
- Camino claro hacia algo más avanzado con **Realtime API** si luego quieres latencia más baja, turn detection e interrupciones. [\[223\]](#)

Eso, para un MVP con plazo corto, vale mucho más que tener “el mejor proveedor por pieza”. [\[224\]](#) [\[223\]](#)

Mi recomendación real para tu caso

Con lo que me has contado, yo haría esto:

MVP inmediato

- LLM: OpenAI.
- TTS: OpenAI.
- STT: todavía no hoy; primero texto manual, luego OpenAI STT. [\[221\]](#) [\[222\]](#)

Siguiente iteración

- Añadir STT de OpenAI.
- Más adelante evaluar si compensa pasar a Realtime API. [\[223\]](#) [\[221\]](#)

Eso te permite decirle al jefe algo bastante defendible:

“Para validar el producto rápido, usamos un único proveedor que cubre voz de entrada,

generación y voz de salida; si luego necesitamos grounding web o mejores respuestas de búsqueda, reevaluamos Perplexity.” [\[222\]](#) [\[221\]](#) [\[225\]](#)

Mi matiz importante

No te diría que OpenAI sea automáticamente “mejor” que Perplexity para el componente de pregunta-respuesta factual actualizada. Ahí Perplexity puede tener ventaja por su orientación a búsqueda y citas. [\[226\]](#) [\[225\]](#)

Pero para llegar antes con menos fricción, sí creo que OpenAI-only es una mejor decisión de MVP. [\[224\]](#) [\[223\]](#)

Qué haría yo si empezaras hoy de cero

- Pediría saldo para OpenAI API.
- Montaría primero texto manual -> OpenAI chat -> OpenAI TTS -> AudioSource -> lipsync. [\[228\]](#) [\[222\]](#)
- Después añadiría OpenAI STT. [\[221\]](#)
- Y solo si el producto demuestra valor, decidiría si mantener todo en OpenAI o sustituir el bloque LLM por Perplexity para respuestas con grounding. [\[225\]](#)

Respuesta clara

Así que sí: para el jefe, y para reducir complejidad, tiene bastante lógica pedir saldo de OpenAI y resolver con la misma API todo el pipeline inicial. [\[222\]](#) [\[223\]](#) [\[221\]](#)

¿Quieres que rehaga ahora mismo tu plan técnico para pasar de “Perplexity + OpenAI TTS” a “OpenAI-only MVP”, incluyendo qué endpoint usar en cada fase y cómo quedaría el código?

✱

OpenAI-only MVP para Unity VR

Este paquete actualiza el loop del MVP para usar solo OpenAI en el prototipo inicial, pero dejando la parte de chat suficientemente desacoplada para que luego puedas sustituirla por tu LLM custom con contexto histórico sin reescribir todo el flujo. OpenAI ofrece endpoints para chat/completions, speech-to-text y text-to-speech, así que es razonable unificar el MVP bajo un único proveedor al principio. [\[237\]](#) [\[238\]](#) [\[239\]](#)

Objetivo arquitectónico

La idea no es atarte a OpenAI en toda la app. La idea es usar OpenAI como implementación inicial de tres interfaces lógicas:

- IChatService
- ITTSService
- ISTTService (preparada para más adelante)

Así, el `ConversationController` no depende de Perplexity ni de un modelo concreto, sino de un proveedor de chat abstracto. Más adelante podrás reemplazar `OpenAIChatClient` por un `CustomCharacterLLMClient` sin tocar el resto del loop conversacional.^[240] ^[237]

Archivos incluidos

- `ConversationContracts.cs`
- `ConversationStateTypes.cs`
- `OpenAIChatClient.cs`
- `OpenAITTSCient.cs`
- `OpenAIConversationController.cs`
- `ConversationDebugView.cs`
- `OpenAI_Only_MVP_Notes.md`

Qué hace el loop actual

1. El usuario escribe texto manual en un `TMP_InputField`.
2. `OpenAIConversationController` lo envía a un servicio de chat desacoplado.
3. La implementación actual de ese servicio es `OpenAIChatClient`, que llama al endpoint de chat completions con `model y messages`.^[241] ^[237]
4. La respuesta se muestra en la UI de debug.
5. `OpenAITTSCient` llama al endpoint `POST /v1/audio/speech` de OpenAI, convertido el controlador en algo rígido atado a OpenAI. OpenAI Chat Completions es aquí solo una implementación temporal detrás de una interfaz intercambiable.^[239] ^[237] ^[241]

Siguiente paso realista

Tu siguiente paso práctico ahora es:

- meter tu `OPENAI_API_KEY` en `OpenAIChatClient` y `OpenAITTSCient`,
- montar la escena con `TMP_InputField`, botón, debug view y `AudioSource`,
- y hacer una primera prueba de texto -> respuesta -> voz.^[237] ^[239]

Consejo importante

Para el MVP local te vale tener la key en inspector, pero en cuanto esto empiece a moverse conviene sacar esas llamadas a un gateway backend propio. Además, para el siguiente hito ya puedes añadir un `ISTTService` basado en OpenAI transcriptions, porque OpenAI también te cubre esa pieza cuando toque pasar de input manual a voz real.^[238] ^[239]

Si quieres, en el siguiente mensaje te pego aquí mismo el contenido de los tres archivos clave para que no tengas ni que abrir los artefactos: `OpenAIConversationController.cs`, `OpenAIChatClient.cs` y `OpenAITTSCient.cs`.

si, ponme todos archivos que tengo que añadir a mi proyecto o modificar para continuar por este camino

Sí. Te pongo aquí todos los archivos que necesitas añadir ahora mismo para seguir por el camino OpenAI-only, dejando el chat desacoplado para sustituirlo después por tu LLM custom. OpenAI te cubre chat/completions y text-to-speech dentro de la misma API, y he dejado el controlador dependiendo de interfaces para que luego solo cambies la implementación de chat. [\[252\]](#) [\[253\]](#) [\[254\]](#)

Archivos a añadir

Crea una carpeta, por ejemplo:

Assets/_Project/Code/Conversation/

y mete estos archivos dentro. La UI usa TextMeshPro, y el loop se apoya en un AudioSource para reproducir la respuesta del avatar. [\[255\]](#) [\[256\]](#)

1) ConversationContracts.cs

```
using System;
using System.Collections;

namespace MVP.Conversation
{
    public interface IChatService
    {
        IEnumerator RequestChat(string userMessage, Action<string, string> onComplete)
    }

    public interface ITTSService
    {
        IEnumerator RequestSpeech(string text, Action<UnityEngine.AudioClip, string> onComplete)
    }

    public interface ISTTService
    {
        IEnumerator Transcribe(byte[] audioBytes, Action<string, string> onComplete)
    }
}
```

2) ConversationStateTypes.cs

```
using System;

namespace MVP.Conversation
{
    public enum ConversationState
```

```

    {
        Idle,
        Processing,
        Speaking,
        Error
    }

    [Serializable]
    public class ConversationTiming
    {
        public double requestStartTime;
        public double llmResponseTime;
        public double ttsReadyTime;
        public double playbackStartTime;

        public double LlmSeconds => llmResponseTime - requestStartTime;
        public double TtsSeconds => ttsReadyTime - llmResponseTime;
        public double TimeToPlaybackSeconds => playbackStartTime - requestStartTime
    }

    [Serializable]
    public class ConversationResult
    {
        public string userText;
        public string assistantText;
        public string error;
        public ConversationTiming timing = new ConversationTiming();
    }
}

```

3) ConversationDebugView.cs

```

using TMPro;
using UnityEngine;

namespace MVP.Conversation
{
    public class ConversationDebugView : MonoBehaviour
    {
        [SerializeField] private TMP_Text stateText;
        [SerializeField] private TMP_Text debugText;

        public void SetState(string value)
        {
            if (stateText != null)
                stateText.text = value;
        }

        public void SetDebug(string value)
        {
            if (debugText != null)
                debugText.text = value;
        }
    }
}

```

```
}  
}
```

4) OpenAIChatClient.cs

Este es el proveedor temporal de chat. Usa chat/completions con model y messages, pero está detrás de IChatService para que luego lo sustituyas por tu cliente custom. [\[257\]](#) [\[252\]](#)

```
using System;  
using System.Collections;  
using System.Text;  
using UnityEngine;  
using UnityEngine.Networking;  
  
namespace MVP.Conversation  
{  
    public class OpenAIChatClient : MonoBehaviour, IChatService  
    {  
        [Header("OpenAI Chat")]  
        [SerializeField] private string apiKey = "YOUR_OPENAI_API_KEY";  
        [SerializeField] private string endpoint = "https://api.openai.com/v1/chat/c  
        [SerializeField] private string model = "gpt-4o-mini";  
        [SerializeField] [TextArea(3, 8)]  
        private string systemPrompt = "You are a historical character speaking insid  
        [SerializeField] private float temperature = 0.8f;  
  
        public IEnumerator RequestChat(string userMessage, Action<string, string> on  
        {  
            var body = new ChatCompletionRequest  
            {  
                model = model,  
                temperature = temperature,  
                messages = new Message[]  
                {  
                    new Message { role = "system", content = systemPrompt },  
                    new Message { role = "user", content = userMessage }  
                }  
            };  
  
            string json = JsonUtility.ToJson(body);  
            byte[] bodyRaw = Encoding.UTF8.GetBytes(json);  
  
            using var request = new UnityWebRequest(endpoint, UnityWebRequest.kHttpV  
            request.uploadHandler = new UploadHandlerRaw(bodyRaw);  
            request.downloadHandler = new DownloadHandlerBuffer();  
            request.SetRequestHeader("Content-Type", "application/json");  
            request.SetRequestHeader("Authorization", $"Bearer {apiKey}");  
  
            yield return request.SendWebRequest();  
  
            if (request.result != UnityWebRequest.Result.Success)  
            {  
                onComplete?.Invoke(null, request.error + "\n" + request.downloadHanc  
                yield break;  
            }  
        }  
    }  
}
```

```

    }

    string responseText = request.downloadHandler.text;
    var parsed = JsonUtility.FromJson<ChatCompletionResponse>(responseText);

    if (parsed == null || parsed.choices == null || parsed.choices.Length == 0)
    {
        onComplete?.Invoke(null, "Respuesta inválida de OpenAI Chat: " + responseText);
        yield break;
    }

    onComplete?.Invoke(parsed.choices[^19_0].message.content, null);
}

[Serializable]
private class ChatCompletionRequest
{
    public string model;
    public float temperature;
    public Message[] messages;
}

[Serializable]
private class Message
{
    public string role;
    public string content;
}

[Serializable]
private class ChatCompletionResponse
{
    public Choice[] choices;
}

[Serializable]
private class Choice
{
    public Message message;
}
}
}

```

5) OpenAITTSClient.cs

Este usa POST /v1/audio/speech. OpenAI documenta ese endpoint y soporta formatos como wav y mp3; aquí lo dejamos en wav porque para Unity y lipsync suele ser más cómodo para arrancar. [\[253\]](#) [\[258\]](#)

```

using System;
using System.Collections;
using System.IO;
using System.Text;
using UnityEngine;

```

```

using UnityEngine.Networking;

namespace MVP.Conversation
{
    public class OpenAITTSCient : MonoBehaviour, ITTSService
    {
        [Header("OpenAI TTS")]
        [SerializeField] private string apiKey = "YOUR_OPENAI_API_KEY";
        [SerializeField] private string endpoint = "https://api.openai.com/v1/audio";
        [SerializeField] private string model = "gpt-4o-mini-tts";
        [SerializeField] private string voice = "coral";
        [SerializeField] [TextArea(2, 5)]
        private string instructions = "Speak in warm, natural Spanish, with clear diction";
        [SerializeField] private string responseFormat = "wav";

        public IEnumerator RequestSpeech(string text, Action<AudioClip, string> onComplete)
        {
            var body = new OpenAITtsRequest
            {
                model = model,
                input = text,
                voice = voice,
                instructions = instructions,
                response_format = responseFormat
            };

            string json = JsonUtility.ToJson(body);
            byte[] bodyRaw = Encoding.UTF8.GetBytes(json);

            using var request = new UnityWebRequest(endpoint, UnityWebRequest.kHttpVerbPOST);
            request.uploadHandler = new UploadHandlerRaw(bodyRaw);
            request.downloadHandler = new DownloadHandlerBuffer();
            request.SetRequestHeader("Content-Type", "application/json");
            request.SetRequestHeader("Authorization", $"Bearer {apiKey}");

            yield return request.SendWebRequest();

            if (request.result != UnityWebRequest.Result.Success)
            {
                onComplete?.Invoke(null, request.error + "\n" + request.downloadHandler.text);
                yield break;
            }

            byte[] audioBytes = request.downloadHandler.data;
            if (audioBytes == null || audioBytes.Length == 0)
            {
                onComplete?.Invoke(null, "OpenAI TTS devolvió audio vacío");
                yield break;
            }

            string extension = responseFormat.ToLowerInvariant() == "mp3" ? ".mp3" : ".wav";
            AudioType audioType = responseFormat.ToLowerInvariant() == "mp3" ? AudioType.MP3 : AudioType.WAV;
            string filePath = Path.Combine(Application.temporaryCachePath, $"openai_tts_{text}_{extension}");

            try
            {
                File.WriteAllBytes(filePath, audioBytes);
            }
            catch { }

            AudioClip clip = AudioClip.Create(filePath, audioBytes.Length / 2, 1, 48000, false);
            clip.SetData(audioBytes, 0);

            onComplete?.Invoke(clip, filePath);
        }
    }
}

```

```

        File.WriteAllBytes(filePath, audioBytes);
    }
    catch (Exception e)
    {
        onComplete?.Invoke(null, "No se pudo escribir el archivo TTS temporal: " + e.Message);
        yield break;
    }

    using var fileRequest = UnityWebRequestMultimedia.GetAudioClip("file://" + filePath);
    yield return fileRequest.SendWebRequest();

    if (fileRequest.result != UnityWebRequest.Result.Success)
    {
        onComplete?.Invoke(null, "Error cargando AudioClip temporal: " + fileRequest.error);
        yield break;
    }

    AudioClip clip = DownloadHandlerAudioClip.GetContent(fileRequest);
    if (clip == null)
    {
        onComplete?.Invoke(null, "No se pudo crear AudioClip desde la respuesta");
        yield break;
    }

    onComplete?.Invoke(clip, null);
}

[Serializable]
private class OpenAITtsRequest
{
    public string model;
    public string input;
    public string voice;
    public string instructions;
    public string response_format;
}
}
}

```

6) OpenAIConversationController.cs

Este es el controlador principal. No depende directamente de OpenAI chat, sino de IChatService y ITTSService, así que luego podrás cambiar solo el bloque de chat. [\[254\]](#)

```

using System.Collections;
using TMPPro;
using UnityEngine;
using UnityEngine.UI;

namespace MVP.Conversation
{
    public class OpenAIConversationController : MonoBehaviour
    {
        [Header("Service Behaviours")]
    }
}

```

```

[SerializeField] private MonoBehaviour chatServiceBehaviour;
[SerializeField] private MonoBehaviour ttsServiceBehaviour;

[Header("Scene References")]
[SerializeField] private AudioSource avatarAudioSource;
[SerializeField] private TMP_InputField inputField;
[SerializeField] private Button sendButton;
[SerializeField] private ConversationDebugView debugView;

[Header("Debug")]
[SerializeField] private bool logToConsole = true;
[SerializeField] private string fallbackPrompt = "Preséntate como un person";

private IChatService chatService;
private ITTSService ttsService;
private ConversationState currentState = ConversationState.Idle;
private bool isBusy;

private void Awake()
{
    chatService = chatServiceBehaviour as IChatService;
    ttsService = ttsServiceBehaviour as ITTSService;

    if (chatService == null)
        Debug.LogError("chatServiceBehaviour no implementa IChatService");
    if (ttsService == null)
        Debug.LogError("ttsServiceBehaviour no implementa ITTSService");

    if (sendButton != null)
        sendButton.onClick.AddListener(SendFromInputField);

    SetState(ConversationState.Idle);
}

public void SendFromInputField()
{
    string text = inputField != null && !string.IsNullOrEmpty(inputField.text)
        ? inputField.text.Trim()
        : fallbackPrompt;

    SendText(text);
}

public void SendText(string text)
{
    if (isBusy || chatService == null || ttsService == null)
        return;

    StartCoroutine(RunConversation(text));
}

private IEnumerator RunConversation(string userText)
{
    isBusy = true;
    SetInteractable(false);
    SetState(ConversationState.Processing);
}

```

```

var result = new ConversationResult { userText = userText };
result.timing.requestStartTime = Time.realtimeSinceStartupAsDouble;

WriteDebug($"USER: {userText}\n\nConsultando servicio de chat...");

string assistantResponse = null;
string chatError = null;
yield return chatService.RequestChat(userText, (text, error) =>
{
    assistantResponse = text;
    chatError = error;
});

result.timing.llmResponseTime = Time.realtimeSinceStartupAsDouble;

if (!string.IsNullOrEmpty(chatError))
{
    result.error = chatError;
    SetState(ConversationState.Error);
    WriteDebug($"USER: {userText}\n\nERROR CHAT:\n{chatError}");
    FinishRequest();
    yield break;
}

result.assistantText = assistantResponse;
WriteDebug($"USER: {userText}\n\nAI: {assistantResponse}\n\nGenerando voz");

AudioClip clip = null;
string ttsError = null;
yield return ttsService.RequestSpeech(assistantResponse, (audioClip, error) =>
{
    clip = audioClip;
    ttsError = error;
});

result.timing.ttsReadyTime = Time.realtimeSinceStartupAsDouble;

if (!string.IsNullOrEmpty(ttsError))
{
    result.error = ttsError;
    SetState(ConversationState.Error);
    WriteDebug($"USER: {userText}\n\nAI: {assistantResponse}\n\nERROR TTS: {ttsError}");
    FinishRequest();
    yield break;
}

SetState(ConversationState.Speaking);
avatarAudioSource.Stop();
avatarAudioSource.clip = clip;
avatarAudioSource.Play();
result.timing.playbackStartTime = Time.realtimeSinceStartupAsDouble;

WriteDebug(
    $"USER: {result.userText}\n\n" +
    $"AI: {result.assistantText}\n\n" +

```

```

        $"Latencia chat: {result.timing.LlmSeconds:F2}s\n" +
        $"Latencia TTS: {result.timing.TtsSeconds:F2}s\n" +
        $"Tiempo hasta playback: {result.timing.TimeToPlaybackSeconds:F2}s"
    );

    if (logToConsole)
    {
        Debug.Log($"[OpenAIConversation] CHAT={result.timing.LlmSeconds:F2}s");
        Debug.Log($"[OpenAIConversation] USER: {result.userText}");
        Debug.Log($"[OpenAIConversation] AI: {result.assistantText}");
    }

    while (avatarAudioSource.isPlaying)
        yield return null;

    SetState(ConversationState.Idle);
    FinishRequest();
}

private void FinishRequest()
{
    isBusy = false;
    SetInteractable(true);
}

private void SetInteractable(bool value)
{
    if (sendButton != null)
        sendButton.interactable = value;
    if (inputField != null)
        inputField.interactable = value;
}

private void SetState(ConversationState state)
{
    currentState = state;
    if (debugView != null)
        debugView.SetState($"State: {currentState}");
}

private void WriteDebug(string text)
{
    if (debugView != null)
        debugView.SetDebug(text);
}
}
}

```

Qué modificar en tu escena

En Unity crea o configura esto:

- `GameObject ConversationSystem`
 - `OpenAIConversationController`

- `OpenAIChatClient`
- `OpenAITTSCient`
- Avatar con `AudioSource`
- `TMP_InputField`
- `Button`
- `ConversationDebugView`
- Dos `TMP_Text`: uno para estado y otro para debug [\[255\]](#)

En el inspector:

- `chatServiceBehaviour` -> arrastra `OpenAIChatClient`
- `ttsServiceBehaviour` -> arrastra `OpenAITTSCient`
- `avatarAudioSource` -> el `AudioSource` del avatar
- `inputField` -> tu `TMP_InputField`
- `sendButton` -> tu botón
- `debugView` -> el componente `ConversationDebugView`

Qué modificar respecto a lo anterior

Si ya tenías la versión con Perplexity:

- elimina o deja de usar `PerplexityClient.cs`
- elimina o deja de usar `SimpleTTSCient.cs`
- sustituye el controlador anterior por `OpenAIConversationController`
- usa estos contratos nuevos para dejar el chat intercambiable [\[253\]](#) [\[254\]](#)

Dos cosas que te recomiendo corregir ya

Hay dos mejoras que yo haría enseguida para evitar problemas:

- **Parser JSON más robusto:** `JsonUtility` es limitado para APIs reales; para OpenAI chat te recomendaría migrar a `Newtonsoft.Json` en la siguiente iteración. [\[252\]](#)
- **API key centralizada:** ahora mismo la meterás en inspector por velocidad, pero mejor luego moverla a config local o backend. [\[259\]](#) [\[253\]](#)

Orden para continuar hoy

Haz esto en este orden:

1. Añade estos 6 archivos.
2. Asigna la API key en `OpenAIChatClient` y `OpenAITTSCient`.
3. Monta la escena.
4. Prueba una frase fija.

5. Cuando eso funcione, conecta lipsync al mismo AudioSource.[\[256\]](#) [\[260\]](#) [\[253\]](#)

Siguiente paso después de esto

En cuanto tengas esto respondiendo, lo siguiente lógico es:

- añadir envío con Enter,
- poner varias frases de test,
- y preparar `OpenAISTTClient` para reemplazar el input manual por voz.[\[261\]](#)

Si quieres, en el siguiente mensaje te doy también la checklist exacta de escena y wiring en Unity paso a paso, para que no te falte ni una referencia en el inspector.

✱

Instalar Newtonsoft.Json en Unity

Para mejorar el parsing JSON del MVP, instala el paquete oficial de Unity `com.unity.nuget.newtonsoft-json`. Unity publica este paquete oficialmente y lo documenta como su distribución de `Newtonsoft.Json` para Unity, con soporte para serialización flexible y parsing robusto de JSON.[\[262\]](#) [\[263\]](#)

Instalación rápida

Opción 1: Package Manager

1. Abre Window > Package Manager.
2. Pulsa el botón +.
3. Elige Add package by name....
4. Escribe:

```
com.unity.nuget.newtonsoft-json
```

5. Instálalo.

Esa vía está alineada con la instalación oficial vía UPM del paquete mantenido por Unity.[\[264\]](#) [\[265\]](#)

Opción 2: manifest.json

Añade esto a `Packages/manifest.json`:

```
{
  "dependencies": {
    "com.unity.nuget.newtonsoft-json": "3.2.1"
  }
}
```

Unity documenta la rama 3.x de este paquete como su distribución moderna de Newtonsoft.Json para Unity.^[263]

Por qué conviene aquí

JsonUtility es demasiado limitado para respuestas reales de APIs como OpenAI Chat Completions, donde la respuesta tiene objetos anidados, arrays y metadatos.

Newtonsoft.Json es más apropiado para leer de forma robusta

choices[^20_0].message.content y errores JSON de la API.ConversationStateTypes.cs, ConversationDebugView.cs ni OpenAIConversationController.cs`. Solo:^[266] ^[262]

- instalar Newtonsoft,
- reemplazar OpenAIChatClient,
- reemplazar OpenAITTSClient.^[262] ^[263]

Mi recomendación práctica

Hazlo así:

- instala primero com.unity.nuget.newtonsoft-json,
- recompila,
- sustituye esos dos archivos,
- y prueba otra vez el flujo texto -> chat -> TTS.^[265] ^[266] ^[267]

Si quieres, en el siguiente mensaje te pego aquí directamente el contenido final de esos dos scripts ya corregidos con Newtonsoft, para que puedas hacer copy-paste sin abrir los archivos.

*
**

Mejora JSON para Unity con Newtonsoft.Json

La versión anterior del cliente de chat usaba JsonUtility, que en Unity es limitada para payloads y respuestas JSON complejas. Para integraciones con OpenAI suele ser más robusto usar Newtonsoft.Json, y Unity ofrece el paquete oficial com.unity.nuget.newtonsoft-json, actualmente documentado por Unity como paquete oficial basado en Newtonsoft.Json 13.0.2.^[278]

Qué instalar en Unity

Opción recomendada: Package Manager

1. Abre Window > Package Manager.
2. Pulsa el botón +.
3. Elige Add package by name.
4. Escribe:

```
com.unity.nuget.newtonsoft-json
```

La documentación oficial de Unity para este paquete confirma esa distribución oficial del paquete Newtonsoft para Unity.^[278]

Qué archivo debes sustituir

Sustituye tu archivo actual:

- `OpenAIChatClient.cs`

por esta nueva versión:

- `OpenAIChatClient_Newtonsoft.cs`

Luego puedes renombrar el archivo a `OpenAIChatClient.cs` para mantener el mismo nombre en el proyecto.

Por qué mejora esto

- `JsonConvert.SerializeObject` y `DeserializeObject` manejan mejor estructuras anidadas que `JsonUtility`.^[279]
- `OpenAI Chat Completions` devuelve respuestas con múltiples campos adicionales, y usar un parser más flexible reduce errores de parseo.^[280] ^[281]
- El cliente queda más preparado para cambios menores de payload sin romper con tanta facilidad.
ence.Reply briefly, clearly, and with strong personality, but stay easy to understand. Keep responses under 90 words unless asked otherwise.";^[282]
[SerializeField] [Range(0f, 2f)] private float temperature = 0.8f;

```
public IEnumerator RequestChat(string userMessage, Action<string, string> onCompletion)
{
    var body = new ChatCompletionRequest
    {
        model = model,
        temperature = temperature,
        messages = new List<Message>
        {
            new Message { role = "system", content = systemPrompt },
            new Message { role = "user", content = userMessage }
        }
    };

    string json = JsonConvert.SerializeObject(body);
    byte[] bodyRaw = Encoding.UTF8.GetBytes(json);

    using var request = new UnityWebRequest(endpoint, UnityWebRequest.kHttpVerbPOST);
    request.uploadHandler = new UploadHandlerRaw(bodyRaw);
    request.downloadHandler = new DownloadHandlerBuffer();
    request.SetRequestHeader("Content-Type", "application/json");
    request.SetRequestHeader("Authorization", $"Bearer {apiKey}");

    yield return request.SendWebRequest();
}
```

```

        string raw = request.downloadHandler.text;

        if (request.result != UnityWebRequest.Result.Success)
        {
            onComplete?.Invoke(null, request.error + "\n" + raw);
            yield break;
        }

        ChatCompletionResponse parsed;
        try
        {
            parsed = JsonConvert.DeserializeObject<ChatCompletionResponse>(raw);
        }
        catch (Exception e)
        {
            onComplete?.Invoke(null, "Error parseando JSON de OpenAI Chat: " + e.Message);
            yield break;
        }

        string content = parsed?.choices?.message?.content;
        if (string.IsNullOrEmpty(content))
        {
            onComplete?.Invoke(null, "Respuesta vacía o inválida de OpenAI Chat.\n");
            yield break;
        }

        onComplete?.Invoke(content.Trim(), null);
    }

    [Serializable]
    private class ChatCompletionRequest
    {
        public string model;
        public float temperature;
        public List<Message> messages;
    }

    [Serializable]
    private class Message
    {
        public string role;
        public string content;
    }

    [Serializable]
    private class ChatCompletionResponse
    {
        public List<Choice> choices;
    }

    [Serializable]
    private class Choice
    {
        public int index;
        public Message message;
    }

```

```
        public string finish_reason;
    }
}
```

Cómo instalar Newtonsoft en Unity

Haz esto:

```
- `Window > Package Manager`
- botón `+`
- `Add package by name`
- pega:
  ```text
 com.unity.nuget.newtonsoft-json
```

Ese es el paquete oficial documentado por Unity para Newtonsoft.Json. [web:218]

## Qué ganas con esto

Ganas tres cosas claras:

- serialización más fiable del body JSON, [web:223]
- parseo más robusto de la respuesta de OpenAI, [web:201][web:230]
- y mejor tolerancia a cambios menores en la respuesta sin que todo reviente por culpa de JsonUtility. [web:228]

## Qué no hace falta tocar

No necesitas tocar:

- OpenAITTSCClient.cs
- OpenAIConversationController.cs
- ConversationContracts.cs
- ConversationStateTypes.cs
- ConversationDebugView.cs

Solo cambia el cliente de chat. [code\_file:1]

## Mi recomendación

Haz esta mejora ya, antes de seguir montando más piezas, porque te quita una fuente bastante típica de errores tontos en Unity cuando empiezas a depender de APIs externas. [web:218][web:228]

¿Quieres que ahora te prepare también una versión igual de robusta para STT con OpenAI, para que el siguiente paso ya sea pasar de input manual a micrófono?

# ok pero dame el nuevo codigo modificado

Sí, este es el nuevo código modificado de `OpenAIChatClient.cs` usando `Newtonsoft.Json` en vez de `JsonUtility`, que es una opción más robusta para parsear respuestas de OpenAI Chat Completions en Unity. Unity ofrece el paquete oficial `com.unity.nuget.newtonsoft-json`, y el endpoint de chat devuelve la respuesta en la estructura `choices -> message -> content`, que este código ya lee correctamente. [\[294\]](#) [\[295\]](#) [\[296\]](#)

## OpenAIChatClient.cs

```
using System;
using System.Collections;
using System.Collections.Generic;
using System.Text;
using Newtonsoft.Json;
using UnityEngine;
using UnityEngine.Networking;

namespace MVP.Conversation
{
 public class OpenAIChatClient : MonoBehaviour, IChatService
 {
 [Header("OpenAI Chat")]
 [SerializeField] private string apiKey = "YOUR_OPENAI_API_KEY";
 [SerializeField] private string endpoint = "https://api.openai.com/v1/chat/c";
 [SerializeField] private string model = "gpt-4o-mini";
 [SerializeField] [TextArea(3, 8)]
 private string systemPrompt = "You are a historical character speaking inside";
 [SerializeField] [Range(0f, 2f)] private float temperature = 0.8f;

 public IEnumerator RequestChat(string userMessage, Action<string, string> onResponse)
 {
 var body = new ChatCompletionRequest
 {
 model = model,
 temperature = temperature,
 messages = new List<Message>
 {
 new Message { role = "system", content = systemPrompt },
 new Message { role = "user", content = userMessage }
 }
 };

 string json = JsonConvert.SerializeObject(body);
 byte[] bodyRaw = Encoding.UTF8.GetBytes(json);

 using var request = new UnityWebRequest(endpoint, UnityWebRequest.kHttpVerbPOST);
 request.uploadHandler = new UploadHandlerRaw(bodyRaw);
 request.downloadHandler = new DownloadHandlerBuffer();
 request.SetRequestHeader("Content-Type", "application/json");
 request.SetRequestHeader("Authorization", $"Bearer {apiKey}");
```

```

 yield return request.SendWebRequest();

 string raw = request.downloadHandler.text;

 if (request.result != UnityWebRequest.Result.Success)
 {
 onComplete?.Invoke(null, request.error + "\n" + raw);
 yield break;
 }

 ChatCompletionResponse parsed;
 try
 {
 parsed = JsonConvert.DeserializeObject<ChatCompletionResponse>(raw);
 }
 catch (Exception e)
 {
 onComplete?.Invoke(null, "Error parseando JSON de OpenAI Chat: " + e);
 yield break;
 }

 string content = parsed?.choices?[^22_0]?.message?.content;
 if (string.IsNullOrEmpty(content))
 {
 onComplete?.Invoke(null, "Respuesta vacía o inválida de OpenAI Chat.");
 yield break;
 }

 onComplete?.Invoke(content.Trim(), null);
 }

 [Serializable]
 private class ChatCompletionRequest
 {
 public string model;
 public float temperature;
 public List<Message> messages;
 }

 [Serializable]
 private class Message
 {
 public string role;
 public string content;
 }

 [Serializable]
 private class ChatCompletionResponse
 {
 public List<Choice> choices;
 }

 [Serializable]
 private class Choice
 {

```

```

 public int index;
 public Message message;
 public string finish_reason;
 }
}
}

```

## Lo único extra que necesitas

Antes de compilar, instala en Unity el paquete oficial:

```
com.unity.nuget.newtonsoft-json
```

Unity documenta ese paquete como su integración oficial de Newtonsoft.Json para proyectos Unity.<sup>[294]</sup>

## Qué sustituye exactamente

Este archivo reemplaza al `OpenAIChatClient.cs` anterior que usaba:

- `JsonUtility.ToJson(...)`
- `JsonUtility.FromJson(...)`

No necesitas tocar `OpenAITTSCClient.cs` ni `OpenAIConversationController.cs` para esta mejora.<sup>[295]</sup>

## Qué mejora aporta

Con este cambio:

- serializas mejor el body JSON,<sup>[297]</sup>
- parseas mejor respuestas reales de OpenAI,<sup>[296]</sup>
- y además obtienes mensajes de error más útiles con el RAW completo si algo falla.<sup>[298]</sup>

Si quieres, el siguiente paso útil es que te pase también una **versión mejorada de OpenAITTSCClient** con el mismo criterio de robustez y un poco mejor preparada para producción.

✱

# una pregunta... la api key que creo es la misma para todo no ?

Sí: puedes usar la misma API key para todo tu MVP, siempre que esa key tenga permisos para los endpoints que vas a llamar. En OpenAI, una misma clave puede servir para chat, TTS y STT dentro del mismo proyecto, y la autenticación normal de esos endpoints se hace con `Authorization: Bearer OPENAI_API_KEY`.<sup>[299]</sup> <sup>[300]</sup> <sup>[301]</sup>

## Respuesta práctica

Para tu caso ahora mismo, una sola key te vale para:

- `chat/completions`
- `audio/speech`
- `audio/transcriptions`<sup>[300]</sup> <sup>[301]</sup> <sup>[299]</sup>

O sea, sí: puedes poner la misma key en `OpenAIChatClient` y `OpenAITTSClient`, y más adelante también en tu futuro `OpenAISTTClient`.<sup>[301]</sup> <sup>[299]</sup>

## El matiz importante

Que se pueda no significa que sea lo ideal a largo plazo. Las guías actuales recomiendan separar claves por app o entorno cuando puedas, y OpenAI además permite gestionar permisos de las API keys al crearlas o editarlas.<sup>[302]</sup> <sup>[303]</sup> <sup>[304]</sup>

## Lo que te recomiendo a ti

Para avanzar rápido:

- usa una sola key de desarrollo ahora mismo,
- nómbrala algo como `unity-vr-dev`,
- y úsala en chat + TTS.<sup>[302]</sup>

Cuando el proyecto madure:

- crea una key para dev,
- otra para staging o demo,
- y si montas backend, usa credenciales del lado servidor en vez de exponer la misma key en el cliente Unity.<sup>[303]</sup> <sup>[302]</sup>

## Respuesta corta

Así que sí: la misma API key te sirve para todo el pipeline OpenAI de este MVP, siempre que tenga permisos adecuados.<sup>[299]</sup> <sup>[300]</sup> <sup>[301]</sup>

## Mi consejo final

Para hoy no te compliques:

- crea una key,
- prueba chat,
- prueba TTS,
- y sigue con esa misma.<sup>[300]</sup> <sup>[302]</sup>

Luego ya la refinamos cuando metas STT y backend. ¿Quieres que ahora te prepare también el `OpenAISTTClient.cs` para dejar cerrada ya la trilogía chat + tts + stt con la misma key?

# Scaffold para los siguientes pasos: micrófono STT + lipsync con TTS

Este scaffold deja preparada la siguiente fase del MVP: capturar audio con el micrófono de Unity, convertirlo a WAV, enviarlo a OpenAI STT mediante `/v1/audio/transcriptions`, y reproducir el TTS del avatar a través de un `AudioSource` que luego será el mismo origen para Oculus LipSync. OpenAI documenta la transcripción por `multipart/form-data` a `/v1/audio/transcriptions` y acepta wav, mientras que Meta documenta que Oculus LipSync analiza entrada desde micrófono o audio file y produce visemas para animar el rostro. [\[316\]](#) [\[317\]](#) [\[318\]](#)

## Qué archivos incluye

- `MicrophoneRecorder.cs`
- `WavUtility.cs`
- `OpenAISTTClient.cs`
- `LipSyncAvatarBridge.cs`
- `VoiceConversationController.cs`

## Flujo previsto

1. Empieza grabación de micrófono con `Microphone.Start`.
2. Se corta la grabación al soltar el botón o tecla.
3. El `AudioClip` grabado se convierte a bytes WAV.
4. `OpenAISTTClient` enviará esos bytes a OpenAI STT usando `multipart form`, que es justo el formato documentado para `/v1/audio/transcriptions`. [\[317\]](#) [\[316\]](#)
5. El texto resultante irá al servicio de chat desacoplado.
6. El texto de respuesta se enviará al TTS.
7. El `AudioSource` del avatar reproducirá el clip resultante.
8. Oculus LipSync podrá leer ese mismo `AudioSource` mediante `OVRipSyncContext`, como documenta Meta para análisis de audio y generación de visemas. [\[318\]](#) [\[319\]](#)

## Setup de lipsync en escena

Cuando metas el plugin de Oculus LipSync en Unity:

1. Añade un `AudioSource` al avatar.
2. Añade `OVRipSyncContext` al mismo objeto o a un objeto ligado a ese `AudioSource`.
3. Añade `OVRipSyncContextMorphTarget` para mapear visemas al `SkinnedMeshRenderer` facial.

4. Configura los blendshapes del avatar con los visemas requeridos por OVR\_LipSync.

Meta documenta que OVR\_LipSyncContext sirve como interfaz con el motor de lipsync y que debe añadirse una vez por cada AudioSource; también documenta que

OVR\_LipSyncContextMorphTarget hace el puente hacia los morph targets del mesh facial. [\[319\]](#)  
[\[318\]](#)

## Ojo con spatial audio

Meta advierte que la espacialización puede degradar la señal para visemas si se procesa antes del análisis. Si usas audio espacial con lipsync, debes configurar la cadena para que la señal cruda alimente el motor de visemas y el espacializado se aplique después. [\[320\]](#)

## Setup recomendado en escena

Te recomiendo esta estructura:

- ConversationSystem
  - VoiceConversationController
  - OpenAIChatClient
  - OpenAITTSClient
  - OpenAISTTClient
  - MicrophoneRecorder
- Avatar
  - AudioSource
  - OVR\_LipSyncContext
  - OVR\_LipSyncContextMorphTarget
  - LipSyncAvatarBridge [\[318\]](#) [\[319\]](#)

## Detalle importante de lipsync

Hay un punto importante: Meta advierte que si aplicas spatial audio antes del análisis de lipsync, la señal se degrada y empeoran los visemas. Si luego usas espacialización, asegúrate de que el motor de lipsync reciba la señal cruda y el espacializado se aplique después. [\[320\]](#)

## Lo que yo haría mañana mismo

Orden práctico:

1. montar MicrophoneRecorder,
2. probar grabación con tecla Space en editor,
3. importar Oculus LipSync,
4. conectar AudioSource + OVR\_LipSyncContext + MorphTarget,

5. confirmar que el avatar mueve boca con un clip local antes incluso de conectar OpenAI.  
[\[321\]](#) [\[319\]](#) [\[318\]](#)

## Recomendación muy importante

Antes de integrar STT real, haría una prueba con un clip WAV local reproducido por el AudioSource del avatar para validar el lipsync aislado. Eso te quita muchas variables y te permite depurar blendshapes, visemas y rig facial antes de mezclar red, STT y TTS. Meta incluso documenta flujos de análisis desde audio file además de micrófono.[\[322\]](#) [\[318\]](#)

Si quieres, el siguiente paso útil es que te pegue aquí mismo el contenido de los 3 archivos clave de esta nueva fase: `MicrophoneRecorder.cs`, `OpenAISTTClient.cs` y `VoiceConversationController.cs`.

✱✱

## pero para hacer el setup necesito que me pases los ficheros

Sí, claro. Aquí te paso los ficheros completos para que puedas hacer el setup ya. Esta estructura deja preparado el flujo de grabación con `Microphone.Start`, conversión a WAV para el endpoint de transcripciones de OpenAI, y reproducción del TTS sobre un `AudioSource` que luego será el origen del lipsync con `Oculus LipSync`.[\[25\\_1\]](#)[\[25\\_2\]](#)[\[25\\_3\]](#)

### Ficheros a añadir

Añádelos en la misma carpeta de tu sistema de conversación, por ejemplo:

`Assets/_Project/Code/Conversation/Voice/`

#### 1) `MicrophoneRecorder.cs`

```
using System;
using UnityEngine;

namespace MVP.Conversation
{
 public class MicrophoneRecorder : MonoBehaviour
 {
 [Header("Microphone")]
 [SerializeField] private string selectedDevice;
 [SerializeField] private int sampleRate = 16000;
 [SerializeField] private int maxLengthSeconds = 10;
 [SerializeField] private bool loopRecording = false;

 public bool IsRecording { get; private set; }
 public AudioClip CurrentClip { get; private set; }

 public void StartRecording()
 {
```

```

 if (IsRecording)
 return;

 if (Microphone.devices == null || Microphone.devices.Length == 0)
 {
 Debug.LogError("No hay micrófonos disponibles en el dispositivo.");
 return;
 }

 if (string.IsNullOrEmpty(selectedDevice))
 selectedDevice = Microphone.devices[^25_0];

 CurrentClip = Microphone.Start(selectedDevice, loopRecording, maxLength,
 IsRecording = CurrentClip != null;
 }

 public AudioClip StopRecording()
 {
 if (!IsRecording)
 return null;

 int position = Microphone.GetPosition(selectedDevice);
 Microphone.End(selectedDevice);
 IsRecording = false;

 if (CurrentClip == null || position <= 0)
 return null;

 float[] samples = new float[position * CurrentClip.channels];
 CurrentClip.GetData(samples, 0);

 AudioClip trimmedClip = AudioClip.Create(
 CurrentClip.name + "_trimmed",
 position,
 CurrentClip.channels,
 CurrentClip.frequency,
 false);

 trimmedClip.SetData(samples, 0);
 return trimmedClip;
 }

 public string[] GetDevices()
 {
 return Microphone.devices;
 }
}

```

## 2) WavUtility.cs

```
using System;
using System.IO;
using UnityEngine;

namespace MVP.Conversation
{
 public static class WavUtility
 {
 public static byte[] FromAudioClip(AudioClip clip)
 {
 if (clip == null)
 return null;

 float[] samples = new float[clip.samples * clip.channels];
 clip.GetData(samples, 0);

 short[] intData = new short[samples.Length];
 byte[] bytesData = new byte[samples.Length * 2];

 const float rescaleFactor = 32767f;
 for (int i = 0; i < samples.Length; i++)
 {
 intData[i] = (short)(samples[i] * rescaleFactor);
 byte[] byteArr = BitConverter.GetBytes(intData[i]);
 byteArr.CopyTo(bytesData, i * 2);
 }

 using var stream = new MemoryStream();
 using var writer = new BinaryWriter(stream);

 int byteRate = clip.frequency * clip.channels * 2;
 int subChunk2Size = bytesData.Length;
 int chunkSize = 36 + subChunk2Size;

 writer.Write(System.Text.Encoding.UTF8.GetBytes("RIFF"));
 writer.Write(chunkSize);
 writer.Write(System.Text.Encoding.UTF8.GetBytes("WAVE"));
 writer.Write(System.Text.Encoding.UTF8.GetBytes("fmt "));
 writer.Write(16);
 writer.Write((short)1);
 writer.Write((short)clip.channels);
 writer.Write(clip.frequency);
 writer.Write(byteRate);
 writer.Write((short)(clip.channels * 2));
 writer.Write((short)16);
 writer.Write(System.Text.Encoding.UTF8.GetBytes("data"));
 writer.Write(subChunk2Size);
 writer.Write(bytesData);

 writer.Flush();
 return stream.ToArray();
 }
 }
}
```

```
}
}
```

### 3) OpenAISTTClient.cs

Este cliente deja preparado STT con el endpoint de transcripciones de OpenAI, que acepta multipart/form-data con archivo de audio y campos como model, response\_format y language.  
[25\_2][25\_4]

```
using System;
using System.Collections;
using System.Text;
using Newtonsoft.Json;
using UnityEngine;
using UnityEngine.Networking;

namespace MVP.Conversation
{
 public class OpenAISTTClient : MonoBehaviour, ISTTService
 {
 [Header("OpenAI STT")]
 [SerializeField] private string apiKey = "YOUR_OPENAI_API_KEY";
 [SerializeField] private string endpoint = "https://api.openai.com/v1/audio";
 [SerializeField] private string model = "gpt-4o-mini-transcribe";
 [SerializeField] private string responseFormat = "text";
 [SerializeField] private string language = "es";

 public IEnumerator Transcribe(byte[] audioBytes, Action<string, string> onComplete)
 {
 if (audioBytes == null || audioBytes.Length == 0)
 {
 onComplete?.Invoke(null, "No hay audio para transcribir.");
 yield break;
 }

 byte[] boundary = Encoding.UTF8.GetBytes("-----" + Date
WWWForm form = new WWWForm();
form.AddBinaryData("file", audioBytes, "microphone.wav", "audio/wav");
form.AddField("model", model);
form.AddField("response_format", responseFormat);
form.AddField("language", language);

 using var request = UnityWebRequest.Post(endpoint, form);
 request.SetRequestHeader("Authorization", $"Bearer {apiKey}");

 yield return request.SendWebRequest();

 string raw = request.downloadHandler.text;
 if (request.result != UnityWebRequest.Result.Success)
 {
 onComplete?.Invoke(null, request.error + "\n" + raw);
 yield break;
 }
 }
 }
}
```

```

 if (responseFormat == "text")
 {
 onComplete?.Invoke(raw?.Trim(), null);
 yield break;
 }

 try
 {
 var parsed = JsonConvert.DeserializeObject<TranscriptionResponse>(raw);
 onComplete?.Invoke(parsed?.text?.Trim(), null);
 }
 catch (Exception e)
 {
 onComplete?.Invoke(null, "Error parseando respuesta STT: " + e.Message);
 }
 }

 [Serializable]
 private class TranscriptionResponse
 {
 public string text;
 }
}

```

#### 4) LipSyncAvatarBridge.cs

Esto es un bridge sencillo para dejarte preparado el wiring del avatar con AudioSource, contexto de lipsync y mesh facial. Meta documenta que el contexto de lipsync se asocia al audio analizado y el morph target context aplica los visemas al SkinnedMeshRenderer.<sup>[25\_3]</sup>  
[25\_5]

```

using UnityEngine;

namespace MVP.Conversation
{
 public class LipSyncAvatarBridge : MonoBehaviour
 {
 [Header("Runtime References")]
 [SerializeField] private AudioSource avatarAudioSource;
 [SerializeField] private MonoBehaviour lipSyncContextBehaviour;
 [SerializeField] private SkinnedMeshRenderer faceRenderer;

 [Header("Optional Debug")]
 [SerializeField] private bool logWarnings = true;

 private void Awake()
 {
 if (avatarAudioSource == null && logWarnings)
 Debug.LogWarning("LipSyncAvatarBridge: falta AudioSource del avatar.");

 if (lipSyncContextBehaviour == null && logWarnings)
 Debug.LogWarning("LipSyncAvatarBridge: falta referencia a OVR_LipSync");
 }
 }
}

```

```

 if (faceRenderer == null && logWarnings)
 Debug.LogWarning("LipSyncAvatarBridge: falta SkinnedMeshRenderer fac
 }

 public void BindAudioSource(AudioSource source)
 {
 avatarAudioSource = source;
 }

 public AudioSource GetAudioSource()
 {
 return avatarAudioSource;
 }
}
}

```

## 5) VoiceConversationController.cs

Este es el controlador de la siguiente fase: grabar voz, pasar por STT, luego chat, luego TTS, y reproducir la respuesta en el avatar.<sup>[25\_6]</sup><sub>[25\_2]</sub>

```

using System.Collections;
using TMPPro;
using UnityEngine;
using UnityEngine.UI;

namespace MVP.Conversation
{
 public class VoiceConversationController : MonoBehaviour
 {
 [Header("Services")]
 [SerializeField] private MonoBehaviour sttServiceBehaviour;
 [SerializeField] private MonoBehaviour chatServiceBehaviour;
 [SerializeField] private MonoBehaviour ttsServiceBehaviour;

 [Header("Recording")]
 [SerializeField] private MicrophoneRecorder microphoneRecorder;
 [SerializeField] private Button holdToTalkButton;
 [SerializeField] private bool useKeyboardDebugShortcut = true;
 [SerializeField] private KeyCode keyboardDebugKey = KeyCode.Space;

 [Header("Playback")]
 [SerializeField] private AudioSource avatarAudioSource;
 [SerializeField] private ConversationDebugView debugView;

 private ISTTService sttService;
 private IChatService chatService;
 private ITTSService ttsService;
 private bool isBusy;
 private bool isHoldingRecord;

 private void Awake()
 {
 sttService = sttServiceBehaviour as ISTTService;

```

```

 chatService = chatServiceBehaviour as IChatService;
 ttsService = ttsServiceBehaviour as ITTSService;
 }

 private void Update()
 {
 if (!useKeyboardDebugShortcut)
 return;

 if (Input.GetKeyDown(keyboardDebugKey))
 BeginRecording();

 if (Input.GetKeyUp(keyboardDebugKey))
 EndRecordingAndSend();
 }

 public void BeginRecording()
 {
 if (isBusy || microphoneRecorder == null)
 return;

 isHoldingRecord = true;
 microphoneRecorder.StartRecording();
 debugView?.SetState("State: Recording");
 debugView?.SetDebug("Grabando desde micrófono...");
 }

 public void EndRecordingAndSend()
 {
 if (!isHoldingRecord || isBusy || microphoneRecorder == null)
 return;

 isHoldingRecord = false;
 AudioClip clip = microphoneRecorder.StopRecording();
 if (clip == null)
 {
 debugView?.SetState("State: Error");
 debugView?.SetDebug("No se pudo obtener audio del micrófono.");
 return;
 }

 byte[] wavBytes = WavUtility.FromAudioClip(clip);
 StartCoroutine(RunVoiceConversation(wavBytes));
 }

 private IEnumerator RunVoiceConversation(byte[] wavBytes)
 {
 isBusy = true;
 debugView?.SetState("State: Processing");
 debugView?.SetDebug("Transcribiendo audio...");

 string userText = null;
 string sttError = null;
 yield return sttService.Transcribe(wavBytes, (text, error) =>
 {
 userText = text;

```

```

 sttError = error;
 });

 if (!string.IsNullOrEmpty(sttError) || string.IsNullOrEmpty(userText))
 {
 debugView?.SetState("State: Error");
 debugView?.SetDebug("ERROR STT:\n" + sttError);
 isBusy = false;
 yield break;
 }

 debugView?.SetDebug("USER: " + userText + "\n\nConsultando chat...");

```

<div align="center">✖</div>

[^25\_1]: <https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Microphone.S>  
 [^25\_2]: <https://platform.openai.com/docs/guides/speech-to-text>  
 [^25\_3]: <https://developers.meta.com/horizon/documentation/unity/audio-ovr lipsync-unity>  
 [^25\_4]: <https://developers.openai.com/api/docs/guides/speech-to-text>  
 [^25\_5]: <https://github.com/avatarsdk/metaperson-oculus-unity-sample>  
 [^25\_6]: <https://developers.openai.com/api/docs/guides/text-to-speech>

---

# te has quedado a medias parece

Sí, se quedó cortado. Aquí te pongo **completo** lo que faltaba, y además te dejo o

## VoiceConversationController.cs completo

```

````csharp
using System.Collections;
using TMPPro;
using UnityEngine;
using UnityEngine.UI;

namespace MVP.Conversation
{
    public class VoiceConversationController : MonoBehaviour
    {
        [Header("Services")]
        [SerializeField] private MonoBehaviour sttServiceBehaviour;
        [SerializeField] private MonoBehaviour chatServiceBehaviour;
        [SerializeField] private MonoBehaviour ttsServiceBehaviour;

        [Header("Recording")]
        [SerializeField] private MicrophoneRecorder microphoneRecorder;
        [SerializeField] private Button holdToTalkButton;
        [SerializeField] private bool useKeyboardDebugShortcut = true;
        [SerializeField] private KeyCode keyboardDebugKey = KeyCode.Space;

        [Header("Playback")]
        [SerializeField] private AudioSource avatarAudioSource;
    }
}

```

```

[SerializeField] private ConversationDebugView debugView;

private ISTTService sttService;
private IChatService chatService;
private ITTSService ttsService;
private bool isBusy;
private bool isHoldingRecord;

private void Awake()
{
    sttService = sttServiceBehaviour as ISTTService;
    chatService = chatServiceBehaviour as IChatService;
    ttsService = ttsServiceBehaviour as ITTSService;
}

private void Update()
{
    if (!useKeyboardDebugShortcut)
        return;

    if (Input.GetKeyDown(keyboardDebugKey))
        BeginRecording();

    if (Input.GetKeyUp(keyboardDebugKey))
        EndRecordingAndSend();
}

public void BeginRecording()
{
    if (isBusy || microphoneRecorder == null)
        return;

    isHoldingRecord = true;
    microphoneRecorder.StartRecording();
    debugView?.SetState("State: Recording");
    debugView?.SetDebug("Grabando desde micrófono...");
}

public void EndRecordingAndSend()
{
    if (!isHoldingRecord || isBusy || microphoneRecorder == null)
        return;

    isHoldingRecord = false;
    AudioClip clip = microphoneRecorder.StopRecording();
    if (clip == null)
    {
        debugView?.SetState("State: Error");
        debugView?.SetDebug("No se pudo obtener audio del micrófono.");
        return;
    }

    byte[] wavBytes = WavUtility.FromAudioClip(clip);
    StartCoroutine(RunVoiceConversation(wavBytes));
}

```

```

private IEnumerator RunVoiceConversation(byte[] wavBytes)
{
    isBusy = true;
    debugView?.SetState("State: Processing");
    debugView?.SetDebug("Transcribiendo audio...");

    string userText = null;
    string sttError = null;
    yield return sttService.Transcribe(wavBytes, (text, error) =>
    {
        userText = text;
        sttError = error;
    });

    if (!string.IsNullOrEmpty(sttError) || string.IsNullOrEmpty(userText))
    {
        debugView?.SetState("State: Error");
        debugView?.SetDebug("ERROR STT:\n" + sttError);
        isBusy = false;
        yield break;
    }

    debugView?.SetDebug("USER: " + userText + "\n\nConsultando chat...");

    string assistantText = null;
    string chatError = null;
    yield return chatService.RequestChat(userText, (text, error) =>
    {
        assistantText = text;
        chatError = error;
    });

    if (!string.IsNullOrEmpty(chatError) || string.IsNullOrEmpty(assistantText))
    {
        debugView?.SetState("State: Error");
        debugView?.SetDebug("ERROR CHAT:\n" + chatError);
        isBusy = false;
        yield break;
    }

    debugView?.SetDebug("USER: " + userText + "\n\nAI: " + assistantText + "\n\n");

    AudioClip ttsClip = null;
    string ttsError = null;
    yield return ttsService.RequestSpeech(assistantText, (clip, error) =>
    {
        ttsClip = clip;
        ttsError = error;
    });

    if (!string.IsNullOrEmpty(ttsError) || ttsClip == null)
    {
        debugView?.SetState("State: Error");
        debugView?.SetDebug("ERROR TTS:\n" + ttsError);
        isBusy = false;
        yield break;
    }
}

```

```

    }

    debugView?.SetState("State: Speaking");
    debugView?.SetDebug("USER: " + userText + "\n\nAI: " + assistantText);

    avatarAudioSource.Stop();
    avatarAudioSource.clip = ttsClip;
    avatarAudioSource.Play();

    while (avatarAudioSource.isPlaying)
        yield return null;

    debugView?.SetState("State: Idle");
    isBusy = false;
}
}
}

```

Los otros ficheros que necesitas junto a este

Para que este controlador compile y tenga sentido en tu proyecto, necesitas tener también estos archivos ya añadidos:

- `MicrophoneRecorder.cs`
- `WavUtility.cs`
- `OpenAISTTClient.cs`
- `ConversationContracts.cs`
- `ConversationDebugView.cs`
- `OpenAIChatClient.cs`
- `OpenAITTSCClient.cs` [\[331\]](#) [\[332\]](#)

Ojo con un detalle

Ahora mismo `holdToTalkButton` está declarado pero todavía no está cableado a eventos de `pointer down/up`. El flujo funcional actual es con la tecla `Space` para pruebas en editor. Eso te permite avanzar hoy con la estructura aunque aún no tengas el botón VR rematado. [\[333\]](#)

Lo más práctico ahora

Para no bloquearte:

- mete todos los scripts,
- compila,
- prueba primero que `Space` graba y corta,
- y luego ya conectamos el botón VR y el `OVRlipSyncContext`. [\[334\]](#) [\[333\]](#)

Si quieres, en el siguiente mensaje te pego también el resto de ficheros completos uno por uno para que tengas ya el pack entero aquí en chat sin depender de los adjuntos.

dime el setup completo en el editor ya que hay ficheros nuevos

Sí. Te dejo el setup completo en el editor para la versión actual con los ficheros nuevos: chat OpenAI temporal, TTS OpenAI, STT desde micrófono y estructura preparada para Oculus LipSync. Unity documenta TMP_InputField, permisos de micrófono en Android y Microphone, y Meta documenta OVR_LipSyncContext y OVR_LipSyncContextMorphTarget para enrutar visemas a blendshapes faciales. [\[335\]](#) [\[336\]](#) [\[337\]](#)

Carpetas y scripts

Primero asegúrate de tener importados estos scripts en tu proyecto:

- ConversationContracts.cs
- ConversationStateTypes.cs
- ConversationDebugView.cs
- OpenAIChatClient.cs
- OpenAITTSClient.cs
- OpenAISTTClient.cs
- MicrophoneRecorder.cs
- WavUtility.cs
- LipSyncAvatarBridge.cs
- VoiceConversationController.cs [\[338\]](#) [\[339\]](#) [\[340\]](#)

Además, instala:

- TextMeshPro Essentials, si Unity te lo pide al crear TMP UI. [\[341\]](#) [\[335\]](#)
- Newtonsoft.Json con com.unity.nuget.newtonsoft-json, porque OpenAIChatClient y OpenAISTTClient lo usan. Unity documenta ese paquete oficial. [\[342\]](#)

Jerarquía de escena

Te recomiendo esta jerarquía mínima:

- XR Origin
- ConversationSystem
- AvatarRoot
- Canvas_Debug
- EventSystem [\[335\]](#)

Dentro de Canvas_Debug:

- `StateText (TextMeshProUGUI)`
- `DebugText (TextMeshProUGUI)`
- `opcional InputField (TMP_InputField)`
- `opcional SendButton (Button)` [\[335\]](#)

Aunque vas hacia voz, yo dejaría `TMP_InputField` y `SendButton` en la escena para debug rápido y fallback. `TMP_InputField` sigue siendo útil para pruebas de prompt y diagnosis. [\[335\]](#)

GameObject ConversationSystem

Crea un empty llamado `ConversationSystem` y añade estos componentes:

- `VoiceConversationController`
- `OpenAIChatClient`
- `OpenAITTSCient`
- `OpenAISTTClient`
- `MicrophoneRecorder` [\[339\]](#) [\[340\]](#)

Configuración de `VoiceConversationController`

Asigna:

- `sttServiceBehaviour -> componente OpenAISTTClient`
- `chatServiceBehaviour -> componente OpenAIChatClient`
- `ttsServiceBehaviour -> componente OpenAITTSCient`
- `microphoneRecorder -> componente MicrophoneRecorder`
- `avatarAudioSource -> AudioSource del avatar`
- `debugView -> componente ConversationDebugView`
- `useKeyboardDebugShortcut -> true para probar en editor`
- `keyboardDebugKey -> Space` [\[343\]](#)

`holdToTalkButton` puedes dejarlo vacío por ahora, porque el script actual funciona con teclado para pruebas. `TMP_InputField` y `Button` tienen eventos Unity estándar, pero en esta fase el flujo principal es mantener pulsado `Space`. [\[344\]](#) [\[335\]](#)

Configuración de `MicrophoneRecorder`

Valores recomendados:

- `sampleRate = 16000`
- `maxLengthSeconds = 10`
- `loopRecording = false`
- `selectedDevice = ""` para que coja el primero [\[345\]](#) [\[339\]](#)

Eso te da un clip corto y ligero, razonable para STT. OpenAI documenta audio transcriptions por archivo y no necesitas todavía grabaciones largas para el MVP.^[339]

GameObject Canvas_Debug

Crea un Canvas en World Space o Screen Space Overlay, como prefieras para debug interno. Dentro añade:

- un TextMeshPro - Text para estado
- un TextMeshPro - Text para debug^[335]

Crea un GameObject ConversationDebugView y añade el componente ConversationDebugView.cs. Asigna:

- stateText -> StateText
- debugText -> DebugText

Este panel te mostrará estados como Recording, Processing, Speaking y errores de STT/chat/TTS.^[343]

AvatarRoot

En tu avatar crea o localiza el objeto que debe emitir la voz y añade:

- AudioSource
- LipSyncAvatarBridge
- luego, cuando importes el plugin, OVRipSyncContext
- y OVRipSyncContextMorphTarget^[336] ^[346]

Configuración de AudioSource

Valores recomendados para empezar:

- Play On Awake = false
- Loop = false
- Spatial Blend = 1 si quieres voz 3D desde el personaje
- Spatial Blend = 0 temporalmente si quieres depurar audio sin variables extra
- Volume = 1^[336]

Configuración de LipSyncAvatarBridge

Asigna:

- avatarAudioSource -> el AudioSource del avatar
- lipSyncContextBehaviour -> déjalo vacío hasta importar Oculus LipSync
- faceRenderer -> el SkinnedMeshRenderer de la cabeza/cara del avatar^[343]

Setup de Oculus LipSync

Cuando importes Oculus LipSync, Meta indica que debes usar un `OVR_LipSyncContext` para procesar el audio y `OVR_LipSyncContextMorphTarget` para aplicar los 15 visemas a blendshapes del mesh facial. [\[336\]](#)

Añade estos componentes al avatar

- `OVR_LipSyncContext`
- `OVR_LipSyncContextMorphTarget` [\[336\]](#)

En `OVR_LipSyncContext`

Asigna el `AudioSource` del avatar, porque Meta documenta que el contexto debe enlazarse al audio que va a analizar. [\[346\]](#)

En `OVR_LipSyncContextMorphTarget`

Asigna:

- `Skinned Mesh Renderer` -> malla facial del avatar
- `Viseme To Blend Targets` -> índices de blendshapes correspondientes [\[346\]](#) [\[336\]](#)

Meta y ejemplos de integración usan este orden de visemas:

- `sil`
- `PP`
- `FF`
- `TH`
- `DD`
- `kk`
- `CH`
- `SS`
- `nn`
- `RR`
- `aa`
- `E`
- `IH`
- `oh`
- `ou` [\[346\]](#)

Si tu avatar no tiene esos blendshapes o no están bien mapeados, el lipsync no funcionará bien aunque el audio sí. [\[336\]](#)

Permisos Android y Quest

Como vas a usar micrófono en Quest, Unity añade `RECORD_AUDIO` cuando usas la clase `Microphone`, y documenta que solicita permiso la primera vez que se usa en Android.^[337]

Aun así, yo te recomiendo comprobar:

- `Player Settings > Android > Other Settings`
- que el proyecto tiene acceso a internet
- y probar en dispositivo real que aparece el prompt de micrófono.^[347] ^[337]

Si quieres control explícito, puedes pedir el permiso por código con `UnityEngine.Android.Permission`. Unity documenta ese flujo también.^[347]

Modo de prueba recomendado

Haz las pruebas por fases, así:

1. **Compila todo sin key** y confirma que no hay errores de scripts.
2. **Prueba solo `MicrophoneRecorder` con Space**: pulsa, habla, suelta; verifica que no rompe y que el flujo pasa por `RunVoiceConversation`.^[345]
3. **Prueba solo audio local + lipsync**: mete un WAV local en el `AudioSource` y confirma que los visemas funcionan antes de tocar red. Meta documenta `sample scenes` justo para validar esto.^[336]
4. Cuando tengas API key, activa:
 - STT
 - Chat
 - TTS ^[340] ^[338] ^[339]

Dos escenas útiles

Yo haría dos escenas separadas:

- `VoicePipeline_Debug`: canvas visible, tecla Space, textos de debug.
- `VR_Character_Main`: escena real limpia con el avatar final.

Eso te deja depurar la tubería sin contaminar la escena de presentación.^[335]

Qué no hace falta conectar aún

Todavía no necesitas:

- botón VR hold-to-talk real,
- ray interactors,
- hand tracking,
- UI diegética final,

- spatial audio fino.

Primero deja funcionando:

- micro,
- STT,
- chat,
- TTS,
- lipsync.[\[339\]](#) [\[336\]](#)

Checklist final

En el editor deberías acabar con esto:

- ConversationSystem con 5 componentes: VoiceConversationController, OpenAIChatClient, OpenAITTSCClient, OpenAISTTClient, MicrophoneRecorder.
- Canvas_Debug con ConversationDebugView.
- AvatarRoot con AudioSource, LipSyncAvatarBridge, y luego OVR_LipSyncContext + OVR_LipSyncContextMorphTarget.
- Newtonsoft.Json instalado.
- TextMeshPro importado.
- Script test con tecla Space.[\[342\]](#) [\[335\]](#) [\[336\]](#)

Mi consejo práctico

Antes de tocar STT real, haz una prueba de lipsync con clip local y otra de micro sin red. Te va a ahorrar mucho tiempo de debugging mezclado y te permite aislar si falla el rig facial, el micro o la API.[\[345\]](#) [\[336\]](#)

Si quieres, en el siguiente mensaje te hago una checklist ultra concreta tipo “clic a clic” del inspector, componente por componente, incluyendo qué campo arrastrar en cada slot.

✱✱

ayudame a encontrar un avatar gratis en internet que me sirva para las pruebas y tambien los steps para importar oculus lipsync

Sí. Para pruebas rápidas y gratis, la opción más práctica hoy es usar un avatar de VRoid/VRM o un avatar tipo Ready Player Me con blendshapes/visemes, porque necesitas algo que tenga cara riggeada y morph targets útiles para lipsync. Meta documenta Oculus LipSync como sistema basado en visemas y morph targets, y VRChat incluso recalca que los avatars con visemes/blendshapes son el flujo normal para lipsync facial.[\[356\]](#) [\[357\]](#) [\[358\]](#)

Avatar recomendado

Yo te recomiendo este orden:

Opción 1: VRoid / VRM

La más segura para pruebas. VRoid es gratis y está pensada justo para crear avatares humanoides con blendshapes faciales y exportarlos como VRM, algo muy útil para pruebas de lipsync. Varias referencias actuales la siguen señalando como la vía sencilla para sacar avatares riggeados con expresiones.^[359] ^[360]

Ventajas:

- Gratis.^[359]
- Cara y expresiones preparadas mejor que muchos modelos genéricos.^[359]
- Muy buena opción para prototipo aunque el look sea anime.^[360]

Opción 2: Ready Player Me

Era una buena opción para prototipos rápidos, y hay referencias de avatares gratuitos exportables. Además, hay documentación comunitaria sobre cómo descargar GLB con morph targets ARKit/Oculus Visemes en la URL.^[361] ^[362]

Peeero: hay señales de cambios/cierre en 2026 en contenido de comunidad, así que para no perder tiempo yo no la pondría como primera opción ahora.^[361]

Opción 3: Avatar gratuito de Sketchfab / sample avatar

Puede servir, pero el riesgo es que no traiga blendshapes faciales útiles o que el rig facial esté incompleto para lipsync.^[356] ^[362]

Mi recomendación real

Para no bloquearte, haría esto:

- crea un avatar gratuito en VRoid Studio,
- expórtalo a VRM,
- impórtalo en Unity con UniVRM,
- y úsalo para el primer test de AudioSource + OVRlipSyncContext + morph targets.^[359]
^[360]

Eso es probablemente el camino con menos fricción para ti ahora mismo.^[359]

Pasos para conseguir el avatar

Camino VRoid

1. Descarga VRoid Studio y crea un avatar base. VRoid se describe como herramienta gratuita para crear avatares humanoides compatibles con VRM. [\[359\]](#)
2. Exporta el avatar en formato VRM. [\[360\]](#)
3. En Unity, importa UniVRM para poder cargar VRM correctamente. La comunidad lo usa justo para este flujo de avatares VRM en Unity. [\[361\]](#) [\[363\]](#)
4. Arrastra el .vrn al proyecto y genera el prefab del avatar. [\[363\]](#)

Oculus LipSync import

Meta documenta el flujo oficial de importación de Oculus LipSync en Unity así:

1. Descarga el paquete de Oculus LipSync Unity desde la página oficial. [\[357\]](#)
2. Extrae el ZIP. [\[357\]](#)
3. En Unity ve a Assets > Import Package > Custom Package. [\[357\]](#)
4. Selecciona el archivo OVR_LipSync.unitypackage dentro de LipSync/UnityPlugin. [\[357\]](#)
5. Deja todo seleccionado y pulsa Import. Meta además recomienda quitar versiones anteriores antes de importar una nueva. [\[357\]](#)

Setup de Oculus LipSync en escena

Meta indica que para usar el plugin debes tener en la escena el script principal OVR_LipSync, y que te deja un prefab de conveniencia llamado LipSyncInterface en Assets > Oculus > LipSync > Prefabs. [\[358\]](#)

Haz esto:

1. Añade a la escena el prefab LipSyncInterface. [\[358\]](#)
2. En tu avatar, añade un AudioSource que reproducirá el TTS. [\[358\]](#)
3. Añade OVR_LipSyncContext al objeto adecuado para analizar esa fuente de audio. Meta indica que es la interfaz principal con el motor de lipsync. [\[358\]](#)
4. Añade OVR_LipSyncContextMorphTarget para mapear visemas a blendshapes del SkinnedMeshRenderer. [\[357\]](#) [\[358\]](#)
5. Asigna el SkinnedMeshRenderer facial del avatar. [\[358\]](#)
6. Rellena el mapping de visemas a blendshapes según los nombres o índices del modelo. [\[356\]](#) [\[358\]](#)

Qué comprobar en el avatar

Antes de perder tiempo, abre el mesh facial y verifica que tiene blendshapes útiles. En flujos de visemas, Unity puede incluso eliminar blendshapes vacíos al importar, y VRChat advierte específicamente que el shape SIL no debe estar completamente vacío para evitar que Unity lo borre. [\[356\]](#)

Eso significa:

- revisa que la cara tenga blendshapes,
- revisa que la boca tenga varios shapes distintos,
- y no asumas que cualquier humanoide FBX te va a servir. [\[356\]](#)

Orden que te recomiendo

Hazlo así:

1. Crear/importar avatar VRM. [\[359\]](#)
2. Confirmar que tiene blendshapes faciales. [\[356\]](#)
3. Importar Oculus LipSync. [\[357\]](#)
4. Añadir LipSyncInterface. [\[358\]](#)
5. Añadir AudioSource + OVRLipSyncContext + OVRLipSyncContextMorphTarget. [\[358\]](#)
6. Reproducir un clip WAV local en el AudioSource antes de conectar OpenAI. [\[358\]](#)

Mi consejo práctico

Si quieres ir a lo más rápido posible, usa VRoid + UniVRM + Oculus LipSync. Es gratis, razonablemente predecible para pruebas y más adecuado que ponerte a cazar un FBX aleatorio de internet sin saber si trae blendshapes faciales. [\[359\]](#) [\[357\]](#) [\[360\]](#)

¿Quieres que ahora te dé el paso a paso exacto para importar un VRM en Unity 6 y luego conectar OVRLipSyncContextMorphTarget al avatar?

✱

pero en VRoid studio se ven como de anime ? no veo que tengan animaciones de lipsync... como configuro eso en el programa

Sí: VRoid Studio es claramente estilo anime, así que si buscas un personaje histórico más realista, te sirve solo como avatar técnico de pruebas, no como look final. Además, VRoid no te “configura lipsync realtime” dentro del programa; lo que te da es el avatar con expresiones/blendshapes exportables en VRM, y el lipsync se resuelve después en Unity o en otra herramienta que consuma esos blendshapes. [\[371\]](#) [\[372\]](#) [\[373\]](#)

Qué hace VRoid realmente

VRoid te sirve para:

- crear un humanoide gratis,
- exportarlo como VRM,
- y conservar expresiones faciales/base blendshapes.^[371] ^[372]

Pero el lipsync realtime no se “activa” en VRoid Studio como si fuera una opción final del programa. La sincronización labial se hace luego en el runtime o en un pipeline externo, usando visemas o shapes como A I U E O en VRM o visemas tipo Oculus/VRChat tras el import.^[372] ^[374]

El punto importante

Para tu proyecto con Oculus LipSync, lo que necesitas no es que VRoid “anime la voz”, sino que el avatar tenga:

- un SkinnedMeshRenderer facial,
- blendshapes útiles,
- y una forma razonable de mapear visemas.^[375] ^[376]

Meta documenta que Oculus LipSync genera visemas en runtime y los aplica mediante morph targets; por tanto, el trabajo importante está en Unity, no en VRoid Studio.^[376] ^[375]

Entonces, ¿cómo se configura?

Si usas VRoid tal cual

No configuras lipsync dentro de VRoid Studio como feature de runtime. Exportas el avatar y luego, en Unity, haces una de estas dos cosas:

- usar shapes ya presentes del VRM, normalmente tipo A I U E O,^[372]
- o convertir/complementar el modelo para tener visemas más adecuados al sistema que uses, por ejemplo Oculus/VRChat, normalmente en Blender u otro paso intermedio.^[377]
^[374]

Lo que probablemente te pase con VRoid

Un VRoid exportado puede traerte expresiones y shapes básicas, pero no necesariamente un set perfecto de visemas Oculus listo para enchufar. Las referencias de comunidad muestran justo ese problema: muchas veces hay que ajustar shape keys, blendshape proxy o mappings después del export/import.^[378] ^[379] ^[380]

Recomendación honesta

Para tu caso, yo separaría dos objetivos:

Objetivo A: probar el pipeline técnico ya

Usa VRoid aunque sea anime. Te sirve para validar:

- import,
- AudioSource,
- OVRLipSyncContext,
- morph target mapping,
- flujo de TTS/lipsync.^[375] ^[376]

Objetivo B: avatar visual final

Más adelante cambia a un avatar más realista, ya con el pipeline probado.^[381]

Si quieres un avatar no anime

Entonces VRoid no es la mejor elección visual. Hay referencias actuales que la describen justamente como herramienta de avatars anime, y hasta comparativas de 2026 la posicionan así frente a opciones más realistas.^[381] ^[373]

En tu caso, para un personaje histórico, yo haría:

- VRoid para prueba técnica inmediata, o
- buscar un humanoide gratis más realista con blendshapes faciales ya incluidos, sabiendo que eso suele ser menos inmediato y más incierto.^[374] ^[382]

La respuesta corta

Así que:

- sí, VRoid se ve anime,^[371] ^[373]
- no, el lipsync realtime no se configura realmente dentro de VRoid Studio,^[372]
- y la configuración importante se hace después, en Unity, mapeando los blendshapes del modelo al sistema de lipsync.^[375] ^[376]

Qué haría yo

Si tu prioridad es avanzar hoy:

1. usa VRoid solo como dummy técnico,
2. valida Oculus LipSync en Unity,
3. y luego sustituyes el avatar por uno más realista.^[381] ^[375]

Si tu prioridad es que ya se vea presentable desde el principio, entonces mejor no pierdas tiempo con VRoid y te ayudo a buscar un avatar humano gratis más realista con blendshapes faciales. [\[374\]](#) [\[381\]](#)

¿Quieres que te busque ahora mismo 3 opciones concretas de avatar gratuito más realista que podamos intentar importar en Unity para tus pruebas?



vroid no tiene ninguna animacion facial siempre sale con boca cerrada... buscame un modelo que me valga para mi prueba

Sí: para tu prueba necesitas un modelo que ya traiga blendshapes faciales, no solo un humanoide bonito. La mejor pista de búsqueda es exactamente esa: modelos con “blendshapes” o, mejor aún, un sample ya pensado para Oculus LipSync. Hay dos caminos razonables: usar un sample de MetaPerson + Oculus LipSync ya preparado, o bajar un modelo gratuito de Sketchfab que indique explícitamente “blendshapes”. [\[389\]](#) [\[390\]](#) [\[391\]](#)

Lo que te recomiendo

La opción más sólida para no perder tiempo es MetaPerson Oculus LipSync sample. Ese repositorio está hecho justo para Unity + Oculus LipSync, y describe que el avatar de la escena ya se anima con Oculus LipSync, incluyendo el wiring de `OVRipSyncContextMorphTarget` para cabeza y dientes inferiores. [\[390\]](#)

Si lo que quieres es un modelo descargable suelto y gratuito, la mejor opción encontrada es este de Sketchfab:

- Realistic Avatar in HQ with Blendshapes

<https://sketchfab.com/3d-models/realistic-avatar-in-hq-with-blendshapes-88b42ee84e5749138f1b0507e6b9e5a9>

El autor lo describe como avatar realista gratuito con blendshapes y rigged, que es justo lo que necesitas para lipsync. [\[391\]](#)

Mi ranking práctico

Opción	Recomendación	Motivo
MetaPerson Oculus sample	La mejor para arrancar ya. [390]	Ya está pensado para Oculus LipSync y te evita adivinar mappings. [390] [392]
Sketchfab “Realistic Avatar in HQ with Blendshapes”	Buena segunda opción. [391]	Es gratis y explícitamente dice que tiene blendshapes. [391]
Buscar modelos genéricos por tag blendshapes o visemes	Solo como plan B. [389] [393]	Hay mucho ruido y puedes perder tiempo con modelos que no encajen bien. [389] [394]

Mi recomendación real

Yo haría esto:

1. probaría primero el **MetaPerson sample**, porque te da una referencia funcional de cómo debe quedar el setup completo con Oculus LipSync; [\[390\]](#) [\[392\]](#)
2. si quieres quedarte con un avatar más neutro o realista sin depender tanto del sample, probaría el modelo de Sketchfab con blendshapes. [\[391\]](#)

Por qué evitaría seguir a ciegas con modelos aleatorios

Aunque en Sketchfab haya tags como blendshape, blendshapes o visemes, eso no te garantiza:

- que el modelo traiga un set útil de boca,
- que el **SkinnedMeshRenderer** esté limpio,
- o que el mapping de visemas sea razonable. [\[389\]](#) [\[394\]](#) [\[393\]](#)

En cambio, el sample de MetaPerson ya documenta que configura específicamente los arrays **visemeToBlendTargets** para cabeza y dientes. [\[390\]](#)

Lo que haría yo hoy

Si quieres avanzar esta tarde:

- clona o descarga el sample de **MetaPerson Oculus Unity sample**, [\[390\]](#)
- mira cómo está hecho el setup de head mesh + lower teeth mesh, [\[390\]](#)
- y si te convence, lo reutilizas o copias el patrón en tu proyecto principal. [\[392\]](#) [\[390\]](#)

Respuesta corta

Sí, he encontrado una opción que sí te vale para la prueba:

- **MetaPerson Oculus LipSync sample** para ir a tiro fijo. [\[390\]](#)
- **Realistic Avatar in HQ with Blendshapes** de Sketchfab si prefieres un asset descargable gratuito. [\[391\]](#)

Siguiente paso

Yo te recomiendo empezar por **MetaPerson sample**, porque además te enseña cómo configurar Oculus LipSync de verdad en Unity. [\[390\]](#) [\[392\]](#)

¿Quieres que te haga ahora un paso a paso concreto de una de estas dos rutas?

- Ruta A: montar el sample de MetaPerson en Unity.
- Ruta B: descargar el modelo de Sketchfab e importarlo a tu proyecto para probar lipsync.

la empresa me pide que haga un escrito para el cliente donde les pregunte que tipo de LLM conversacional están haciendo para saber como tengo que estructurar yo el código para cuando hagas el reemplazo del chat conversacional de open ai sea facil... entonces crea dicho texto bastante resumido de la arquitectura actual y dí que esperamos algo como un endpoint, y que si eso es lo que nos van a dar? La idea es tener esto claro por si altera a nuestro diseño.

La persona que lo va a leer es un perfil técnico así que no debería tener problema. Igualmente no te extiendas mucho. Tiene que caber en un correo

Te dejo un texto que puedes pegar casi tal cual en el correo:

Buenas,

Estamos avanzando con el MVP del personaje conversacional en VR y, de cara a integrar vuestro LLM a medida más adelante, necesitamos aclarar cómo será el punto de integración para no tomar decisiones de arquitectura que luego haya que rehacer.

En este momento tenemos montado el flujo conversacional así:

- Captura de voz del usuario (micrófono en dispositivo).
- STT (speech-to-text) → obtenemos texto puro de lo que dice el usuario.
- Llamada a un servicio de chat (ahora mismo OpenAI) → recibe el texto del usuario + algo de contexto y devuelve el texto de respuesta del personaje.
- TTS (text-to-speech) → convertimos esa respuesta en audio para el avatar (lipsync incluido).

Nuestro objetivo es que la parte “servicio de chat” sea intercambiable. Idealmente, vuestro LLM encajaría como un servicio al que podamos llamar desde Unity a través de un **endpoint** HTTP (REST o similar), de forma que:

- reciba al menos:
 - el texto del usuario,
 - el contexto necesario (por ejemplo, identificador de personaje, idioma y opcionalmente algo de historial),
- devuelva:
 - el texto de respuesta del personaje,

- opcionalmente metadatos (estado emocional, tags, intención, etc.) que podamos usar en el futuro para animaciones o lógica adicional.

¿Podéis confirmarnos si vuestra solución nos proporcionará algo de este estilo (un endpoint HTTP que podamos invocar desde el cliente, o desde un pequeño backend nuestro), o si tenéis en mente otro tipo de interfaz (SDK propio, WebSocket, gRPC, colas, etc.)?

Cualquier detalle adicional sobre el contrato de esa API (formato de request/response, autenticación, límites de longitud del input, etc.) nos ayudará ahora a diseñar la capa de abstracción para que el reemplazo de OpenAI por vuestro modelo sea lo más directo posible.

Gracias y quedamos atentos.

pero que contexto se envia cuando dices que enviamos algo de contexto en el servicio de chat ?

Cuando digo “algo de contexto” me refiero a la información adicional que el servicio de chat necesita para responder como ese personaje y no como un asistente genérico. En APIs tipo chat, el contexto normalmente se modela como instrucciones del sistema, historial reciente y metadatos de sesión, porque la respuesta no depende solo del último texto del usuario sino también del rol/persona y del estado de la conversación. [\[404\]](#) [\[405\]](#) [\[406\]](#)

En tu caso concreto

Para este proyecto, el contexto mínimo razonable sería algo así:

- **Identidad del personaje:** quién es, época, tono, límites de conocimiento, estilo de habla. Esto equivale a la capa de “persona” o instrucciones del sistema. [\[407\]](#) [\[408\]](#)
- **Idioma y reglas de salida:** por ejemplo “responder en español”, “máximo 2-3 frases”, “tono conversacional”, “sin inventar datos”. Ese tipo de instrucciones suele ir en system/developer message. [\[405\]](#) [\[409\]](#)
- **Historial reciente:** las últimas interacciones user/assistant, para que el modelo mantenga continuidad y recuerde a qué se refería el usuario hace un turno. OpenAI documenta precisamente que el estado conversacional se mantiene reenviando el historial o una representación del mismo. [\[410\]](#) [\[406\]](#)
- **Metadatos de sesión:** characterId, sessionId, idioma, quizá nivel de detalle o modo de experiencia. Esto no siempre va dentro del prompt; a veces viaja como campos aparte en tu propia API. [\[411\]](#) [\[412\]](#)

Lo que yo no enviaría al principio

Para el MVP, no enviaría un contexto enorme ni una base histórica completa en cada request. Lo normal es mandar:

- instrucciones fijas del personaje,
- últimas 3-6 intervenciones,

- y algunos metadatos compactos.

Eso encaja con las prácticas normales de gestión de contexto y evita inflar coste/latencia.^[413]
^[406]

Un ejemplo muy simple

Por ejemplo, tu servicio de chat podría recibir algo así:

```
{
  "sessionId": "abc-123",
  "characterId": "napoleon",
  "language": "es",
  "systemContext": "Eres Napoleón Bonaparte. Responde en español, de forma breve, y",
  "history": [
    { "role": "user", "content": "¿Quién eres?" },
    { "role": "assistant", "content": "Soy Napoleón Bonaparte..." }
  ],
  "userMessage": "¿Cuál fue tu mayor error?"
}
```

Eso refleja bastante bien cómo se estructuran muchas integraciones conversacionales basadas en mensajes con contexto acumulado.^[406] ^[404] ^[405]

Lo importante para el correo

Si quieres decirlo de forma precisa y breve al cliente, “contexto” en ese texto significa:

- configuración del personaje,
- historial breve de conversación,
- y metadatos de sesión.^[407] ^[406]

Frase que puedes meter

Podrías sustituir esa parte por esta:

“Cuando hablamos de contexto, nos referimos principalmente a la configuración/persona del personaje, el idioma y un historial breve de turnos previos de la conversación, además de metadatos como identificador de sesión o de personaje.” ^[405] ^[406] ^[407]

Mi recomendación para tu arquitectura

Yo estructuraría tu capa de chat para que acepte siempre estas 4 piezas, aunque al principio algunas vayan vacías:

- `userMessage`
- `characterContext`
- `conversationHistory`

- sessionMetadata

Así luego te dará igual si el backend del cliente usa OpenAI, un LLM propio o una capa RAG encima.^[413] ^[406]

Si quieres, te reescribo ahora el correo entero con esta aclaración metida y más fino para enviar.



entonces no es probable que ellos simplemente iban a usar openAI con un contexto bien definido? en lugar de crear un custom gpt o algo así?

Sí, de hecho eso es bastante probable. Lo más normal en una primera fase no es que estén entrenando un modelo desde cero ni que tengan algo “ultra custom”, sino que estén montando una capa sobre un LLM existente —muy posiblemente OpenAI u otro proveedor— con un prompt/contexto bien definido, algo de historial, quizá retrieval/RAG, y una API propia delante. En productos conversacionales reales, esa suele ser la forma más rápida y razonable de llegar a algo usable.^[419] ^[420] ^[421]

Lo importante conceptualmente

Hay tres niveles muy distintos:

- Usar OpenAI con buen contexto/prompt: lo más probable y rápido.^[422] ^[419]
- Montar una API propia encima de un proveedor: también muy probable, porque les da control de autenticación, logging, contexto y futura sustitución del modelo.^[421] ^[423]
- Entrenar un modelo realmente custom: mucho menos probable para una primera versión, porque es bastante más costoso y complejo.^[421]

Sobre “custom GPT”

Además, un Custom GPT de ChatGPT no suele ser lo que se integra tal cual en una app Unity por API. OpenAI diferencia el producto GPTs dentro de ChatGPT del uso programático vía API, y la documentación/help reciente separa claramente GPTs de Assistants/API. Incluso OpenAI tiene artículos específicos tipo “GPTs vs Assistants”, y ha estado migrando la parte de Assistants hacia Responses/API, lo que refuerza que “Custom GPT” y “API integrable en tu app” no son exactamente la misma cosa.^[424] ^[425] ^[426]

Lo más probable en tu cliente

La apuesta más realista es esta:

- internamente pueden estar usando OpenAI u otro LLM base,^[427]
- con una persona/contexto del personaje histórico,^[422]
- quizá con historial y documentos/contexto adicional,^[420]

- y exponiéndote a ti un **endpoint propio** para desacoplar su backend de tu cliente Unity.
[\[423\]](#)

Eso, de hecho, es justo lo que a ti te conviene, porque te da un contrato claro y te evita acoplarte a su proveedor real. [\[423\]](#) [\[421\]](#)

Entonces, ¿qué deberías preguntar?

No tanto “¿habéis creado un custom GPT?”, porque eso puede llevar a confusión de producto. Lo más útil para tu arquitectura es preguntar:

- si te van a exponer un **endpoint**,
- qué **input** espera,
- qué **output** devuelve,
- y si internamente usan un LLM base con contexto, RAG o una capa propia. [\[419\]](#) [\[423\]](#)

Respuesta corta

Así que sí: es muy probable que simplemente vayan a usar un LLM base como OpenAI con un contexto bien definido y una capa propia encima, más que un “modelo custom” en el sentido fuerte. [\[419\]](#) [\[421\]](#) [\[422\]](#)

Cómo lo diría yo internamente

Yo lo resumiría así:

“Lo más probable es que su solución no sea un modelo entrenado desde cero, sino una API propia orquestando un LLM base con instrucciones, contexto e historial. Para nosotros, lo relevante no es el proveedor interno sino el contrato técnico del endpoint.” [\[420\]](#) [\[421\]](#) [\[423\]](#)

Si quieres, te reescribo ahora el correo para que haga esa pregunta de forma más precisa y sin usar el término “custom GPT”, que aquí puede confundir bastante.

✱

reescribe entonces tambien diciendo que si lo que van a proporcionar es solo el contexto para dar las pautas al avatar o si un endpoint o que ?

Te propongo esta versión de correo, ajustada a lo que comentas:

Buenas,

Estamos avanzando con el MVP del personaje conversacional en VR y, de cara a integrar vuestro LLM a medida, necesitamos tener claro qué tipo de integración nos vais a proporcionar para evitar cambios de arquitectura posteriores.

Actualmente nuestro flujo es:

- Capturamos la voz del usuario (micrófono).
- Aplicamos STT para obtener el texto.
- Llamamos a un servicio de chat (ahora OpenAI) que recibe el texto del usuario + un pequeño contexto del personaje y devuelve el texto de respuesta.
- Aplicamos TTS sobre esa respuesta para que el avatar la pronuncie (con lipsync sobre el modelo 3D).

Nuestra intención es que la pieza “servicio de chat” sea intercambiable. Para poder diseñar bien esa capa de abstracción, necesitamos saber:

- ¿Vuestra solución nos proporcionará **únicamente un conjunto de directrices/contexto** (por ejemplo, instrucciones del personaje, reglas de estilo, límites de conocimiento) para que lo apliquemos nosotros sobre un proveedor de LLM (como OpenAI),
o
- ¿tenéis previsto exponernos **un endpoint propio** (REST/WebSocket/otra interfaz) al que podamos enviar:
 - texto del usuario,
 - contexto necesario (identificador de personaje, idioma, parte del historial, etc.), y del que recibamos directamente el texto de respuesta del personaje?

En caso de optar por un endpoint, nos ayudaría también saber a alto nivel qué esperáis en el request (campos mínimos) y qué devolveríais en el response (solo texto o también metadatos como estado emocional, tags, etc.), para asegurarnos de que el diseño de nuestra capa de integración no se queda corto.

Gracias y quedamos atentos a vuestra respuesta.

me han respondido esto:

Hola equipo,

Gracias por adelantar las preguntas — responderlas ahora nos ahorra refactors.

Opción: endpoint propio, no solo directrices. Motivo: el servicio de chat incorpora RAG sobre un corpus del personaje que yo mantengo (ingesta, embeddings, retrieval, orquestación de prompt, guardrails, versionado). Ese peso no tiene sentido que lo absorba vuestra capa — vosotros llamáis con el texto del usuario y yo devuelvo la respuesta lista para TTS.

Contrato propuesto (lo afinamos al integrar):

Request — POST /chat

```
{
  "session_id": "uuid",    // gestionado por vosotros; historial lo mantengo server-side
  "user_text": "string",   // salida del STT
  "character_id": "string", // por si hay múltiples personajes
  "metadata": {
    "locale": "es-ES",
    "user_id": "opcional"
```

```

}
}
Response
{
  "session_id": "uuid",
  "response_text": "string", // input para vuestro TTS
  "emotion": "neutral|happy|thinking|concerned|...",
  "intent_tags": ["greeting", "knowledge_answer", "fallback", "out_of_scope"],
  "sources": [{"title": "...", "score": 0.87}], // opcional, trazabilidad
  "metadata": {
    "latency_ms": 1240,
    "model": "...",
    "rag_hits": 3
  }
}

```

Notas de diseño:

Estado conversacional: lo gestiono server-side vía `session_id`. No necesitáis pasar historial salvo que queráis forzar reset (POST /session/reset).

Streaming: os recomiendo que integréis también POST /chat/stream vía SSE o WebSocket.

Podéis arrancar el TTS con los primeros tokens y recortar 1-2 s de latencia percibida si vuestro TTS soporta streaming de entrada.

`emotion` e `intent_tags`: os permiten disparar animaciones/expresiones del avatar sin parsear el texto. Si tenéis ya una lista de emociones que el rig soporta, pasádmela y alineamos el vocabulario — mejor que yo me adapte a vuestro enum que al revés.

`metadata` extensible en request y response para añadir campos sin romper el contrato.

Auth: Bearer token estático para MVP; OAuth2 si pasamos a producción multi-cliente.

Una pregunta al hilo: ¿qué TTS estáis usando y soporta export de visemas con timings, o hacéis audio-to-viseme sobre la waveform? Lo pregunto porque es el factor que determina si el pipeline STT→chat→TTS clásico es lo óptimo (sí lo es si tenéis visemas del TTS) o si conviene explorar alternativas. Mi recomendación por defecto es mantener este pipeline: las arquitecturas speech-to-speech end-to-end tipo Realtime API complican mucho la inyección de RAG y el lipsync viseme-based, así que no las veo para este MVP.

Cuando tengáis feedback decidme tiempos por favor.

Muchas gracias

explícame si rompe algo de nuestro de diseño, y que cambios habría que hacer y que proponen básicamente

No rompe tu diseño, en realidad encaja bastante bien con lo que ya estabas montando: tu capa de chat sigue siendo un servicio intercambiable (`IChatService`), solo que ahora, en vez de llamar a OpenAI directamente, hablarás con su `endpoint` /chat. El resto del pipeline (STT → chat → TTS → lipsync) se mantiene. [\[439\]](#) [\[440\]](#)

Qué están proponiendo exactamente

En resumen, ellos quieren darte:

- Un endpoint propio (POST /chat y versión streaming /chat/stream), no solo “un prompt para OpenAI”.
- Dentro de ese endpoint, ellos se encargan de:
 - RAG sobre el corpus del personaje (ingesta, embeddings, retrieval).
 - Orquestación de prompt, guardrails, versionado, selección de modelo.
 - Manejar el estado de conversación server-side, usando tu session_id.^[441] ^[439]
- Tú solo tienes que enviar:
 - session_id (lo generas/guardas tú).
 - user_text (salida del STT).
 - character_id (por si hay varios personajes).
 - metadata simple (locale, user_id opcional).^[441]
- Ellos te devuelven:
 - response_text (texto listo para TTS).
 - emotion (enum que quieren alinear con tu rig).
 - intent_tags (para disparar diferentes comportamientos).
 - sources (trazabilidad opcional).
 - metadata (latencia, modelo, nº de hits RAG, extensible).^[442] ^[441]

Es decir: ellos se quedan toda la inteligencia conversacional y contexto, tú gestionas voz, VR, animaciones y experiencia.

¿Rompe tu diseño actual?

No, por varias razones:

- Tu IChatService ya está planteado como una interfaz: ahora mismo la implementación es OpenAIChatClient, pero puedes cambiarla por CharacterBackendChatClient que llame a POST /chat sin tocar el resto del código.^[443] ^[441]
- Tu ConversationController ya asume un patrón RequestChat(userText, callback) que devuelve assistantText; solo tendrías que ampliarlo para recibir también emotion e intent_tags y usarlos si quieres.^[439]
- Tu pipeline sigue siendo: STT (local/OpenAI) → servicio de chat → TTS → lipsync. Ellos mismos dicen que este pipeline es el que recomiendan para un MVP, justo como habíamos elegido.^[439]

Lo único que cambia es quién hace el “cerebro” y qué datos adicionales tienes para animaciones.

Cambios que sí tendrás que hacer

1) Cambiar implementación de `IChatService`

Ahora mismo tu `OpenAIChatClient`:

- llama a `https://api.openai.com/v1/chat/completions`,
- manda `system prompt` + mensaje de usuario,
- parsea `choices[0].message.content`.^[440]

Con su propuesta necesitarás un `CharacterBackendChatClient` que:

- mande un `POST` a su `/chat` con `body`:
 - `session_id`
 - `user_text`
 - `character_id`
 - `metadata` (al menos `locale`)
- parse su `JSON`:
 - `response_text`
 - `emotion`
 - `intent_tags`
 - opcionalmente `sources`, `metadata`.^[441]

En tu interfaz puedes seguir devolviendo al controlador, como mínimo, el `response_text` para TTS, y opcionalmente también la emoción/intents.

2) Gestionar `session_id` en tu lado

Ellos quieren manejar contexto server-side, pero necesitan que tú les pases un identificador de sesión:

- cuando empieza la experiencia, generas un `session_id` (UUID),
- lo guardas en memoria,
- lo envías en cada llamada `POST /chat`.^[439] ^[441]

Solo si quieres resetear la sesión, llamarías a su `POST /session/reset` (o lo que definan).^[439]

Tu diseño ya contempla una sesión de usuario VR, así que es solo añadir este campo y no mandar historial en cada request.

3) Adaptar tus “slots” para emoción e intentos

Ellos exponen:

- `emotion: neutral|happy|thinking|concerned|...`
- `intent_tags: p.ej. greeting, knowledge_answer, fallback, out_of_scope`

Esto **no rompe nada**, pero abre la puerta a enriquecer tus animaciones:

- `emotion` ↔ set de expresiones faciales / blendshapes / anim states.
- `intent_tags` ↔ gestos o cambios de postura. [\[444\]](#) [\[442\]](#)

Lo importante que piden es:

“Siténéis ya una lista de emociones que el rig soporta, pasádmela y alineamos el vocabulario”.

Es una **buena señal**: quieren adaptarse a tu enum, no al revés.

Cambios mínimos:

- Define un enum `CharacterEmotion { Neutral, Happy, Thinking, Concerned, ... }`.
- Define un mapping simple `string → enum` en tu cliente.
- Añade hooks en tu controller para notificar al sistema de animación. [\[444\]](#)

4) Considerar `/chat/stream` para futuro

Proponen también `POST /chat/stream` vía SSE o WebSocket. Su idea:

- tú empiezas a recibir tokens de texto parcial,
- alimentas el TTS en streaming,
- y reduces 1-2 segundos de latencia percibida. [\[443\]](#) [\[439\]](#)

Esto no es obligatorio para el MVP, pero sí condiciona el diseño si luego quieres streaming:

- tu `IChatService` podría tener un modo “streaming” con callback incremental,
- en TTS necesitarías soporte de entrada en streaming (o segmentar tú el texto). [\[439\]](#)

Para ahora, puedes:

- implementar solo `POST /chat normal`,
- pero diseñar tu capa de chat con la idea de que en el futuro acepte un modo streaming sin reescribir todo.

5) Autenticación

Para MVP: Bearer token estático.

No te rompe nada; es similar a OpenAI. [\[445\]](#) [\[446\]](#)

Solo implica:

- guardar un token de su backend (no el de OpenAI) en configuración,
- añadir `Authorization: Bearer XXX` en tus requests.

A futuro hablan de OAuth2 multi-cliente, pero eso afecta sobre todo a su backend; para ti sería más un cambio de cómo obtienes el token.

Qué están diciendo sobre TTS y visemas

Te preguntan:

“¿qué TTS estáis usando y soporta export de visemas con timings, o hacéis audio-to-viseme sobre la waveform?”

Lo que están midiendo:

- Si tu TTS da visemas con timings, el pipeline STT→chat→TTS es perfecto: puedes sincronizar lipsync directamente con los datos del TTS. [\[447\]](#)
- Si no, y tú haces lipsync analizando la waveform (que es lo que haces con Oculus LipSync), sigue siendo correcto el pipeline clásico que tenemos. [\[448\]](#) [\[447\]](#)

Ellos mismos dicen:

“Mi recomendación por defecto es mantener este pipeline... las arquitecturas speech-to-speech end-to-end tipo Realtime API complican mucho la inyección de RAG y el lipsync viseme-based, así que no las veo para este MVP.”

Es decir:

- están alineados con nuestro diseño actual (STT → chat → TTS),
- no quieren que os vayáis a un “Realtime API” end-to-end que mezcle todo (voz-voz) porque complica RAG y lipsync.

Resumen en castellano sencillo

- No rompe tu diseño; lo confirma y lo especifica.
- Cambia solo:
 - quién hace el chat (su endpoint en vez de OpenAI directo),
 - cómo pasas contexto (session_id y metadatos en vez de historial completo),
 - y te añade señal extra útil (emotion, intent_tags, sources).
- TTS y lipsync siguen igual; ellos recomiendan explícitamente no liarse con pipelines speech-to-speech end-to-end para este MVP.

Qué haría yo ahora en tu código

1. Mantener `IChatService`.
2. Crear `CharacterBackendChatClient` : `MonoBehaviour`, `IChatService` con:
 - campos `baseUrl`, `authToken`, `characterId`, `locale`.
 - método que haga `POST /chat` y devuelva `response_text` + emoción/intents.
3. Añadir `sessionId` manejado por tu VR app (UUID por sesión).

4. Ajustar `ConversationResult` para guardar también `emotion` e `intentTags`.

5. Añadir un pequeño mapper de `emotion` a tu sistema de animación.

Si quieres, en el siguiente mensaje te puedo escribir el borrador de esa nueva implementación `CharacterBackendChatClient` sobre tu interfaz actual, usando la forma de `request/response` que te han enviado.

✱

Quiero aclaraciones en lo siguiente:

- Por que usar el `session` y `session id` y como implementarlo desde Unity? La aplicacion es algo dinamica que van a ir intercambiando un usuario despues de otro en un congreso sin necesidad de login ni nada... para que necesitamos entonces un `session id`?
- que es eso del RAG ?
- Explicame esa opcion que proponen via streaming de via SSE o WbSocket desde Unity que cambios habria que hacer? no entiendo qu es un modo “streaming” con callback incremental.
Lo que tu mencionas es diseñar la capa de chat con la idea de que en el futuro acepte un modo streaming.
- Explica eso de Bearer token estático

Voy punto por punto, pensando en tu caso concreto (demo en congreso, sin login, Unity standalone).

1) Por qué `session_id` y cómo usarlo en Unity

Qué es y por qué lo quieren

`session_id` es un identificador opaco de conversación. Su backend lo usa para:

- Mantener contexto sin que tú tengas que reenviar historial en cada request.
Con ese `session_id` ellos pueden:
 - recordar turnos anteriores,
 - mantener consistencia del personaje,
 - saber qué ya se ha dicho en esa conversación.^[449] ^[450]
- Separar sesiones cuando cambia la persona que está usando el casco (aunque tú no tengas login).^[449]

Tú **no** necesitas un login para eso. Basta con que cada “turno de usuario” (o cada demo) tenga su propio id de sesión.

Cómo implementarlo en Unity para un congreso

Escenario típico: mucha gente se pone el casco, uno detrás de otro, sin cuenta.

Solución sencilla:

- Cuando arranca la experiencia o cuando pulsas “Empezar” en el menú:
 - generas un Guid (`System.Guid.NewGuid().ToString()`),
 - lo guardas en una variable estática o en un `ScriptableObject/singleton`, tipo `CurrentSessionId`.
- Usas el mismo `session_id` para toda la conversación de esa persona.
- Cuando quieras “resetear” porque se baja el casco y se lo pone otro:
 - generas un nuevo `session_id`,
 - opcionalmente llamas a su POST `/session/reset` si quieren que limpies también su estado.^[451] ^[449]

Ejemplo simple:

```
public static class SessionManager
{
    public static string CurrentSessionId { get; private set; }

    public static void StartNewSession()
    {
        CurrentSessionId = Guid.NewGuid().ToString();
    }
}
```

En tu `CharacterBackendChatClient`, al construir el body:

```
body.session_id = SessionManager.CurrentSessionId;
```

Aunque los usuarios sean anónimos, para el LLM es muy útil saber dónde acaba una conversación y empieza otra. Así no mezcla contextos de personas distintas.^[449]

2) Qué es RAG

RAG = Retrieval-Augmented Generation.

En cristiano: el modelo no responde solo “de memoria”, sino que antes de generar la respuesta:

1. Coge tu pregunta.
2. Busca en un corpus de documentos (bases de datos, PDFs, textos sobre el personaje).
3. Recupera los fragmentos relevantes.
4. Mete esos fragmentos en el prompt junto con la pregunta.

5. Genera la respuesta usando esa información.^[452] ^[449]

Ventajas:

- Respuestas más fieles al contenido real del personaje (biografía, citas).
- Menos alucinaciones.
- Puedes actualizar el contenido sin reentrenar el modelo (añades o cambias documentos).^[452]

Eso es lo que él dice cuando habla de:

“corpus del personaje que yo mantengo (ingesta, embeddings, retrieval, orquestación de prompt, guardrails, versionado)”.

Eso es RAG + capa de orquestación.

Para ti lo importante es: “mi servicio ya hace ese trabajo, tú solo me pasas `user_text` y yo te devuelvo `response_text`”.

3) Qué es “modo streaming” (SSE/WebSocket) y qué cambiaría en Unity

Qué es streaming en este contexto

Ahora mismo el flujo “no streaming” sería:

1. Unity envía un `POST /chat` con `user_text`.
2. El servidor piensa, prepara toda la respuesta.
3. Devuelve de golpe `response_text`.
4. Tú pasas esa respuesta completa al TTS.

Con streaming, en lugar de esperar al texto entero, el servidor te va enviando trozos de la respuesta conforme los va generando (“tokens” o frases).^[452] ^[449]

Ejemplo mental:

- Sin streaming: recibes “Hola, soy Napoleón. Puedo contarte mi historia.” todo de golpe.
- Con streaming: primero te llega “Hola, ”, luego “soy Napoleón. ”, luego “Puedo contarte mi historia.”, etc.

La idea es que puedas empezar a TTS-ear con lo que ya tienes, sin esperar al final, y así reducir la latencia percibida 1-2 segundos.^[452]

SSE / WebSocket desde Unity

- SSE (Server-Sent Events): conexión HTTP que el servidor mantiene abierta y te envía eventos de texto poco a poco. Unity no tiene soporte nativo súper cómodo, pero se puede implementar con `UnityWebRequest` + coroutines o con una lib extra (o hacerlo desde un backend intermedio).

- **WebSocket**: conexión bidireccional más clásica. Hay clientes WebSocket para Unity (C#) que podrías usar para recibir los tokens en tiempo real.^[451] ^[452]

Qué cambios de código implicaría para ti

En tu diseño actual:

- `IChatService → RequestChat(userText, onComplete)`
Te da un `assistantText` único al final.

Para streaming necesitarías algo así como:

- `RequestChatStreaming(userText, onToken, onComplete)`

Donde:

- `onToken(string partialText)` se llama cada vez que llega un trozo nuevo.
- `onComplete(fullText, error)` se llama al final.

Y en TTS, en vez de esperar al texto final, podrías:

- ir acumulando tokens y mandar textos parciales a un TTS que soporte streaming,
o
- segmentar por frases y lanzar TTS cuando tengas una frase completa.

¿Necesitas esto para el MVP?

No. Y el propio proveedor dice “os recomiendo que integréis también `/chat/stream...`”, no que sea obligatorio.

Lo que te propuse antes de “diseñar la capa pensando en streaming futuro” significa:

- no casarte en todas partes con la idea de **una sola cadena** que llega de golpe;
- quizá tener una abstracción que internamente pueda:
 - trabajar en modo “bloque” (lo que haces ahora),
 - o en modo “incremental”.^[452]

Para este mes de desarrollo, yo haría:

- implementar solo `POST /chat` clásico,
- pero diseñar `IChatService` y tu controller de forma que más adelante puedas añadir un método `RequestChatStreaming` sin reventarlo todo.

4) Qué es un Bearer token estático

Es el mismo esquema que estás usando ya con OpenAI, pero contra su servicio.

- **Bearer token** = token de acceso que se envía en la cabecera `Authorization`:
 - `Authorization: Bearer <token>`^[453]

- **Estático** = un valor fijo que tú configuras para el MVP (no caduca a cada rato, no lo refrescas con OAuth).

En tu código:

```
request.SetRequestHeader("Authorization", $"Bearer {chatApiToken}");
```

Donde chatApiToken te lo dan ellos (algo tipo sk_live_xxx pero suyo).

A futuro mencionan OAuth2 si pasan a multi-cliente; eso significaría que el token cambiaría dinámicamente (login, refresh, etc.), pero para el MVP es simplemente:

- te dan un string,
- lo metes en config,
- lo mandas siempre igual.^[454] ^[453]

Cómo se traduce todo esto a tu diseño

- session_id:
 - sirve para que ellos mantengan contexto sin que tú tengas que mandar historial.
 - en Unity: un Guid por sesión/demostración, guardado en una variable global sencilla.
- RAG:
 - ellos enriquecen las respuestas con documentos del personaje.
 - tú no haces nada; solo te importa saber que el chat es “mejor” y está atado a un corpus.
- Streaming SSE/WebSocket:
 - es un “modo avanzado” para reducir latencia, recibiendo texto parcial.
 - para MVP, puedes ignorarlo y quedarte con /chat normal.
- Bearer token estático:
 - exactamente igual que tu token de OpenAI, pero del backend del cliente.
 - para MVP, se configura una vez y ya está.

Si quieres, el siguiente paso puede ser que te proponga la interfaz exacta de CharacterBackendChatClient (incluyendo session_id, emotion, intent_tags y token Bearer) para que solo tengas que pegar el script y empezar a integrarlo.

✱

si, prepara eso, y tambien el tema de los enums de emotions e intent tags y el hook con el animator controller. Y cualquier otra cosa que haya que modificar o preparar en nuestro código actual. El tema del streaming como comentas para el MVP no, pero si dejamos un mock hecho para una integración sencilla a futuro si despues de las pruebas queremos ganar algun segundo de latencia

Te propongo una ampliación de tu arquitectura actual que encaja con su contrato y deja hueco para el futuro streaming sin liar el MVP.

Voy a darte:

1. Nuevos tipos (enums `Emotion`, `IntentTag` y resultado de chat).
2. Implementación de `CharacterBackendChatClient` : `IChatService` con Bearer token, `session_id`, `emotion`, `intent_tags`.
3. Ajustes en tu `ConversationResult` y `ConversationController` para usar emoción/intents.
4. Hook mínimo hacia el `Animator` (o sistema de animaciones) del avatar.
5. Un mock de interfaz para streaming futuro, sin implementarlo aún.

1) Nuevos tipos: emociones e intents

Crea (o amplía) un fichero, por ejemplo `ConversationTypes.cs`:

```
using System;
using System.Collections.Generic;

namespace MVP.Conversation
{
    [Serializable]
    public enum CharacterEmotion
    {
        Neutral,
        Happy,
        Thinking,
        Concerned,
        Angry,
        Sad
        // Añade más si el rig lo soporta
    }

    [Serializable]
    public enum IntentTag
    {
```

```

        Unknown,
        Greeting,
        KnowledgeAnswer,
        Fallback,
        OutOfScope
        // Añade más si el backend los define
    }

    [Serializable]
    public class ChatServiceResult
    {
        public string responseText;
        public CharacterEmotion emotion;
        public List<IntentTag> intentTags;
        public string rawJson;          // Opcional, debug
    }
}

```

La idea es mapear las strings que ellos mandan ("happy", "greeting", etc.) a estos enums.

2) Nuevo IChatService y cliente del backend

Situ IChatService actual solo devuelve string, merece la pena ampliarlo un poco para poder pasar también emoción/intents. Puedes hacerlo no rompiendo el MVP actual manteniendo ambos métodos:

```

public interface IChatService
{
    // Método actual (bloque) para compatibilidad
    IEnumerator RequestChat(string userMessage, Action<string, string> onComplete);

    // Método enriquecido con emoción/intents
    IEnumerator RequestChatRich(string userMessage, Action<ChatServiceResult, string> onComplete);
}

```

Tu OpenAIChatClient puede implementar solo RequestChat por ahora, y el nuevo cliente del backend implementará ambos (el simple usando el rico por debajo).

CharacterBackendChatClient.cs

```

using System;
using System.Collections;
using System.Collections.Generic;
using System.Text;
using Newtonsoft.Json;
using UnityEngine;
using UnityEngine.Networking;

namespace MVP.Conversation
{
    public class CharacterBackendChatClient : MonoBehaviour, IChatService
    {

```

```

[Header("Backend Chat API")]
[SerializeField] private string baseUrl = "https://your-backend.example.com";
[SerializeField] private string chatPath = "/chat";
[SerializeField] private string bearerToken = "YOUR_BACKEND_TOKEN";
[SerializeField] private string characterId = "default_character";
[SerializeField] private string locale = "es-ES";

[Header("Debug")]
[SerializeField] private bool logRequests = false;
[SerializeField] private bool logResponses = false;

private string ChatUrl => baseUrl.TrimEnd('/') + chatPath;

// Compat: versión simple que devuelve solo texto
public IEnumerator RequestChat(string userMessage, Action<string, string> onComplete)
{
    ChatServiceResult richResult = null;
    string error = null;

    yield return RequestChatRich(userMessage, (result, err) =>
    {
        richResult = result;
        error = err;
    });

    if (!string.IsNullOrEmpty(error))
    {
        onComplete?.Invoke(null, error);
        yield break;
    }

    onComplete?.Invoke(richResult?.responseText, null);
}

public IEnumerator RequestChatRich(string userMessage, Action<ChatServiceResult, string> onComplete)
{
    var requestBody = new ChatRequestDto
    {
        session_id = SessionManager.CurrentSessionId,
        user_text = userMessage,
        character_id = characterId,
        metadata = new ChatRequestMetadata
        {
            locale = locale,
            user_id = null // si quieres pasarlo más adelante
        }
    };

    string json = JsonConvert.SerializeObject(requestBody);
    byte[] bodyRaw = Encoding.UTF8.GetBytes(json);

    using var request = new UnityWebRequest(ChatUrl, UnityWebRequest.kHttpVerbPOST);
    request.uploadHandler = new UploadHandlerRaw(bodyRaw);
    request.downloadHandler = new DownloadHandlerBuffer();
    request.SetRequestHeader("Content-Type", "application/json");
    request.SetRequestHeader("Authorization", $"Bearer {bearerToken}");
}

```

```

        if (logRequests)
            Debug.Log($"[CharacterBackendChatClient] POST {ChatUrl}\n{json}");

        yield return request.SendWebRequest();

        string raw = request.downloadHandler.text;

        if (request.result != UnityWebRequest.Result.Success)
        {
            onComplete?.Invoke(null, request.error + "\n" + raw);
            yield break;
        }

        if (logResponses)
            Debug.Log($"[CharacterBackendChatClient] RESPONSE {raw}");

        ChatResponseDto dto;
        try
        {
            dto = JsonConvert.DeserializeObject<ChatResponseDto>(raw);
        }
        catch (Exception e)
        {
            onComplete?.Invoke(null, "Error parseando JSON de backend chat: " +
                e.Message);
            return;
        }

        if (dto == null || string.IsNullOrEmpty(dto.response_text))
        {
            onComplete?.Invoke(null, "Respuesta vacía o inválida del backend chat");
            return;
        }

        var result = new ChatServiceResult
        {
            responseText = dto.response_text.Trim(),
            emotion = MapEmotion(dto.emotion),
            intentTags = MapIntentTags(dto.intent_tags),
            rawJson = raw
        };

        onComplete?.Invoke(result, null);
    }

    private CharacterEmotion MapEmotion(string emotion)
    {
        if (string.IsNullOrEmpty(emotion))
            return CharacterEmotion.Neutral;

        switch (emotion.ToLowerInvariant())
        {
            case "happy": return CharacterEmotion.Happy;
            case "thinking": return CharacterEmotion.Thinking;
            case "concerned": return CharacterEmotion.Concerned;
            case "angry": return CharacterEmotion.Angry;
        }
    }

```

```

        case "sad": return CharacterEmotion.Sad;
        default: return CharacterEmotion.Neutral;
    }
}

private List<IntentTag> MapIntentTags(List<string> tags)
{
    var list = new List<IntentTag>();
    if (tags == null || tags.Count == 0)
        return list;

    foreach (var t in tags)
    {
        if (string.IsNullOrEmpty(t))
            continue;

        switch (t.ToLowerInvariant())
        {
            case "greeting":
                list.Add(IntentTag.Greeting);
                break;
            case "knowledge_answer":
                list.Add(IntentTag.KnowledgeAnswer);
                break;
            case "fallback":
                list.Add(IntentTag.Fallback);
                break;
            case "out_of_scope":
                list.Add(IntentTag.OutOfScope);
                break;
            default:
                list.Add(IntentTag.Unknown);
                break;
        }
    }

    return list;
}

[Serializable]
private class ChatRequestMetadata
{
    public string locale;
    public string user_id;
}

[Serializable]
private class ChatRequestDto
{
    public string session_id;
    public string user_text;
    public string character_id;
    public ChatRequestMetadata metadata;
}

[Serializable]

```

```

        private class SourceDto
        {
            public string title;
            public float score;
        }

        [Serializable]
        private class ResponseMetadataDto
        {
            public int latency_ms;
            public string model;
            public int rag_hits;
        }

        [Serializable]
        private class ChatResponseDto
        {
            public string session_id;
            public string response_text;
            public string emotion;
            public List<string> intent_tags;
            public List<SourceDto> sources;
            public ResponseMetadataDto metadata;
        }
    }
}

```

3) Gestión de `session_id`

Un helper sencillo:

```

using System;

namespace MVP.Conversation
{
    public static class SessionManager
    {
        public static string CurrentSessionId { get; private set; }

        public static void StartNewSession()
        {
            CurrentSessionId = Guid.NewGuid().ToString();
        }
    }
}

```

Llamas a `SessionManager.StartNewSession()`:

- al arrancar la app,
- o cada vez que quieras “resetear” porque cambia de persona en la demo.

4) Ampliar ConversationResult y el controller

En ConversationStateTypes.cs, amplía:

```
[Serializable]
public class ConversationResult
{
    public string userText;
    public string assistantText;
    public CharacterEmotion emotion;
    ...

    public List<IntentTag> intentTags = new List<IntentTag>();
    ...

    public string error;
    public ConversationTiming timing = new ConversationTiming();
}
```

En tu OpenAIConversationController o uno nuevo para backend, usa RequestChatRich cuando el servicio lo implemente:

```
// En lugar de string assistantResponse = null;
ChatServiceResult chatResult = null;
string chatError = null;

// ...
yield return chatService.RequestChatRich(userText, (result, error) =>
{
    chatResult = result;
    chatError = error;
});

// ...
if (!string.IsNullOrEmpty(chatError))
{
    // como antes
    yield break;
}

result.assistantText = chatResult.responseText;
result.emotion = chatResult.emotion;
result.intentTags = chatResult.intentTags;

// aquí, antes de TTS, puedes notificar emoción/intents
emotionController?.ApplyEmotion(result.emotion, result.intentTags);
```

Situ IChatService no implementa RequestChatRich (por ejemplo, en la versión OpenAI simple), puedes hacer un fallback que derive RequestChat y ponga emotion = Neutral.

5) Hook con el Animator

Crea un pequeño componente, por ejemplo `AvatarEmotionController.cs`:

```
using System.Collections.Generic;
using UnityEngine;

namespace MVP.Conversation
{
    public class AvatarEmotionController : MonoBehaviour
    {
        [SerializeField] private Animator animator;

        // Nombres de parámetros en el Animator
        [Header("Animator Parameters")]
        [SerializeField] private string emotionParam = "Emotion";
        [SerializeField] private string intentParam = "Intent";

        private void Awake()
        {
            if (animator == null)
                animator = GetComponentInChildren<Animator>();
        }

        public void ApplyEmotion(CharacterEmotion emotion, List<IntentTag> intents)
        {
            if (animator == null)
                return;

            int emotionInt = EmotionToInt(emotion);
            animator.SetInteger(emotionParam, emotionInt);

            // intent principal: el primero de la lista, si existe
            var mainIntent = intents != null && intents.Count > 0 ? intents[0] : IntentNone;
            int intentInt = IntentToInt(mainIntent);
            animator.SetInteger(intentParam, intentInt);

            // aquí puedes también disparar triggers si quieres
            // por ejemplo: animator.SetTrigger("React");
        }

        private int EmotionToInt(CharacterEmotion emotion)
        {
            switch (emotion)
            {
                case CharacterEmotion.Happy: return 1;
                case CharacterEmotion.Thinking: return 2;
                case CharacterEmotion.Concerned: return 3;
                case CharacterEmotion.Angry: return 4;
                case CharacterEmotion.Sad: return 5;
                default: return 0; // Neutral
            }
        }

        private int IntentToInt(IntentTag intent)
        {

```

```

        switch (intent)
        {
            case IntentTag.Greeting: return 1;
            case IntentTag.KnowledgeAnswer: return 2;
            case IntentTag.Fallback: return 3;
            case IntentTag.OutOfScope: return 4;
            default: return 0;
        }
    }
}
}
}

```

En el Animator del avatar:

- añade parámetros `int Emotion` y `int Intent`.
- crea blend trees o states que reaccionen a esos ints (por ejemplo sub-state de idle basado en emoción).

En el ConversationController, referencia este componente:

```
[SerializeField] private AvatarEmotionController emotionController;
```

y llama a `emotionController.ApplyEmotion(...)` justo después de recibir la respuesta de chat, antes de lanzar TTS.

6) Mock de streaming para futuro

Para dejar preparado el “modo streaming” sin implementarlo ahora, puedes extender la interfaz con un método opcional:

```

public interface IChatService
{
    IEnumerator RequestChat(string userMessage, Action<string, string> onComplete);
    IEnumerator RequestChatRich(string userMessage, Action<ChatServiceResult, string> onComplete);

    // Streaming futuro (mock): por ahora, implementación por defecto que llama al
    IEnumerator RequestChatStreaming(
        string userMessage,
        Action<string> onPartialResponse,
        Action<string, string> onCompleted);
}

```

En CharacterBackendChatClient ahora mismo:

```

public IEnumerator RequestChatStreaming(
    string userMessage,
    Action<string> onPartialResponse,
    Action<string, string> onCompleted)
{
    // MOCK: usa la versión normal por ahora
    string fullText = null;
}

```

```

    string error = null;

    yield return RequestChat(userMessage, (text, err) =>
    {
        fullText = text;
        error = err;
    });

    if (!string.IsNullOrEmpty(error))
    {
        onCompleted?.Invoke(null, error);
        yield break;
    }

    // Simula streaming con un único "chunk"
    onPartialResponse?.Invoke(fullText);
    onCompleted?.Invoke(fullText, null);
}

```

Más adelante, cuando quieras integrar /chat/stream vía SSE/WebSocket:

- sustituyes esta implementación por la real,
- sin tocar el resto de la app, porque la firma del método ya estaba.

7) Resumen de cambios en tu proyecto

- Añadir CharacterEmotion, IntentTag, ChatServiceResult.
- Extender IChatService con RequestChatRich y RequestChatStreaming.
- Crear CharacterBackendChatClient con:
 - baseUrl, bearerToken, characterId, locale.
 - mapeo de emotions/intents.
- SessionManager para manejar session_id.
- Ampliar ConversationResult con emotion + intentTags.
- Modificar tu ConversationController para:
 - usar RequestChatRich cuando esté disponible,
 - notificar al AvatarEmotionController.
- Añadir AvatarEmotionController y dos ints en el Animator.
- Dejar RequestChatStreaming mockeado para futuro streaming.

Con esto, estás alineado con lo que el backend te ofrece, tu código sigue limpio y modular, y el futuro soporte de streaming será básicamente implementar un método sin cambiar firmas ni arquitectura.

dame el openaiconversationcontroller completo porque me has pasado bloques y prefiero que este completo

Aquí tienes un OpenAIConversationController completo, adaptado a lo que hemos ido definiendo: usa IChatService (con soporte para RequestChatRich), gestiona tiempos, actualiza la ConversationDebugView, llama al TTS y notifica emociones/intents al AvatarEmotionController.

Ajusta el namespace/nombres a los que uses tú.

```
using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.Events;
using TMPro;

namespace MVP.Conversation
{
    public class OpenAIConversationController : MonoBehaviour
    {
        [Header("Dependencies")]
        [SerializeField] private MonoBehaviour chatServiceBehaviour; // IChatService
        [SerializeField] private MonoBehaviour ttsServiceBehaviour; // ITtsService
        [SerializeField] private MicrophoneRecorder microphoneRecorder;
        [SerializeField] private AudioSource avatarAudioSource;
        [SerializeField] private ConversationDebugView debugView;
        [SerializeField] private AvatarEmotionController emotionController;

        [Header("Keyboard Debug")]
        [SerializeField] private bool useKeyboardDebugShortcut = true;
        [SerializeField] private KeyCode keyboardDebugKey = KeyCode.Space;

        [Header("Optional UI")]
        [SerializeField] private TMP_InputField debugInputField;
        [SerializeField] private UnityEngine.UI.Button debugSendButton;

        private IChatService chatService;
        private ITtsService ttsService;

        private bool isRunningConversation;
        private Coroutine currentConversationRoutine;

        private void Awake()
        {
            if (chatServiceBehaviour != null)
                chatService = chatServiceBehaviour as IChatService;

            if (ttsServiceBehaviour != null)
                ttsService = ttsServiceBehaviour as ITtsService;

            if (chatService == null)
```

```

        Debug.LogError("[OpenAIConversationController] Chat service behaviour");

    if (ttsService == null)
        Debug.LogError("[OpenAIConversationController] TTS service behaviour");

    if (avatarAudioSource == null)
        Debug.LogError("[OpenAIConversationController] Avatar AudioSource is null");

    if (microphoneRecorder == null)
        Debug.LogError("[OpenAIConversationController] MicrophoneRecorder is null");

    // Optional UI hookup
    if (debugSendButton != null && debugInputField != null)
    {
        debugSendButton.onClick.AddListener(() =>
        {
            if (!string.IsNullOrEmpty(debugInputField.text))
            {
                StartTextConversation(debugInputField.text);
            }
        });
    }
}

private void Update()
{
    if (!useKeyboardDebugShortcut)
        return;

    // Push-to-talk de prueba con la barra espaciadora (solo editor/desktop)
    if (Input.GetKeyDown(keyboardDebugKey))
    {
        if (!isRunningConversation)
            StartCoroutine(StartVoiceConversation());
    }
}

#region Public entry points

public void StartTextConversation(string userText)
{
    if (isRunningConversation)
        return;

    if (string.IsNullOrEmpty(userText))
        return;

    if (currentConversationRoutine != null)
        StopCoroutine(currentConversationRoutine);

    currentConversationRoutine = StartCoroutine(RunTextConversation(userText));
}

public IEnumerator StartVoiceConversation()
{
    if (isRunningConversation)

```

```

        yield break;

    if (currentConversationRoutine != null)
        StopCoroutine(currentConversationRoutine);

    currentConversationRoutine = StartCoroutine(RunVoiceConversation());
    yield return currentConversationRoutine;
}

#endregion

#region Core flows

private IEnumerator RunTextConversation(string userText)
{
    isRunningConversation = true;
    var result = new ConversationResult { userText = userText };
    result.timing.StartTotal();

    UpdateDebugState("User (text)", "Procesando entrada de texto...");

    // 1) Chat
    result.timing.StartChat();
    ChatServiceResult chatResult = null;
    string chatError = null;

    if (chatService == null)
    {
        chatError = "Chat service not configured.";
    }
    else
    {
        yield return chatService.RequestChatRich(userText, (r, err) =>
        {
            chatResult = r;
            chatError = err;
        });
    }

    result.timing.StopChat();

    if (!string.IsNullOrEmpty(chatError))
    {
        result.error = chatError;
        UpdateDebugState("Error", chatError);
        isRunningConversation = false;
        yield break;
    }

    result.assistantText = chatResult.responseText;
    result.emotion = chatResult.emotion;
    result.intentTags = chatResult.intentTags ?? new List<IntentTag>();

    // Hook de emoción / intent
    emotionController?.ApplyEmotion(result.emotion, result.intentTags);
}

```

```

// 2) TTS
yield return PlayAssistantReplyWithTts(result);

result.timing.StopTotal();
isRunningConversation = false;
UpdateDebugState("Idle", "Conversación completada.");
}

private IEnumerator RunVoiceConversation()
{
    isRunningConversation = true;
    var result = new ConversationResult();
    result.timing.StartTotal();

    // 1) Grabar audio
    UpdateDebugState("Recording", "Grabando voz del usuario...");
    result.timing.StartStt();

    AudioClip clip = null;
    string micError = null;

    yield return microphoneRecorder.RecordOnce((c, err) =>
    {
        clip = c;
        micError = err;
    });

    if (!string.IsNullOrEmpty(micError))
    {
        result.error = micError;
        UpdateDebugState("Error", micError);
        isRunningConversation = false;
        yield break;
    }

    if (clip == null)
    {
        result.error = "No se grabó audio desde el micrófono.";
        UpdateDebugState("Error", result.error);
        isRunningConversation = false;
        yield break;
    }

    result.timing.StopStt(); // aquí solo medimos la grabación; la transcrip

    // 2) STT → texto usuario
    UpdateDebugState("STT", "Transcribiendo audio del usuario...");

    string userText = null;
    string sttError = null;

    if (ttsService is ISttService sttService) // si tienes STT separado
    {
        yield return sttService.Transcribe(clip, (text, err) =>
        {
            userText = text;

```

```

        sttError = err;
    });
}
else
{
    sttError = "No STT service configured.";
}

if (!string.IsNullOrEmpty(sttError))
{
    result.error = sttError;
    UpdateDebugState("Error", sttError);
    isRunningConversation = false;
    yield break;
}

if (string.IsNullOrEmptyOrWhiteSpace(userText))
{
    result.error = "La transcripción de STT está vacía.";
    UpdateDebugState("Error", result.error);
    isRunningConversation = false;
    yield break;
}

result.userText = userText;

// 3) Chat
UpdateDebugState("Chat", "Consultando servicio de chat...");
result.timing.StartChat();

ChatServiceResult chatResult = null;
string chatError = null;

if (chatService == null)
{
    chatError = "Chat service not configured.";
}
else
{
    yield return chatService.RequestChatRich(userText, (r, err) =>
    {
        chatResult = r;
        chatError = err;
    });
}

result.timing.StopChat();

if (!string.IsNullOrEmpty(chatError))
{
    result.error = chatError;
    UpdateDebugState("Error", chatError);
    isRunningConversation = false;
    yield break;
}

```

```

        result.assistantText = chatResult.responseText;
        result.emotion = chatResult.emotion;
        result.intentTags = chatResult.intentTags ?? new List<IntentTag>();

        // Hook de emoción / intent
        emotionController?.ApplyEmotion(result.emotion, result.intentTags);

        // 4) TTS
        yield return PlayAssistantReplyWithTts(result);

        result.timing.StopTotal();
        isRunningConversation = false;
        UpdateDebugState("Idle", "Conversación completada.");
    }

private IEnumerator PlayAssistantReplyWithTts(ConversationResult result)
{
    if (ttsService == null)
    {
        result.error = "TTS service not configured.";
        UpdateDebugState("Error", result.error);
        yield break;
    }

    if (string.IsNullOrEmpty(result.assistantText))
    {
        result.error = "assistantText está vacío, no hay nada que sintetizar.";
        UpdateDebugState("Error", result.error);
        yield break;
    }

    UpdateDebugState("TTS", "Generando audio de la respuesta...");

    result.timing.StartTts();

    AudioClip ttsClip = null;
    string ttsError = null;

    yield return ttsService.SynthesizeToClip(result.assistantText, (clip, err) =>
    {
        ttsClip = clip;
        ttsError = err;
    });

    result.timing.StopTts();

    if (!string.IsNullOrEmpty(ttsError))
    {
        result.error = ttsError;
        UpdateDebugState("Error", ttsError);
        yield break;
    }

    if (ttsClip == null)
    {
        result.error = "TTS devolvió un clip nulo.";
    }
}

```

```

        UpdateDebugState("Error", result.error);
        yield break;
    }

    // Reproducción en el avatar
    UpdateDebugState("Speaking", "Reproduciendo audio en el avatar...");
    avatarAudioSource.Stop();
    avatarAudioSource.clip = ttsClip;
    avatarAudioSource.Play();

    // Esperar a que termine
    yield return new WaitForSeconds(ttsClip.length);
}

#endregion

#region Debug helpers

private void UpdateDebugState(string state, string message)
{
    if (debugView != null)
    {
        debugView.SetState(state);
        debugView.SetMessage(message);
    }
}

#endregion
}
}

```

Notas:

- `IChatService` es la interfaz que ya te propuse con `RequestChat/RequestChatRich`.
- `ITtsService` e `ISttService` son tus wrappers para OpenAI TTS y STT; la firma principal que usa aquí es:
 - `ITtsService.SynthesizeToClip(string text, Action<AudioClip,string> onComplete)`
 - `ISttService.Transcribe(AudioClip clip, Action<string,string> onComplete)`
- `ConversationResult` contiene tiempos y emoción/intents; `ConversationTiming` es tu struct de tiempos.
- El controller llama al `AvatarEmotionController` justo después de recibir la respuesta de chat.
- Para el MVP solo se usa el modo “bloque” (no streaming), pero la estructura permite que luego cambies la parte de chat/TTS por una versión streaming sin tocar demasiado este controlador.

Si me pegas la firma concreta de tus interfaces `ITtsService/ISttService` actuales, te lo ajusto al milímetro para que compile sin cambiar nada más.

1. <https://www.youtube.com/watch?v=rKDuYzS-D2E>
2. <https://docs.unity3d.com/6000.3/Documentation/Manual/com.unity.xr.meta-openxr.html>
3. <https://developers.meta.com/horizon/documentation/unity/unity-perf/>
4. <https://developers.meta.com/horizon/documentation/unity/unity-project-setup/>
5. <https://docs.unity3d.com/6000.3/Documentation/Manual/xr-meta-quest-packages.html>
6. <https://developers.meta.com/horizon/documentation/unity/audio-ovrlipsync-using-unity/>
7. <https://developers.meta.com/horizon/documentation/unity/meta-xr-audio-sdk-unity/>
8. <https://developers.meta.com/horizon/documentation/unity/meta-xr-audio-sdk-features/>
9. <https://developers.meta.com/horizon/documentation/unity/unity-best-practices-intro/>
10. <https://www.youtube.com/watch?v=olVERGcLq6w>
11. <https://docs.unity3d.com/6000.4/Documentation/Manual/urp/configure-for-better-performance.html>
12. https://www.youtube.com/watch?v=VlmyOJJ_E3c
13. <https://www.youtube.com/watch?v=2KUliN2R2pQ>
14. <https://developers.meta.com/horizon/downloads/package/meta-xr-audio-sdk/>
15. <https://www.youtube.com/watch?v=Ebrxcd0e8-4>
16. <https://developers.meta.com/horizon/documentation/unity/audio-ovrlipsync-unity/>
17. <https://www.youtube.com/watch?v=HCrLTtMCIg>
18. <https://steamcommunity.com/app/438100/discussions/1/6292161380824485775/?l=romanian>
19. <https://www.youtube.com/watch?v=Q4sPGTVyInY>
20. <https://steamcommunity.com/app/438100/discussions/1/6292161380824485775/?l=spanish>
21. <https://developers.meta.com/horizon/documentation/native/audio-ovrlipsync-using-native/>
22. <https://communityforums.atmeta.com/discussions/dev-dashboard/why-does-the-oculus-lipsync-for-unity-plugin-not-support-the-ios-app-store-/1099014>
23. <https://developers.meta.com/horizon/documentation/unreal/audio-ovrlipsync/>
24. <https://www.youtube.com/watch?v=RR9teRouRJs>
25. https://www.reddit.com/r/vrdev/comments/152pe9p/why_are_my_quest_builds_failing_to_go_above_72fps/
26. https://www.reddit.com/r/virtualreality/comments/1isluxn/how_to_unlock_72_fps_cap_in_unity_xr_interaction/
27. <https://www.youtube.com/watch?v=MXXNizHks3A>
28. <https://developers.meta.com/horizon/documentation/unity/unity-tutorial-basic-audio/>
29. <https://unity.com/topics/render-pipelines-strategy-for-2026>
30. https://www.reddit.com/r/Unity3D/comments/1rd5fra/tips_for_optimisation_in_unity_62_urp/
31. <https://developers.meta.com/horizon/documentation/unity/unity-project-setup/>
32. <https://developers.meta.com/horizon/documentation/unity/po-perf-opt-mobile/>
33. <https://developers.meta.com/horizon/documentation/unity/unity-perf/>

34. <https://docs.unity3d.com/6000.3/Documentation/Manual/xr-meta-quest-packages.html>
35. <https://developers.meta.com/horizon/documentation/unity/unity-tutorial-hello-vr/>
36. <https://developers.meta.com/horizon/documentation/unity/audio-ovrlipsync-unity/>
37. <https://developers.meta.com/horizon/documentation/unity/audio-ovrlipsync-using-unity/>
38. <https://github.com/avatarsdk/metaperson-oculus-unity-sample>
39. <https://github.com/Unity-Technologies/mr-example-meta-openxr>
40. <https://www.youtube.com/watch?v=cmlw6qNOLH4>
41. <https://www.youtube.com/watch?v=RR9teRouRJs>
42. <https://dev.to/taikonauten/part-1-setting-up-unity-2h83>
43. <https://developer.android.com/develop/xr/unity/setup>
44. <https://xrbootcamp.com/increasing-fps-in-unity-vr-games-course-review/>
45. <https://www.youtube.com/watch?v=KxU0wTfG3o0>
46. <https://developers.meta.com/horizon/documentation/unity/unity-project-setup/>
47. <https://developers.meta.com/horizon/documentation/unity/audio-ovrlipsync-unity/>
48. <https://docs.unity3d.com/Packages/com.unity.xr.meta-openxr@2.1/manual/get-started/link.html>
49. <https://docs.unity3d.com/Packages/com.unity.xr.meta-openxr@2.1/manual/get-started/project-settings.html>
50. <https://developers.meta.com/horizon/documentation/unity/unity-tutorial-hello-vr/>
51. <https://developers.meta.com/horizon/documentation/unity/audio-ovrlipsync-sample-unity/>
52. <https://developers.meta.com/horizon/documentation/unity/unity-openxr-settings-quest/>
53. <https://github.com/avatarsdk/metaperson-oculus-unity-sample>
54. <https://developers.meta.com/horizon/documentation/unity/unity-perf/>
55. <https://developers.meta.com/horizon/documentation/unity/audio-ovrlipsync-using-unity/>
56. https://www.reddit.com/r/MetaQuestVR/comments/1pnxkz6/unity_meta_xr_sdk_vr_mode_does_not_start_when/
57. https://www.youtube.com/watch?v=T6Y0rV_1GQc
58. <https://docs.unity3d.com/2022.3/Documentation/Manual/com.unity.xr.meta-openxr.html>
59. <https://www.youtube.com/watch?v=KxU0wTfG3o0>
60. <https://www.youtube.com/watch?v=Q4sPGTVyInY>
61. <https://www.youtube.com/watch?v=YkgJVlfAywk>
62. <https://www.youtube.com/watch?v=olVERGcLq6w>
63. <https://docs.unity3d.com/6000.4/Documentation/Manual/android-sdksetup.html>
64. <https://developers.meta.com/horizon/documentation/unity/unity-project-setup/>
65. <https://docs.unity3d.com/6000.3/Documentation/Manual/xr-meta-quest-build-profile.html>
66. <https://developers.meta.com/horizon/documentation/unity/unity-build/>
67. <https://docs.unity3d.com/Packages/com.unity.xr.meta-openxr@2.1/manual/get-started/link.html>

68. [https://connect-prd-cdn.unity.com/20221020/5064c5a0-e379-4c4e-bde9-75dfa2259840/Meta Quest Deployment Notes.pdf](https://connect-prd-cdn.unity.com/20221020/5064c5a0-e379-4c4e-bde9-75dfa2259840/Meta_Quest_Deployment_Notes.pdf)
69. <https://www.youtube.com/watch?v=23WUfV1U6mQ>
70. <https://www.youtube.com/shorts/jWPZjVmfKIs>
71. <https://www.delasign.com/blog/meta-quest-2-setup-run-unity-project/>
72. <https://unity.com/blog/porting-apps-games-over-android-xr-unity-6>
73. https://www.youtube.com/watch?v=N_oLhu6blm8
74. <https://docs.unity.cn/Packages/com.unity.xr.meta-openxr@2.4/manual/get-started/project-settings.html>
75. https://www.reddit.com/r/Unity3D/comments/1h39esu/downloading_android_build_support/
76. <https://communityforums.atmeta.com/discussions/dev-unity/oculus-quest-and-building-for-unity/772994/replies/772999>
77. <https://timcoster.com/2023/09/05/building-quest-2-vr-projects-with-unity/>
78. <https://docs.unity3d.com/6000.3/Documentation/Manual/xr-meta-quest-packages.html>
79. <https://docs.github.com/en/migrations/importing-source-code/using-the-command-line-to-import-source-code/adding-locally-hosted-code-to-github>
80. <https://www.freecodecamp.org/news/git-push-local-branch-to-remote-how-to-publish-a-new-branch-in-git/>
81. <https://www.youtube.com/watch?v=wyXLUJT7dLU>
82. <https://www.theserverside.com/video/How-to-use-the-git-remote-add-origin-command-to-push-remotely>
83. <https://www.theserverside.com/blog/Coffee-Talk-Java-News-Stories-and-Opinions/How-to-push-an-existing-project-to-GitHub>
84. <https://www.youtube.com/watch?v=ueQs5pQ8ZMM>
85. <https://docs.github.com/enterprise/2.7/user/articles/adding-an-existing-project-to-github-using-the-command-line>
86. <https://www.youtube.com/watch?v=rUyYCoCkidM>
87. <https://gist.github.com/alexpchin/102854243cd066f8b88e>
88. <https://docs.github.com/en/enterprise/2.13/user/articles/adding-an-existing-project-to-github-using-the-command-line>
89. <https://stackoverflow.com/questions/17291995/push-existing-project-into-github>
90. <https://stackoverflow.com/questions/5181845/git-push-existing-repo-to-a-new-and-different-remote-repo-server>
91. <https://www.digitalocean.com/community/tutorials/how-to-push-an-existing-project-to-github>
92. <https://gist.github.com/c0ldlimit/4089101>
93. <https://stackoverflow.com/questions/72433618/how-do-i-push-my-local-git-repository-to-a-branch-in-a-remote-github-repository>
94. <https://developers.meta.com/horizon/documentation/unity/audio-ovrlipsync-unity/>
95. <https://developers.meta.com/horizon/documentation/unity/audio-ovrlipsync-sample-unity/>
96. <https://developers.meta.com/horizon/documentation/unity/unity-perf/>

97. <https://github.com/avatarsdk/metaperson-oculus-unity-sample>
98. <https://docs.unity3d.com/Packages/com.unity.xr.meta-openxr@2.1/manual/get-started/link.html>
99. <https://www.ovrnews.com/oculus-releases-ovr lipsync-plugin-for-unity/>
100. https://www.reddit.com/r/oculusdev/comments/1ntrgnh/lowlatency_mac_audio_into_unity_on_meta_quest_3/
101. https://www.youtube.com/watch?v=T6Y0rV_1GQc
102. <https://www.youtube.com/watch?v=olVERGcLq6w>
103. <https://qa.fmod.com/t/quest-vr-best-practices-or-settings-for-minimal-latency-lag/18267>
104. <https://www.youtube.com/watch?v=LKYfRI26paA>
105. <https://communityforums.atmeta.com/tag/lipsync>
106. <https://github.com/hecomi/uLipSync>
107. https://communityforums.atmeta.com/discussions/Questions_Discussions/low-latency-mac-audio-into-unity-on-meta-quest-3/1346148
108. <https://www.youtube.com/watch?v=rKDuYzS-D2E>
109. <https://docs.unity3d.com/6000.4/Documentation/ScriptReference/Networking.DownloadHandlerAudioClip.html>
110. <https://docs.unity3d.com/2020.1/Documentation/Manual/com.unity.modules.unitywebrequestaudio.html>
111. <https://docs.perplexity.ai/api-reference/sonar-post>
112. <https://docs.perplexity.ai/docs/getting-started/quickstart>
113. https://rdr.io/cran/gptsstudio/man/create_completion_perplexity.html
114. <https://www.youtube.com/watch?v=zahrwl1125k>
115. https://docs.unity3d.com/Packages/com.unity.textmeshpro@1.3/api/TMPPro.TMP_InputField.html
116. <https://www.perplexity.ai/api-platform>
117. <https://www.youtube.com/watch?v=mLjylUyng2o>
118. <https://developers.openai.com/api/reference/typescript/resources/chat/subresources/completions/methods/create/>
119. <https://docs.unity.cn/es/2020.1/Manual/com.unity.modules.unitywebrequestaudio.html>
120. <https://docs.unity3d.com/2018.4/Documentation/Manual/com.unity.modules.unitywebrequestaudio.html>
121. <https://stackoverflow.com/questions/64963261/unity-android-load-local-mp3-or-wav-as-audioclip>
122. <https://www.youtube.com/watch?v=ac7PlcQs178>
123. <https://docs.unity.cn/es/2019.4/Manual/com.unity.modules.unitywebrequestaudio.html>
124. <https://www.youtube.com/watch?v=Ebrxcd0e8-4>
125. <https://docs.perplexity.ai/api-reference/sonar-post>
126. <https://docs.unity3d.com/6000.4/Documentation/ScriptReference/Networking.DownloadHandlerAudioClip.html>
127. <https://docs.perplexity.ai/docs/getting-started/quickstart>
128. <https://docs.unity3d.com/2020.1/Documentation/Manual/com.unity.modules.unitywebrequestaudio.html>

129. <https://www.youtube.com/watch?v=ac7PlcQs178>
130. https://docs.unity3d.com/Packages/com.unity.textmeshpro@1.3/api/TMPro.TMP_InputField.html
131. <https://developers.openai.com/api/docs/guides/text-to-speech>
132. <https://elevenlabs.io/text-to-speech-api>
133. <https://developers.openai.com/api/reference/typescript/resources/audio/subresources/speech/methods/create/>
134. <https://elevenlabs.io/blog/text-to-speech-api-up-to-40-faster-globally>
135. <https://docs.cloud.google.com/text-to-speech/docs/create-audio-text-client-libraries>
136. <https://docs.cloud.google.com/text-to-speech/docs/create-audio>
137. <https://docs.perplexity.ai/api-reference/sonar-post>
138. <https://developers.meta.com/horizon/documentation/unity/audio-ovrlipsync-unity/>
139. https://docs.openvino.ai/2026/model-server/ovms_docs_rest_api_s2t.html
140. <https://developers.openai.com/api/docs/guides/audio>
141. <https://www.youtube.com/watch?v=cDakxPKEKxk>
142. https://docs.videosdk.live/ai_agents/plugins/tts/openai
143. <https://stackoverflow.com/questions/69670438/google-cloud-text-to-speech-to-read-output-data-file-and-output-the-speech-in-me>
144. <https://smallest.ai/blog/tts-benchmark-2025-smallestai-vs-elevenlabs-report>
145. <https://github.com/mapluisch/OpenAI-Realtime-API-for-Unity>
146. <https://inworld.ai/resources/best-text-to-speech-apis>
147. <https://github.com/mapluisch/OpenAI-Text-To-Speech-for-Unity>
148. <https://openai.com/api/pricing/>
149. <https://speechactors.com/article/google-cloud-pricing-and-plans/>
150. <https://elevenlabs.io/pricing/api>
151. <https://costgoat.com/pricing/openai-tts>
152. <https://www.cekura.ai/blogs/elevenlabs-pricing>
153. <https://developers.openai.com/api/docs/guides/text-to-speech>
154. <https://elevenlabs.io/text-to-speech-api>
155. <https://www.camb.ai/blog-post/text-to-speech-price-comparison>
156. <https://magichour.ai/blog/elevenlabs-pricing>
157. <https://texttolab.com/pricing>
158. <https://www.g2.com/products/google-cloud-speech-to-text/pricing>
159. <https://deepgram.com/learn/best-text-to-speech-apis-2026>
160. <https://www.getapp.com/all-software/a/cloud-text-to-speech/>
161. <https://blog.lunabloomai.com/voice-ai-platforms/>
162. <https://audixa.ai/elevenlabs-text-to-speech-pricing-free-tier-2026/>

163. <https://audixa.ai/openai-text-to-speech-pricing/>

164. <https://www.finout.io/blog/openai-pricing-in-2026>

165. <https://platform.openai.com/docs/guides/text-to-speech>

166. <https://docs.unity3d.com/6000.3/Documentation/Manual/com.unity.xr.meta-openxr.html>

167. <https://docs.unity3d.com/6000.4/Documentation/ScriptReference/Networking.UnityWebRequestMultimedia.GetAudioClip.html>

168. <https://developers.openai.com/api/docs/quickstart>

169. <https://developers.openai.com/api/docs/guides/text-to-speech>

170. <https://platform.openai.com/docs/quickstart/account-setup>

171. <https://www.chaterimo.com/en/blog/how-to-openai-api-account/>

172. <https://www.stetcosmin.com/articles/openai-api-key-setup-signup-billing-security.html>

173. <https://www.clipboard-ai.com/blog/how-to-create-openai-api-key>

174. <https://tokenmix.ai/blog/openai-api-key-guide>

175. <https://www.apideck.com/blog/how-to-get-your-chatgpt-openai-api-key>

176. <https://www.datacamp.com/de/tutorial/how-to-use-the-openai-text-to-speech-api>

177. <https://platform.openai.com/docs/guides/text-to-speech>

178. <https://www.youtube.com/watch?v=LVzxjPO-4pE>

179. <https://www.youtube.com/watch?v=OB99E7Y1cMA>

180. <https://www.youtube.com/watch?v=J3y1dOpz9R4>

181. <https://vi-voice.com/api-guide>

182. <https://crazyrouter.com/en/blog/text-to-speech-api-comparison-2026>

183. <https://www.toriihq.com/articles/how-to-monitor-spending-openai>

184. <https://developers.openai.com/api/docs/guides/text-to-speech>

185. <https://developers.openai.com/api/docs/quickstart>

186. <https://help.openai.com/en/articles/8264644-how-can-i-set-up-prepaid-billing>

187. <https://tokenmix.ai/blog/openai-api-key-guide?lang=es>

188. <https://tokenmix.ai/blog/openai-api-billing-explained>

189. <https://developers.openai.com/api/docs/quickstart>

190. <https://www.clipboard-ai.com/blog/how-to-create-openai-api-key>

191. <https://juma.ai/blog/how-to-set-up-your-open-ai-billing>

192. <https://www.stetcosmin.com/articles/openai-api-key-setup-signup-billing-security.html>

193. <https://tokenmix.ai/blog/openai-api-key-guide>

194. <https://developers.openai.com/api/docs/guides/text-to-speech>

195. <https://platform.openai.com/docs/guides/text-to-speech>

196. <https://www.youtube.com/watch?v=BiegmTOIXis>

197. <https://www.ai.cc/blogs/how-to-buy-openai-api-credits-guide/>

198. <https://www.youtube.com/watch?v=mdkdDdLxc5g>

199. <https://www.linkedin.com/pulse/how-buy-openai-api-credits-globaltechcouncil-oqtrc>

200. <https://www.youtube.com/watch?v=J3y1dOpz9R4>

201. <https://fozzels.com/news/changes-with-openai-billing-from-post-pay-to-pre-pay.html>

202. <https://www.relay.app/blog/how-to-buy-credits-for-the-openai-platform-and-api>

203. <https://platform.openai.com/docs/guides/text-to-speech>

204. <https://developers.openai.com/api/docs/guides/text-to-speech>

205. <https://help.openai.com/en/articles/8264644-how-can-i-set-up-prepaid-billing>

206. <https://openai.com/api/pricing/>

207. <https://elevenlabs.io/text-to-speech-api>

208. <https://texttolab.com/openai>

209. <https://github.com/Haervwe/openedai-speech-bilingual-tts-1>

210. <https://chatterboxtts.com/docs/multilingual>

211. <https://tts.ai/tools/multilingual-tts/>

212. <https://cpaas-docs.digitalservices.wholesale.telekom.com/voice/getting-started/voice-and-languages/>

213. <https://github.com/matatonic/openedai-speech>

214. <https://artificial-intelligence-wiki.com/ai-development/ai-frameworks-and-libraries/openai-tts-api-guide/>

215. <https://qiita.com/n0bisuke/items/10c9a9aa02bfb62fd158>

216. <https://dev.to/czmilo/qwen3-tts-the-complete-2026-guide-to-open-source-voice-cloning-and-ai-speech-generation-1in6>

217. <https://www.youtube.com/watch?v=swkqLSW3dxk>

218. <https://www.rockapi.ru/docs/en/openai-texttospeech/>

219. <https://www.youtube.com/watch?v=UwyDNkw2Fis>

220. <https://community.openai.com/t/what-languages-does-the-new-tts-api-support/485153>

221. <https://developers.openai.com/api/docs/guides/speech-to-text>

222. <https://developers.openai.com/api/docs/guides/text-to-speech>

223. <https://openai.com/index/introducing-the-realtime-api/>

224. <https://developers.openai.com/api/docs/guides/audio>

225. <https://nexos.ai/blog/perplexity-vs-chatgpt/>

226. <https://www.conbersa.ai/learn/chatgpt-vs-perplexity>

227. <https://developers.openai.com/api/docs/quickstart>

228. <https://platform.openai.com/docs/guides/audio>

229. <https://www.latent.space/p/realtime-api>

230. <https://www.qwairy.co/blog/provider-citation-behavior-q3-2025>

231. <https://www.scribd.com/document/893206083/Audio-and-Speech-OpenAI-API>

232. https://docs.m5stack.com/en/stackflow/openai_api/speech_to_text

233. <https://docs.spring.io/spring-ai/reference/api/audio/speech/openai-speech.html>

234. <https://community.openai.com/t/chatcompletion-stream-to-tts/679722>

235. <https://www.datastudios.org/post/chatgpt-vs-perplexity-ai-full-report-and-comparison-comparison-of-features-capabilities-pricing>

236. https://docs.openvino.ai/2026/model-server/ovms_docs_rest_api_s2t.html

237. <https://developers.openai.com/api/reference/chat-completions/overview/>

238. <https://developers.openai.com/api/docs/guides/speech-to-text>

239. <https://developers.openai.com/api/docs/guides/text-to-speech>

240. <https://developers.openai.com/api/docs/guides/migrate-to-responses>

241. <https://developers.openai.com/api/docs/guides/completions>

242. https://docs.openvino.ai/2026/model-server/ovms_docs_rest_api_t2s.html

243. https://docs.openvino.ai/2025/model-server/ovms_docs_rest_api_chat.html

244. <https://developers.openai.com/api/docs/guides/audio>

245. <https://www.datacamp.com/de/tutorial/how-to-use-the-openai-text-to-speech-api>

246. https://www.linkedin.com/posts/andrejkaurin_openai-is-forcing-bunch-of-developers-to-activity-7396123535179587584-6yaV

247. <https://www.gladia.io/blog/best-open-source-speech-to-text-models>

248. <https://developers.openai.com/api/docs/models>

249. <https://developers.openai.com/api/reference/typescript/resources/chat/subresources/completions/methods/create/>

250. <https://docs.aimlapi.com/api-references/service-endpoints/complete-model-list>

251. https://www.youtube.com/watch?v=dg_TWk8Zfjk

252. <https://developers.openai.com/api/reference/chat-completions/overview/>

253. <https://developers.openai.com/api/docs/guides/text-to-speech>

254. <https://developers.openai.com/api/docs/guides/migrate-to-responses>

255. https://docs.unity3d.com/Packages/com.unity.textmeshpro@1.3/api/TMPro.TMP_InputField.html

256. <https://docs.unity3d.com/6000.4/Documentation/ScriptReference/Networking.UnityWebRequestMultimedia.GetAudioClip.html>

257. <https://developers.openai.com/api/docs/guides/completions>

258. <https://platform.openai.com/docs/guides/text-to-speech>

259. <https://developers.openai.com/api/docs/quickstart>

260. <https://developers.meta.com/horizon/documentation/unity/audio-ovrlipsync-unity/>

261. <https://developers.openai.com/api/docs/guides/speech-to-text>

262. <https://docs.unity3d.com/Packages/com.unity.nuget.newtonsoft-json@2.0/manual/index.html>

263. <https://docs.unity3d.com/Packages/com.unity.nuget.newtonsoft-json@3.2/manual/index.html>

264. <https://github.com/applejag/Newtonsoft.Json-for-Unity/wiki/Install-official-via-UPM>

265. <https://tigerforge.altervista.org/docs/unidb-1-0/installation/newtonsoft-json/>

266. <https://developers.openai.com/api/docs/guides/text-to-speech>

267. <https://platform.openai.com/docs/api-reference/chat>

268. <https://www.truefoundry.com/docs/ai-gateway/chat-completions-response-schema>

269. <https://dev.to/stormhub/openai-chat-completion-with-json-output-format-l5i>

270. https://www.youtube.com/watch?v=3H6xkl_EsvQ

271. <https://community.gamedev.tv/t/how-to-install-newtonsoft-json-in-unity-or-visual-studio-code/253560>

272. <https://www.educative.io/answers/configure-openais-chat-completions-api-for-json-responses>

273. <https://openupm.com/packages/com.newtonsoft.json/>

274. <https://developers.openai.com/api/reference/resources/chat/>

275. <https://github.com/applejag/Newtonsoft.Json-for-Unity>

276. <https://www.nuget.org/packages/newtonsoft.json/>

277. <https://www.newtonsoft.com>

278. <https://docs.unity3d.com/Packages/com.unity.nuget.newtonsoft-json@3.2/manual/index.html>

279. <https://www.newtonsoft.com>

280. <https://developers.openai.com/api/reference/chat-completions/overview/>

281. <https://developers.openai.com/api/reference/resources/chat/>

282. <https://community.openai.com/t/in-unity-encountering-jsonserializationexception-while-using-chatgpt-api/538828>

283. <https://dev.to/stormhub/openai-chat-completion-with-json-output-format-l5i>

284. <https://learn.microsoft.com/en-us/azure/foundry/openai/how-to/json-mode>

285. <https://www.educative.io/answers/configure-openais-chat-completions-api-for-json-responses>

286. <https://community.openai.com/t/how-do-i-use-the-new-json-mode/475890>

287. https://www.youtube.com/watch?v=3H6xkl_EsvQ

288. <https://stackoverflow.com/questions/78557135/how-do-i-print-each-element-in-an-openai-chatcompletion-response-as-json-on-a-server>

289. <https://openupm.com/packages/com.newtonsoft.json/>

290. <https://community.gamedev.tv/t/how-to-install-newtonsoft-json-in-unity-or-visual-studio-code/253560>

291. <https://github.com/applejag/Newtonsoft.Json-for-Unity/wiki/Install-official-via-UPM>

292. <https://www.nuget.org/packages/newtonsoft.json/>

293. <https://github.com/applejag/Newtonsoft.Json-for-Unity>

294. <https://docs.unity3d.com/Packages/com.unity.nuget.newtonsoft-json@3.2/manual/index.html>

295. <https://developers.openai.com/api/reference/chat-completions/overview/>

296. <https://developers.openai.com/api/reference/resources/chat/>

297. <https://www.newtonsoft.com>

298. <https://community.openai.com/t/in-unity-encountering-jsonserializationexception-while-using-chatgpt-api/538828>
299. <https://platform.openai.com/docs/guides/speech-to-text>
300. <https://developers.openai.com/api/docs/guides/text-to-speech>
301. <https://developers.openai.com/api/reference/chat-completions/overview/>
302. <https://tokenmix.ai/blog/openai-api-key-guide?lang=es>
303. <https://developers.openai.com/api/docs/guides/rbac>
304. <https://help.openai.com/es-es/articles/8867743-assign-api-key-permissions>
305. <https://learn.microsoft.com/en-us/answers/questions/1610875/can-i-have-multiple-keys-for-the-same-azure-openai>
306. <https://community.openai.com/t/translation-api-returns-incorrect-api-key-while-the-same-key-works-for-chat/975181>
307. <https://addepto.com/blog/what-is-an-openai-api-and-how-to-use-it/>
308. <https://community.openai.com/t/api-key-permissions-ui-model-request-scope-only-appears-when-model-capabilities-is-set-to-request/1374480>
309. <https://towardsdatascience.com/use-openai-whisper-for-automated-transcriptions/>
310. <https://developers.openai.com/api/docs/guides/your-data>
311. <https://www.toriihq.com/articles/how-to-add-user-openai>
312. <https://community.openai.com/t/how-can-i-build-and-use-my-own-openai-api-key-for-my-project/1365719>
313. <https://community.openai.com/t/how-to-manage-multiple-openai-api-keys-in-an-application/607940>
314. <https://github.com/Aider-AI/aider/issues/3125>
315. <https://blog.laozhang.ai/en/posts/openai-api-key-organization-id>
316. <https://platform.openai.com/docs/guides/speech-to-text>
317. <https://developers.openai.com/api/docs/guides/speech-to-text>
318. <https://developers.meta.com/horizon/documentation/unity/audio-ovrlipsync-unity/>
319. <https://github.com/avatarsdk/metaperson-oculus-unity-sample>
320. <https://developers.meta.com/horizon/documentation/unity/audio-ovrlipsync-using-unity/>
321. <https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Microphone.Start.html>
322. <https://developers.meta.com/horizon/documentation/unity/audio-ovrlipsync-precomputed-unity/>
323. https://github.com/AntonSHBK/speech_to_text_service
324. <https://gist.github.com/R-WebsterNoble/70614880b0d3940d3b2b741fbbb311a2>
325. <https://stackoverflow.com/questions/77929688/unsure-how-to-format-multipart-form-data-for-open-ai-st-api-call-on-esp32>
326. <https://github.com/reservamos/speech-to-text-demo>
327. <https://www.youtube.com/watch?v=-ADNOjEs1pA>
328. <https://codingplanx.ai/docs/gpt-4o-transcribe>

329. <https://www.1ft-seabass.jp/memo/2023/09/17/unity-with-whisper-api/>

330. <https://github.com/reservamos/speech-to-text-demo/blob/main/README.md>

331. <https://developers.openai.com/api/docs/guides/text-to-speech>

332. <https://platform.openai.com/docs/guides/speech-to-text>

333. <https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Microphone.Start.html>

334. <https://developers.meta.com/horizon/documentation/unity/audio-ovrlipsync-unity/>

335. https://docs.unity3d.com/Packages/com.unity.ugui@3.0/api/TMPPro.TMP_InputField.html

336. <https://developers.meta.com/horizon/documentation/unity/audio-ovrlipsync-using-unity/>

337. <https://docs.unity3d.com/6000.3/Documentation/Manual/android-permissions-in-unity.html>

338. <https://developers.openai.com/api/reference/chat-completions/overview/>

339. <https://platform.openai.com/docs/guides/speech-to-text>

340. <https://developers.openai.com/api/docs/guides/text-to-speech>

341. <https://www.youtube.com/watch?v=ac7PlcQs178>

342. <https://docs.unity3d.com/Packages/com.unity.nuget.newtonsoft-json@3.2/manual/index.html>

343. <https://www.youtube.com/watch?v=rKDyZs-D2E>

344. https://docs.unity3d.com/Packages/com.unity.ugui@2.0/api/TMPPro.TMP_InputField.html

345. <https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Microphone.Start.html>

346. <https://github.com/avatarsdk/metaperson-oculus-unity-sample>

347. <https://docs.unity.cn/es/2021.1/Manual/android-RequestingPermissions.html>

348. <https://www.youtube.com/watch?v=XMKDShd5dNE>

349. https://docs.unity.cn/Packages/com.unity.textmeshpro@1.4/api/TMPPro.TMP_InputField.html

350. <https://stackoverflow.com/questions/36270253/request-permission-for-microphone-on-android-m>

351. <https://christinacreatessgames.itch.io/input-fields-the-unity-ugui-guide>

352. <https://www.youtube.com/watch?v=zahrwl1125k>

353. https://docs.avatarsdk.com/unity-plugin/3.0.1/using_oculus_lipsync_with_avatar_sdk.html

354. https://github.com/k1llyOu/RustEdit-Unity/blob/master/Library/PackageCache/com.unity.textmeshpro@2.0.1/Scripts/Runtime/TMP_InputField.cs

355. <https://docs.unity.com/en-us/vivox-core/developer-guide/android/android-sdk-permission-requirements>

356. <https://creators.vrchat.com/avatars/creating-your-first-avatar/>

357. <https://developers.meta.com/horizon/documentation/unity/audio-ovrlipsync-unity/>

358. <https://developers.meta.com/horizon/documentation/unity/audio-ovrlipsync-using-unity/>

359. <https://wifitalents.com/best/vrm-software/>

360. <https://yelzkizi.org/make-a-3d-vtuber-avatar/>

361. https://www.youtube.com/watch?v=to_aligBkpU

362. <https://sketchfab.com/3d-models/ready-player-me-male-avatar-ca294f737d0b4293bb29bfcd8a0a27dd>

363. <https://www.toolify.ai/gpts/ultimate-guide-uploading-vrchat-avatars-for-pc-quest-vroid-66121>

364. <https://www.youtube.com/watch?v=mFauSWvjKl4>

365. <https://github.com/met4citizen/talkinghead>

366. <https://www.youtube.com/watch?v=oQm6bdnXzGU>

367. <https://www.freelancer.com/job-search/vrchat/>

368. <https://aiaustin.wordpress.com/2023/03/21/ready-player-me-avatars-in-second-life-and-opensim/>

369. <https://developers.meta.com/horizon/documentation/native/audio-ovrlipsync-native/>

370. <https://threedium.io/create/3d-models/avatars/vtuber>

371. <https://vroid.com/en/studio>

372. https://vrm.dev/en/univrm/blendshape/univrm_blendshape/

373. <https://gitnux.org/best/vrm-software/>

374. <https://steamcommunity.com/app/438100/discussions/1/1696040635916280112/?l=russian>

375. <https://developers.meta.com/horizon/documentation/unity/audio-ovrlipsync-unity/>

376. <https://developers.meta.com/horizon/documentation/unity/audio-ovrlipsync-using-unity/>

377. <https://www.youtube.com/watch?v=Qyrz4JoMP28>

378. https://www.reddit.com/r/VRoid/comments/1r8mOnf/help_with_making_vroid_model_in_vrchat_lipsync_in/

379. https://www.reddit.com/r/VRchat/comments/i1yvzw/help_visemes_work_in_blender_and_unity_but_dont/

380. https://www.reddit.com/r/blender/comments/zlq3hj/facial_expressions_and_lip_syncing_lost_when/

381. <https://streamoji.com/blog/ready-player-me-alternatives-2026>

382. <https://creators.vrchat.com/avatars/creating-your-first-avatar/>

383. <https://www.youtube.com/watch?v=iGKoEE1fg6U>

384. <https://lobehub.com/zh/skills/logos-liber-atlas-agent-teams-vroid>

385. <https://threedium.io/create/3d-models/avatars/vtuber>

386. <https://www.youtube.com/watch?v=QasZOBgKJQ0>

387. <https://vroid.pixiv.help/hc/en-us/articles/41131955805977-How-facial-expressions-are-handled-when-exporting-VRChat-avatars-as-VRM>

388. https://www.reddit.com/r/VRchat/comments/1n8yh8t/help_avatars_mouth_wont_move/

389. <https://sketchfab.com/tags/blendshape>

390. <https://github.com/avatarsdk/metaperson-oculus-unity-sample>

391. <https://sketchfab.com/3d-models/realistic-avatar-in-hq-with-blendshapes-88b42ee84e5749138f1b0507e6b9e5a9>

392. <https://www.youtube.com/watch?v=olVERGcLq6w>

393. <https://sketchfab.com/tags/visemes>

394. <https://sketchfab.com/tags/blendshapes>

395. <https://www.youtube.com/watch?v=a5zQBx2KF2Y>

396. <https://sketchfab.com/3d-models/human-3d-model-free-fb776f93e0724143aff0683ce7c3cc98>

397. <https://www.colariver.com/post/maya-unity-facial-rig-blendshape-workflow>

398. <https://github.com/avatarsdk/metaperson-oculus-ue-sample>

399. <https://sketchfab.com/tags/stylized-human>

400. <https://www.youtube.com/watch?v=IFZIJYrG1Y>

401. <https://sketchfab.com/tags/humanos>

402. https://www.reddit.com/r/Unity3D/comments/r0gcfe/i_made_a_bunch_of_free_3d_head_models_they_have/

403. <https://www.youtube.com/watch?v=5salbIRbmNc>

404. <https://developers.openai.com/api/reference/resources/chat/>

405. <https://learn.microsoft.com/en-us/azure/foundry/openai/how-to/chatgpt>

406. <https://developers.openai.com/api/docs/guides/conversation-state>

407. <https://developers.google.com/assistant/conversation-design/create-a-persona>

408. <https://learn.microsoft.com/en-us/azure/ai-foundry/openai/concepts/advanced-prompt-engineering?view=foundationry-classic>

409. <https://www.rootstrap.com/blog/how-to-write-system-instructions-for-openais-gpt-4-chat-api>

410. <https://community.openai.com/t/how-to-preserve-the-context-session-of-a-conversation-with-the-api/324986>

411. <https://www.personal.ai/pai-academy/chat-prompting-and-carrying-context-with-a-persona>

412. <https://developers.liveperson.com/messaging-interactions-api-methods-conversations.html>

413. <https://community.openai.com/t/best-practices-for-cost-efficient-high-quality-context-management-in-long-ai-chats/1373996>

414. <https://platform.openai.com/docs/api-reference/chat>

415. <https://campus.datacamp.com/courses/working-with-the-openai-api/building-conversations-with-the-openai-api?ex=1>

416. <https://docs.langdock.com/api-endpoints/completion/openai>

417. <https://www.baeldung.com/cs/chatgpt-api-roles>

418. <https://community.openai.com/t/how-to-pass-conversation-history-back-to-the-api/697083>

419. <https://developers.openai.com/api/reference/resources/chat/>

420. <https://developers.openai.com/api/docs/guides/conversation-state>

421. <https://learn.microsoft.com/en-us/azure/foundry/openai/how-to/chatgpt>

422. <https://learn.microsoft.com/en-us/azure/ai-foundry/openai/concepts/advanced-prompt-engineering?view=foundationry-classic>

423. <https://developers.liveperson.com/messaging-interactions-api-methods-conversations.html>

424. <https://community.openai.com/t/access-my-new-custom-gpt-thru-the-api-is-it-possible-if-not-please-vote-for-it/756300>

425. <https://developers.openai.com/api/docs/assistants/migration>

426. <https://help.openai.com/en/articles/8673914-gpts-vs-assistants>

427. <https://developers.openai.com/api/docs/models>

428. <https://community.openai.com/t/does-the-custom-gpts-have-a-future/1371528>

429. <https://www.metactio.com/blogs/what-is-the-openai-api-a-comprehensive-guide-for-app-developers>

430. <https://www.eesel.ai/blog/custom-gpt-api>

431. <https://psycray.com/openapi-chat-gpt-demystifying-openais-gpt-chat-api/>

432. <https://baresquare.com/blog/openai-assistants-vs-custom-gpts>

433. <https://stackoverflow.com/questions/77478768/how-to-enable-openai-custom-gpt-to-access-an-api>

434. https://www.reddit.com/r/OpenAI/comments/1i1az8i/why_is_chat_gpt_not_aware_of_open_ai_api/

435. <https://www.youtube.com/watch?v=C6PXNmtGJTW>

436. <https://help.openai.com/en/articles/20001051-retiring-gpt-4o-and-other-chatgpt-models>

437. <https://www.youtube.com/watch?v=6Lab-pxoZ9c>

438. <https://community.openai.com/t/use-chatgpt-for-openai-documentation/548377>

439. <https://developers.openai.com/api/docs/guides/conversation-state>

440. <https://developers.openai.com/api/reference/resources/chat/>

441. <https://developers.liveperson.com/messaging-interactions-api-methods-conversations.html>

442. <https://www.personal.ai/pai-academy/chat-prompting-and-carrying-context-with-a-persona>

443. <https://learn.microsoft.com/en-us/azure/foundry/openai/how-to/chatgpt>

444. <https://developers.google.com/assistant/conversation-design/create-a-persona>

445. <https://www.metactio.com/blogs/what-is-the-openai-api-a-comprehensive-guide-for-app-developers>

446. <https://baresquare.com/blog/openai-assistants-vs-custom-gpts>

447. <https://developers.meta.com/horizon/documentation/unity/audio-ovrlipsync-unity/>

448. <https://developers.meta.com/horizon/documentation/unity/audio-ovrlipsync-using-unity/>

449. <https://developers.openai.com/api/docs/guides/conversation-state>

450. <https://community.openai.com/t/how-to-preserve-the-context-session-of-a-conversation-with-the-api/324986>

451. <https://developers.liveperson.com/messaging-interactions-api-methods-conversations.html>

452. <https://learn.microsoft.com/en-us/azure/foundry/openai/how-to/chatgpt>

453. <https://www.metactio.com/blogs/what-is-the-openai-api-a-comprehensive-guide-for-app-developers>

454. <https://help.openai.com/en/articles/20001051-retiring-gpt-4o-and-other-chatgpt-models>

455. https://docs.unity3d.com/Packages/com.unity.textmeshpro@1.3/api/TMPro.TMP_InputField.html

456. <https://github.com/imclab/openai.unity>

457. <https://github.com/mapluisch/OpenAI-Realtime-API-for-Unity>

458. <https://developers.openai.com/api/docs/guides/migrate-to-responses>

459. <https://inworld.ai/resources/best-voice-ai-tts-apis-for-real-time-voice-agents-2026-benchmarks>

460. <https://www.unitycatalog.io/blogs/unity-catalog-and-openai>

461. <https://developers.openai.com/api/docs/guides/text-to-speech>

462. https://docs.unity.cn/Packages/com.unity.ugui@2.0/api/TMPro.TMP_InputField.html

- 463. <https://stackoverflow.com/questions/69088276/tmp-inputfield-in-inspector-can-add-method-calls-for-things-like-onvaluechange>
- 464. <https://www.youtube.com/watch?v=ac7PlcQs178>
- 465. <https://getstream.io/video/sdk/unity/tutorial/audio-room/>
- 466. <https://github.com/srcnalt/openai-unity>
- 467. <https://github.com/mapluisch/OpenAI-Text-To-Speech-for-Unity>
- 468. <https://platform.openai.com/docs/api-reference/chat>
- 469. <https://dev.to/1geek/unity-openai-vision-and-voice-3930>